```
================================================================================
DEFMAN-2026: IEEE STD 1044-2009 ANOMALY CLASSIFICATION & DORA TELEMETRY DOCUMENT
================================================================================
Project:      DefMan (Defect Manager) -- 2026 Edition
Standards:    IEEE Std 1044-2009 (Software Anomaly Classification)
              DORA / Flow Engineering Metrics (MTTR, Lead Time, Escape & Regression)
Database:     MySQL 8.0+ / MariaDB 10.6+ (3NF InnoDB + Legacy 2004 Views)
Backend:      PHP 8.3+ Strict-Typed PDO REST API
Frontend:     React 19 + TypeScript + PixiJS v8 (WebGPU / WebGL2)
================================================================================

================================================================================
SECTION 1. EXECUTIVE SUMMARY & ARCHITECTURAL TRANSFORMATION
================================================================================

1.1 Rationale: Replacing 1992 IEEE Fault & Waterfall Phase-Cost Multipliers
--------------------------------------------------------------------------------
The original DefMan 2004 analytical model was built around the 1992 draft of
IEEE Std 1044 and 1980s waterfall phase-cost multiplier heuristics:
  - It assumed a fixed hourly rate (e.g., 30 EUR/hour) multiplied by arbitrary
    phase-lag cost multipliers (‘fixeffort‘ -> ‘fixcost‘ -> ‘costofdefect‘).
  - It conflated observed failure symptoms with underlying engineering root
    causes in a single flat 1992 fault string (‘Logic‘, ‘Computational‘,
    ‘Data handling‘, ‘Document quality‘, ‘Enhancement‘, ‘Caused by a prev. Fix‘).
  - It recorded only coarse calendar dates (‘DATE‘) rather than high-resolution
    timestamps (‘DATETIME‘), making sub-day Triage Lead Time and Mean Time To
    Recovery (MTTR) impossible to compute accurately.
  - It lacked deployment environment tracking (‘DEV‘, ‘CI‘, ‘STAGING‘, ‘PROD‘),
    commit/build traceability (‘introducing_ref‘, ‘resolution_ref‘), and explicit
    regression flags (‘is_regression‘).

DefMan 2026 replaces these obsolete waterfall cost multipliers with:
  1. IEEE Std 1044-2009 Anomaly Classification (separating observed Failure
     Symptom Category from Engineering Root Cause Type).
  2. DORA & Flow Engineering Telemetry computed directly from real timestamps,
     environments, regression flags, and actual engineering effort hours.

--------------------------------------------------------------------------------
1.2 IEEE Std 1044-2009 Dual-Axis Anomaly Classification
--------------------------------------------------------------------------------
  A. Observed Symptom Category (‘symptom_category‘):
     - Application Crash / Hang
     - Data Corruption / Loss
     - Incorrect Calculation / Output
     - Authentication / Security Failure
     - UI / Rendering Anomaly
     - Performance / Latency Degradation
     - API / Integration Failure
     - Other Symptom

  B. Engineering Root Cause Type (‘root_cause_type‘ / ‘IeeeFaultType‘):
     - Code / Logic
     - Computational
     - Interface / Integration
     - Data / Schema
     - Concurrency / Timing
     - Configuration / Environment
     - Security / Access Control
     - Requirements / Specification
     - Architecture / Design
     - Documentation
     - Regression
     - Other

--------------------------------------------------------------------------------
1.3 DORA & Flow Engineering Metrics (AnalysisRepository.php)
```

```
--------------------------------------------------------------------------------
All synthetic EUR phase-cost multipliers have been replaced in
‘AnalysisRepository::getDoraTelemetrySummary()‘ and ‘getAnalyzerQueue()‘ with
real SQL aggregations over ‘defect_reports‘ and ‘defect_analyses‘:

  1. Mean Time To Resolution (MTTR):
     - Formula: AVG(TIMESTAMPDIFF(SECOND, dr.reported_at, da.resolved_at) / 3600.0)
     - Measures actual elapsed hours from defect report creation (‘reported_at‘)
       to verified engineering resolution (‘resolved_at‘).

  2. Triage Lead Time:
     - Formula: AVG(TIMESTAMPDIFF(SECOND, dr.reported_at, da.triaged_at) / 3600.0)
     - Measures elapsed hours from defect submission (‘reported_at‘) to developer
       triage and root-cause classification (‘triaged_at‘).

  3. Production Escape Rate:
     - Formula: (COUNT(detected_environment = ’PROD’) / COUNT(*)) * 100
     - Tracks the percentage of anomalies escaping pre-production quality gates
       (‘DEV‘, ‘CI‘, ‘STAGING‘) into ‘PROD‘, alongside per-environment counts.

  4. Regression Rate:
     - Formula: (COUNT(is_regression = TRUE) / COUNT(*)) * 100
     - Tracks the ratio of anomalies introduced by a prior fix, refactor, or
       release (‘is_regression = 1‘), linked via ‘introducing_ref‘.

  5. Actual Effort Hours by IEEE 1044-2009 Root Cause:
     - Aggregates ‘SUM(actual_effort_hours)‘, ‘AVG(actual_effort_hours)‘,
       ‘AVG(mttr_hours)‘, ‘prod_escape_count‘, and ‘regression_count‘ grouped
       by ‘root_cause_type‘.

--------------------------------------------------------------------------------
1.4 Schema & Legacy Mapping Matrix
--------------------------------------------------------------------------------
1992 / 2004 Construct            -> 2026 IEEE 1044-2009 & DORA Construct
--------------------------------------------------------------------------------
analyse.classification           -> defect_analyses.root_cause_type (VARCHAR(100))
                                    + defect_analyses.symptom_category (VARCHAR(100))
                                    + fault_classifications.code (FK)
raport.Environment (free text)   -> defect_reports.environment (TEXT)
                                    + detected_environment ENUM(’DEV’,’CI’,’STAGING’,’PROD’)
"Caused by a prev. Fix"          -> is_regression BOOLEAN NOT NULL DEFAULT FALSE
                                    + root_cause_type = ’Regression’
(Untracked build/commit origin)  -> defect_analyses.introducing_ref (VARCHAR(128))
(Untracked fix commit/PR)        -> defect_analyses.resolution_ref (VARCHAR(128))
analyse.fixeffort                -> defect_analyses.actual_effort_hours (DECIMAL(8,2))
analyse.fixcost / costofdefect   -> Deprecated (nullable legacy columns; replaced
                                    by DORA MTTR & Root Cause Effort Breakdown)
analyse.assigned (DATE)          -> defect_analyses.triaged_at (DATETIME)
analyse.resocomplete (DATE)      -> defect_analyses.resolved_at (DATETIME)

--------------------------------------------------------------------------------
1.5 Backward Compatibility Preservation (‘03_legacy_views.sql‘)
--------------------------------------------------------------------------------
Legacy 2004 SQL queries against ‘product‘, ‘raport‘, and ‘analyse‘ continue to
resolve without error:
  - ‘COALESCE(da.root_cause_type, fc.code) AS classification‘
  - ‘COALESCE(da.assigned_on, DATE(da.triaged_at)) AS assigned‘
  - ‘COALESCE(da.fix_completed_on, DATE(da.resolved_at)) AS fixcomplete‘
  - ‘COALESCE(da.actual_effort_hours, da.fix_effort) AS fixeffort‘
  - ‘COALESCE(da.resolution_completed_on, DATE(da.resolved_at)) AS resocomplete‘
================================================================================

================================================================================
SECTION 2.1 Target 3NF Schema (database/01_schema_3nf.sql)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\database\01_schema_3nf.sql
================================================================================
```

```
-- ============================================================================
-- File: 01_schema_3nf.sql
-- Description: Target 3NF Relational Schema for DefMan-2026 (MySQL 8.0+ / MariaDB 10.6+)
--              Includes IEEE Std 1044-2009 Anomaly Classification & DORA Telemetry
-- Engine: InnoDB | Charset: utf8mb4 | Collation: utf8mb4_unicode_ci
-- ============================================================================

SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;
SET SQL_MODE = 'STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,
    NO_ENGINE_SUBSTITUTION';

-- ----------------------------------------------------------------------------
-- 1. Table: `products` (standardized from `product`)
-- ----------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS `products` (
  `id` SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(100) NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_products_name` (`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ----------------------------------------------------------------------------
-- 2. Table: `companies` (extracted from `users.company` -- 3NF)
-- ----------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS `companies` (
  `id` SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(100) NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_companies_name` (`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ----------------------------------------------------------------------------
-- 3. Table: `users` (normalized: `company_id` FK; removed `logged` & reset cols)
-- User Levels:
--   0 = Applying (Pending Admin Approval)
--   1 = Public   (Unauthenticated / Public reporter)
--   2 = Customer (Authenticated defect reporter)
--   3 = Analyzer (Developer / QA triage & IEEE 1044-2009 / DORA telemetry)
--   4 = Admin    (Full system access, user approval & product CRUD)
-- ----------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS `users` (
  `id` SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `login` VARCHAR(64) NOT NULL,
  `password` VARCHAR(255) NOT NULL COMMENT 'Argon2id password hash (PASSWORD_ARGON2ID)',
  `email` VARCHAR(255) NOT NULL,
  `userlevel` TINYINT UNSIGNED NOT NULL DEFAULT 0 COMMENT '0=Applying..4=Admin',
  `company_id` SMALLINT UNSIGNED NULL DEFAULT NULL,
  `phone` VARCHAR(50) NULL DEFAULT NULL,
  `name` VARCHAR(100) NULL DEFAULT NULL,
  `activate` TINYINT(1) NOT NULL DEFAULT 0 COMMENT '0=Pending/Inactive, 1=Active',
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_users_login` (`login`),
  UNIQUE KEY `uk_users_email` (`email`),
  KEY `idx_users_company_id` (`company_id`),
  KEY `idx_users_level_activate` (`userlevel`, `activate`),
  CONSTRAINT `fk_users_company`
    FOREIGN KEY (`company_id`) REFERENCES `companies` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- ------------------------------------------------------------------------
-- 4. Table: ‘user_sessions‘ (active rows derive user online status)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS ‘user_sessions‘ (
  ‘id‘ BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘user_id‘ SMALLINT UNSIGNED NOT NULL,
  ‘token_hash‘ CHAR(64) NOT NULL COMMENT ’SHA-256 hash of 256-bit random_bytes token’,
  ‘ip_address‘ VARCHAR(45) NULL DEFAULT NULL,
  ‘user_agent‘ VARCHAR(255) NULL DEFAULT NULL,
  ‘expires_at‘ DATETIME NOT NULL,
  ‘created_at‘ TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_user_sessions_token_hash‘ (‘token_hash‘),
  KEY ‘idx_user_sessions_user_expires‘ (‘user_id‘, ‘expires_at‘),
  KEY ‘idx_user_sessions_expires_at‘ (‘expires_at‘),
  CONSTRAINT ‘fk_user_sessions_user‘
    FOREIGN KEY (‘user_id‘) REFERENCES ‘users‘ (‘id‘)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 5. Table: ‘password_resets‘ (decoupled from ‘users‘ -- 3NF)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS ‘password_resets‘ (
  ‘id‘ INT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘user_id‘ SMALLINT UNSIGNED NOT NULL,
  ‘token_hash‘ CHAR(64) NOT NULL COMMENT ’SHA-256 hash of recovery token’,
  ‘expires_at‘ DATETIME NOT NULL,
  ‘created_at‘ TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_password_resets_token_hash‘ (‘token_hash‘),
  KEY ‘idx_password_resets_user_expires‘ (‘user_id‘, ‘expires_at‘),
  CONSTRAINT ‘fk_password_resets_user‘
    FOREIGN KEY (‘user_id‘) REFERENCES ‘users‘ (‘id‘)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 6. Table: ‘sdlc_phases‘ (lookup table for SDLC origin phases -- 1NF)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS ‘sdlc_phases‘ (
  ‘id‘ TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘name‘ VARCHAR(64) NOT NULL,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_sdlc_phases_name‘ (‘name‘)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 7. Table: ‘fault_classifications‘ (IEEE Std 1044-2009 anomaly classification -- 3NF)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS ‘fault_classifications‘ (
  ‘id‘ SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘code‘ VARCHAR(100) NOT NULL,
  ‘description‘ VARCHAR(255) NULL DEFAULT NULL,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_fault_classifications_code‘ (‘code‘)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 8. Table: ‘defect_reports‘ (standardized & normalized from ‘raport‘)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS ‘defect_reports‘ (
  ‘id‘ INT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘product_id‘ SMALLINT UNSIGNED NOT NULL,
  ‘environment‘ TEXT NULL DEFAULT NULL,
  ‘detected_environment‘ ENUM(’DEV’, ’CI’, ’STAGING’, ’PROD’) NOT NULL DEFAULT ’PROD’,
```

```
  `symptom_category` VARCHAR(100) NULL DEFAULT NULL COMMENT 'IEEE Std 1044-2009 observed
    symptom classification',
  `is_regression` BOOLEAN NOT NULL DEFAULT FALSE,
  `reporter_user_id` SMALLINT UNSIGNED NULL DEFAULT NULL,
  `guest_name` VARCHAR(100) NULL DEFAULT NULL,
  `guest_email` VARCHAR(255) NULL DEFAULT NULL,
  `reported_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `title` VARCHAR(255) NOT NULL,
  `description` TEXT NOT NULL,
  `discovered_by` VARCHAR(100) NULL DEFAULT NULL,
  `person_type` ENUM('Customer', 'Developer', 'Tester') NOT NULL DEFAULT 'Customer',
  `occurred_on` DATE NOT NULL,
  `location` VARCHAR(255) NULL DEFAULT NULL,
  `initial_severity` ENUM('minor', 'major', 'critical') NOT NULL DEFAULT 'minor',
  `initial_priority` ENUM('most_urgent', 'average', 'less_urgent') NOT NULL DEFAULT
    'average',
  `additional_info` TEXT NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_defect_reports_product_id` (`product_id`),
  KEY `idx_defect_reports_reporter_user_id` (`reporter_user_id`),
  KEY `idx_defect_reports_reported_at` (`reported_at` DESC),
  KEY `idx_defect_reports_detected_env` (`detected_environment`, `reported_at` DESC),
  KEY `idx_defect_reports_severity_reported` (`initial_severity`, `reported_at` DESC),
  KEY `idx_defect_reports_priority_reported` (`initial_priority`, `reported_at` DESC),
  CONSTRAINT `fk_defect_reports_product`
    FOREIGN KEY (`product_id`) REFERENCES `products` (`id`)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT `fk_defect_reports_reporter`
    FOREIGN KEY (`reporter_user_id`) REFERENCES `users` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT `chk_defect_reports_reporter_identity`
    CHECK (
      `reporter_user_id` IS NOT NULL
      OR (`guest_name` IS NOT NULL AND `guest_email` IS NOT NULL)
    )
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 9. Table: `defect_analyses` (IEEE Std 1044-2009 & DORA flow engineering metrics)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS `defect_analyses` (
  `report_id` INT UNSIGNED NOT NULL COMMENT '1:1 Primary & Foreign Key to
    defect_reports.id',
  `developer_description` TEXT NULL DEFAULT NULL,
  `classification_id` SMALLINT UNSIGNED NULL DEFAULT NULL,
  `symptom_category` VARCHAR(100) NULL DEFAULT NULL COMMENT 'IEEE Std 1044-2009 failure
    symptom category',
  `root_cause_type` VARCHAR(100) NULL DEFAULT NULL COMMENT 'IEEE Std 1044-2009 root cause
    classification',
  `detected_environment` ENUM('DEV', 'CI', 'STAGING', 'PROD') NOT NULL DEFAULT 'PROD',
  `is_regression` BOOLEAN NOT NULL DEFAULT FALSE,
  `introducing_ref` VARCHAR(128) NULL DEFAULT NULL COMMENT 'Commit SHA, PR, or build tag
    that introduced the anomaly',
  `resolution_ref` VARCHAR(128) NULL DEFAULT NULL COMMENT 'Commit SHA, PR, or release tag
    that resolved the anomaly',
  `actual_effort_hours` DECIMAL(8,2) UNSIGNED NULL DEFAULT NULL COMMENT 'Actual engineering
    effort expended in hours',
  `triaged_at` DATETIME NULL DEFAULT NULL COMMENT 'Timestamp when anomaly was triaged (DORA
    Triage Lead Time)',
  `resolved_at` DATETIME NULL DEFAULT NULL COMMENT 'Timestamp when anomaly was resolved
    (DORA MTTR)',
  `date_originated` DATE NULL DEFAULT NULL,
  `activity_originated` TEXT NULL DEFAULT NULL,
  `deliverable` VARCHAR(255) NULL DEFAULT NULL,
  `what_to_do` TEXT NULL DEFAULT NULL,
  `assigned_on` DATE NULL DEFAULT NULL,
```

```
  `uniqueness` ENUM('new', 'duplicate') NOT NULL DEFAULT 'new',
  `duplicate_of_report_id` INT UNSIGNED NULL DEFAULT NULL,
  `priority` ENUM('most_urgent', 'average', 'less_urgent') NOT NULL DEFAULT 'average',
  `severity` ENUM('minor', 'major', 'critical') NOT NULL DEFAULT 'minor',
  `workaround` TEXT NULL DEFAULT NULL,
  `proposed_fix` TEXT NULL DEFAULT NULL,
  `status` ENUM('submitted', 'assigned', 'postponed', 'opened', 'fixed', 'resolved',
    'closed') NOT NULL DEFAULT 'submitted',
  `fix_description` TEXT NULL DEFAULT NULL,
  `fix_started_on` DATE NULL DEFAULT NULL,
  `fix_completed_on` DATE NULL DEFAULT NULL,
  `fix_impact` TEXT NULL DEFAULT NULL,
  `fix_effort` DECIMAL(8,2) UNSIGNED NULL DEFAULT NULL,
  `fix_cost` DECIMAL(10,2) UNSIGNED NULL DEFAULT NULL,
  `resolution_description` TEXT NULL DEFAULT NULL,
  `resolution_completed_on` DATE NULL DEFAULT NULL,
  `resolution_closed_on` DATE NULL DEFAULT NULL,
  `cost_of_defect` DECIMAL(10,2) UNSIGNED NULL DEFAULT NULL,
  `activity_found` TEXT NULL DEFAULT NULL,
  PRIMARY KEY (`report_id`),
  KEY `idx_defect_analyses_classification_id` (`classification_id`),
  KEY `idx_defect_analyses_root_cause` (`root_cause_type`),
  KEY `idx_defect_analyses_detected_env` (`detected_environment`),
  KEY `idx_defect_analyses_regression` (`is_regression`),
  KEY `idx_defect_analyses_triaged_resolved` (`triaged_at`, `resolved_at`),
  KEY `idx_defect_analyses_duplicate_of` (`duplicate_of_report_id`),
  KEY `idx_defect_analyses_status_priority` (`status`, `priority`, `severity`),
  CONSTRAINT `fk_defect_analyses_report`
    FOREIGN KEY (`report_id`) REFERENCES `defect_reports` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_defect_analyses_classification`
    FOREIGN KEY (`classification_id`) REFERENCES `fault_classifications` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT `fk_defect_analyses_duplicate_of`
    FOREIGN KEY (`duplicate_of_report_id`) REFERENCES `defect_reports` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 10. Table: `analysis_phases` (junction table for defect_analyses <-> sdlc_phases -- 1NF)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS `analysis_phases` (
  `report_id` INT UNSIGNED NOT NULL,
  `phase_id` TINYINT UNSIGNED NOT NULL,
  PRIMARY KEY (`report_id`, `phase_id`),
  KEY `idx_analysis_phases_phase_id` (`phase_id`),
  CONSTRAINT `fk_analysis_phases_analysis`
    FOREIGN KEY (`report_id`) REFERENCES `defect_analyses` (`report_id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_analysis_phases_phase`
    FOREIGN KEY (`phase_id`) REFERENCES `sdlc_phases` (`id`)
    ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

SET FOREIGN_KEY_CHECKS = 1;


================================================================================
SECTION 2.2 Unified Docker Initialization Schema (database/schema.sql)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\database\schema.sql
================================================================================

-- ========================================================================
-- DefMan (Defect Manager) -- Modernized MySQL 8.0+ 3NF DDL Schema
-- Normalized to Third Normal Form (3NF) with IEEE Std 1044-2009 Anomaly
-- Classification, DORA Flow Telemetry, and Backward-Compatibility Views
```

```
-- Engine: InnoDB | Charset: utf8mb4 | Collation: utf8mb4_unicode_ci
-- ============================================================================

SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;
SET SQL_MODE = 'STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,
    NO_ENGINE_SUBSTITUTION';

-- Drop legacy views/tables if re-initializing
DROP VIEW IF EXISTS `analyse`;
DROP VIEW IF EXISTS `raport`;
DROP VIEW IF EXISTS `product`;

DROP TABLE IF EXISTS `analysis_phases`;
DROP TABLE IF EXISTS `defect_analyses`;
DROP TABLE IF EXISTS `defect_reports`;
DROP TABLE IF EXISTS `fault_classifications`;
DROP TABLE IF EXISTS `sdlc_phases`;
DROP TABLE IF EXISTS `password_resets`;
DROP TABLE IF EXISTS `user_sessions`;
DROP TABLE IF EXISTS `users`;
DROP TABLE IF EXISTS `companies`;
DROP TABLE IF EXISTS `products`;

-- ----------------------------------------------------------------------------
-- 1. Table: `products`
-- ----------------------------------------------------------------------------
CREATE TABLE `products` (
  `id` SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(100) NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_products_name` (`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ----------------------------------------------------------------------------
-- 2. Table: `companies` (3NF lookup table extracted from `users`)
-- ----------------------------------------------------------------------------
CREATE TABLE `companies` (
  `id` SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(100) NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_companies_name` (`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ----------------------------------------------------------------------------
-- 3. Table: `users`
-- User Levels:
--   0 = Applying (Pending Admin Approval - form_apply.php)
--   1 = Public   (Unauthenticated / Public reporter - form_raportpub.php)
--   2 = Customer (Authenticated defect reporter - form_raport.php)
--   3 = Analyzer (Developer / QA triage & IEEE 1044-2009 / DORA telemetry -
    form_analyze.php)
--   4 = Admin    (Full system access, user approval & product CRUD -
    form_usermanagement.php)
-- ----------------------------------------------------------------------------
CREATE TABLE `users` (
  `id` SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `login` VARCHAR(64) NOT NULL COMMENT 'Legacy max 6 chars in 2004 UI; widened for modern
    auth',
  `password` VARCHAR(255) NOT NULL COMMENT 'Argon2id password hash (PASSWORD_ARGON2ID)',
  `email` VARCHAR(255) NOT NULL,
  `userlevel` TINYINT UNSIGNED NOT NULL DEFAULT 0 COMMENT '0=Applying, 1=Public, 2=Customer,
    3=Analyzer, 4=Admin',
  `company_id` SMALLINT UNSIGNED NULL DEFAULT NULL,
```

```
  `phone` VARCHAR(50) NULL DEFAULT NULL,
  `name` VARCHAR(100) NULL DEFAULT NULL,
  `activate` TINYINT(1) NOT NULL DEFAULT 0 COMMENT '0=Pending/Inactive, 1=Active',
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_users_login` (`login`),
  UNIQUE KEY `uk_users_email` (`email`),
  KEY `idx_users_company_id` (`company_id`),
  KEY `idx_users_level_activate` (`userlevel`, `activate`),
  CONSTRAINT `fk_users_company`
    FOREIGN KEY (`company_id`) REFERENCES `companies` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 4. Table: `user_sessions` (Cryptographically secure bearer/session tokens)
-- ------------------------------------------------------------------------
CREATE TABLE `user_sessions` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `user_id` SMALLINT UNSIGNED NOT NULL,
  `token_hash` CHAR(64) NOT NULL COMMENT 'SHA-256 hash of 256-bit random_bytes token',
  `ip_address` VARCHAR(45) NULL DEFAULT NULL,
  `user_agent` VARCHAR(255) NULL DEFAULT NULL,
  `expires_at` DATETIME NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_user_sessions_token_hash` (`token_hash`),
  KEY `idx_user_sessions_user_expires` (`user_id`, `expires_at`),
  KEY `idx_user_sessions_expires_at` (`expires_at`),
  CONSTRAINT `fk_user_sessions_user`
    FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 5. Table: `password_resets` (Decoupled password recovery tokens -- 3NF)
-- ------------------------------------------------------------------------
CREATE TABLE `password_resets` (
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT,
  `user_id` SMALLINT UNSIGNED NOT NULL,
  `token_hash` CHAR(64) NOT NULL COMMENT 'SHA-256 hash of random_bytes(32) recovery token',
  `expires_at` DATETIME NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_password_resets_token_hash` (`token_hash`),
  KEY `idx_password_resets_user_expires` (`user_id`, `expires_at`),
  CONSTRAINT `fk_password_resets_user`
    FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 6. Table: `sdlc_phases` (1NF lookup table for SDLC origin phases)
-- ------------------------------------------------------------------------
CREATE TABLE `sdlc_phases` (
  `id` TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(64) NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_sdlc_phases_name` (`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 7. Table: `fault_classifications` (3NF lookup table for IEEE Std 1044-2009 anomaly types)
-- ------------------------------------------------------------------------
CREATE TABLE `fault_classifications` (
```

```
  `id` SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `code` VARCHAR(100) NOT NULL,
  `description` VARCHAR(255) NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_fault_classifications_code` (`code`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- -----------------------------------------------------------------------------
-- 8. Table: `defect_reports` (3NF normalized defect reports + environment telemetry)
-- -----------------------------------------------------------------------------
CREATE TABLE `defect_reports` (
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT,
  `product_id` SMALLINT UNSIGNED NOT NULL,
  `environment` TEXT NULL DEFAULT NULL,
  `detected_environment` ENUM('DEV', 'CI', 'STAGING', 'PROD') NOT NULL DEFAULT 'PROD',
  `symptom_category` VARCHAR(100) NULL DEFAULT NULL COMMENT 'IEEE Std 1044-2009 observed
    symptom classification',
  `is_regression` BOOLEAN NOT NULL DEFAULT FALSE,
  `reporter_user_id` SMALLINT UNSIGNED NULL DEFAULT NULL,
  `guest_name` VARCHAR(100) NULL DEFAULT NULL,
  `guest_email` VARCHAR(255) NULL DEFAULT NULL,
  `reported_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `title` VARCHAR(255) NOT NULL,
  `description` TEXT NOT NULL,
  `discovered_by` VARCHAR(100) NULL DEFAULT NULL,
  `person_type` ENUM('Customer', 'Developer', 'Tester') NOT NULL DEFAULT 'Customer',
  `occurred_on` DATE NOT NULL,
  `location` VARCHAR(255) NULL DEFAULT NULL,
  `initial_severity` ENUM('minor', 'major', 'critical') NOT NULL DEFAULT 'minor',
  `initial_priority` ENUM('most_urgent', 'average', 'less_urgent') NOT NULL DEFAULT
    'average',
  `additional_info` TEXT NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_defect_reports_product_id` (`product_id`),
  KEY `idx_defect_reports_reporter_user_id` (`reporter_user_id`),
  KEY `idx_defect_reports_reported_at` (`reported_at` DESC),
  KEY `idx_defect_reports_detected_env` (`detected_environment`, `reported_at` DESC),
  KEY `idx_defect_reports_severity_reported` (`initial_severity`, `reported_at` DESC),
  KEY `idx_defect_reports_priority_reported` (`initial_priority`, `reported_at` DESC),
  CONSTRAINT `fk_defect_reports_product`
    FOREIGN KEY (`product_id`) REFERENCES `products` (`id`)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT `fk_defect_reports_reporter`
    FOREIGN KEY (`reporter_user_id`) REFERENCES `users` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT `chk_defect_reports_reporter_identity`
    CHECK (
      `reporter_user_id` IS NOT NULL
      OR (`guest_name` IS NOT NULL AND `guest_email` IS NOT NULL)
    )
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- -----------------------------------------------------------------------------
-- 9. Table: `defect_analyses` (3NF normalized 1:1 extension with IEEE 1044-2009 & DORA
    fields)
-- -----------------------------------------------------------------------------
CREATE TABLE `defect_analyses` (
  `report_id` INT UNSIGNED NOT NULL COMMENT '1:1 Primary & Foreign Key to
    defect_reports.id',
  `developer_description` TEXT NULL DEFAULT NULL,
  `classification_id` SMALLINT UNSIGNED NULL DEFAULT NULL,
  `symptom_category` VARCHAR(100) NULL DEFAULT NULL COMMENT 'IEEE Std 1044-2009 failure
    symptom category',
  `root_cause_type` VARCHAR(100) NULL DEFAULT NULL COMMENT 'IEEE Std 1044-2009 root cause
    classification',
  `detected_environment` ENUM('DEV', 'CI', 'STAGING', 'PROD') NOT NULL DEFAULT 'PROD',
```

```
  `is_regression` BOOLEAN NOT NULL DEFAULT FALSE,
  `introducing_ref` VARCHAR(128) NULL DEFAULT NULL COMMENT 'Commit SHA, PR, or build tag
    that introduced the anomaly',
  `resolution_ref` VARCHAR(128) NULL DEFAULT NULL COMMENT 'Commit SHA, PR, or release tag
    that resolved the anomaly',
  `actual_effort_hours` DECIMAL(8,2) UNSIGNED NULL DEFAULT NULL COMMENT 'Actual engineering
    effort expended in hours',
  `triaged_at` DATETIME NULL DEFAULT NULL COMMENT 'Timestamp when anomaly was triaged (DORA
    Triage Lead Time)',
  `resolved_at` DATETIME NULL DEFAULT NULL COMMENT 'Timestamp when anomaly was resolved
    (DORA MTTR)',
  `date_originated` DATE NULL DEFAULT NULL,
  `activity_originated` TEXT NULL DEFAULT NULL,
  `deliverable` VARCHAR(255) NULL DEFAULT NULL,
  `what_to_do` TEXT NULL DEFAULT NULL,
  `assigned_on` DATE NULL DEFAULT NULL,
  `uniqueness` ENUM('new', 'duplicate') NOT NULL DEFAULT 'new',
  `duplicate_of_report_id` INT UNSIGNED NULL DEFAULT NULL,
  `priority` ENUM('most_urgent', 'average', 'less_urgent') NOT NULL DEFAULT 'average',
  `severity` ENUM('minor', 'major', 'critical') NOT NULL DEFAULT 'minor',
  `workaround` TEXT NULL DEFAULT NULL,
  `proposed_fix` TEXT NULL DEFAULT NULL,
  `status` ENUM('submitted', 'assigned', 'postponed', 'opened', 'fixed', 'resolved',
    'closed') NOT NULL DEFAULT 'submitted',
  `fix_description` TEXT NULL DEFAULT NULL,
  `fix_started_on` DATE NULL DEFAULT NULL,
  `fix_completed_on` DATE NULL DEFAULT NULL,
  `fix_impact` TEXT NULL DEFAULT NULL,
  `fix_effort` DECIMAL(8,2) UNSIGNED NULL DEFAULT NULL,
  `fix_cost` DECIMAL(10,2) UNSIGNED NULL DEFAULT NULL,
  `resolution_description` TEXT NULL DEFAULT NULL,
  `resolution_completed_on` DATE NULL DEFAULT NULL,
  `resolution_closed_on` DATE NULL DEFAULT NULL,
  `cost_of_defect` DECIMAL(10,2) UNSIGNED NULL DEFAULT NULL,
  `activity_found` TEXT NULL DEFAULT NULL,
  PRIMARY KEY (`report_id`),
  KEY `idx_defect_analyses_classification_id` (`classification_id`),
  KEY `idx_defect_analyses_root_cause` (`root_cause_type`),
  KEY `idx_defect_analyses_detected_env` (`detected_environment`),
  KEY `idx_defect_analyses_regression` (`is_regression`),
  KEY `idx_defect_analyses_triaged_resolved` (`triaged_at`, `resolved_at`),
  KEY `idx_defect_analyses_duplicate_of` (`duplicate_of_report_id`),
  KEY `idx_defect_analyses_status_priority` (`status`, `priority`, `severity`),
  CONSTRAINT `fk_defect_analyses_report`
    FOREIGN KEY (`report_id`) REFERENCES `defect_reports` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_defect_analyses_classification`
    FOREIGN KEY (`classification_id`) REFERENCES `fault_classifications` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT `fk_defect_analyses_duplicate_of`
    FOREIGN KEY (`duplicate_of_report_id`) REFERENCES `defect_reports` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 10. Table: `analysis_phases` (1NF junction table for defect_analyses <-> sdlc_phases)
-- ------------------------------------------------------------------------
CREATE TABLE `analysis_phases` (
  `report_id` INT UNSIGNED NOT NULL,
  `phase_id` TINYINT UNSIGNED NOT NULL,
  PRIMARY KEY (`report_id`, `phase_id`),
  KEY `idx_analysis_phases_phase_id` (`phase_id`),
  CONSTRAINT `fk_analysis_phases_analysis`
    FOREIGN KEY (`report_id`) REFERENCES `defect_analyses` (`report_id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_analysis_phases_phase`
```

```
    FOREIGN KEY (‘phase_id‘) REFERENCES ‘sdlc_phases‘ (‘id‘)
    ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- Backward-Compatibility Views (‘product‘, ‘raport‘, ‘analyse‘)
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW ‘product‘ AS
SELECT
  p.‘id‘,
  p.‘name‘,
  p.‘created_at‘
FROM ‘products‘ p;

CREATE OR REPLACE VIEW ‘raport‘ AS
SELECT
  dr.‘id‘ AS ‘ID‘,
  p.‘name‘ AS ‘Product‘,
  dr.‘product_id‘ AS ‘product_id‘,
  dr.‘environment‘ AS ‘Environment‘,
  dr.‘reporter_user_id‘ AS ‘CustomerID‘,
  COALESCE(u.‘login‘, dr.‘guest_name‘, ’’) AS ‘Who‘,
  COALESCE(u.‘email‘, dr.‘guest_email‘, ’’) AS ‘Contactemail‘,
  DATE_FORMAT(dr.‘reported_at‘, ’%Y-%m-%d %H:%i’) AS ‘Whenreported‘,
  dr.‘title‘ AS ‘Title‘,
  dr.‘description‘ AS ‘Description‘,
  dr.‘discovered_by‘ AS ‘Discoveredby‘,
  dr.‘person_type‘ AS ‘Persontype‘,
  dr.‘occurred_on‘ AS ‘Occured‘,
  dr.‘location‘ AS ‘Location‘,
  dr.‘initial_severity‘ AS ‘Severity‘,
  CASE dr.‘initial_priority‘
    WHEN ’most_urgent’ THEN ’m_urgent’
    WHEN ’average’     THEN ’average’
    WHEN ’less_urgent’ THEN ’l_urgent’
  END AS ‘Priority‘,
  DATE(dr.‘reported_at‘) AS ‘Submitted‘,
  dr.‘additional_info‘ AS ‘Addinfo‘
FROM ‘defect_reports‘ dr
INNER JOIN ‘products‘ p
  ON p.‘id‘ = dr.‘product_id‘
LEFT JOIN ‘users‘ u
  ON u.‘id‘ = dr.‘reporter_user_id‘;

CREATE OR REPLACE VIEW ‘analyse‘ AS
SELECT
  da.‘report_id‘ AS ‘id‘,
  da.‘developer_description‘ AS ‘devdesc‘,
  COALESCE(da.‘root_cause_type‘, fc.‘code‘) AS ‘classification‘,
  da.‘date_originated‘ AS ‘dateoriginated‘,
  ph.‘phases_csv‘ AS ‘phase‘,
  da.‘activity_originated‘ AS ‘actoriginated‘,
  da.‘deliverable‘ AS ‘deliverable‘,
  da.‘what_to_do‘ AS ‘whattodo‘,
  DATE(dr.‘reported_at‘) AS ‘submitted‘,
  COALESCE(da.‘assigned_on‘, DATE(da.‘triaged_at‘)) AS ‘assigned‘,
  CASE da.‘uniqueness‘
    WHEN ’duplicate’ THEN ’Already submitted’
    WHEN ’new’       THEN ’New defect’
  END AS ‘uniqueness‘,
  CASE da.‘priority‘
    WHEN ’most_urgent’ THEN ’most urgent’
    WHEN ’average’     THEN ’average’
    WHEN ’less_urgent’ THEN ’less urgent’
  END AS ‘priority‘,
  da.‘severity‘ AS ‘severity‘,
```

```
  da.‘workaround‘ AS ‘workaround‘,
  da.‘proposed_fix‘ AS ‘proposedfix‘,
  da.‘status‘ AS ‘status‘,
  da.‘fix_description‘ AS ‘fixdesc‘,
  da.‘fix_started_on‘ AS ‘fixstarted‘,
  da.‘fix_completed_on‘ AS ‘fixcomplete‘,
  da.‘fix_impact‘ AS ‘fiximpact‘,
  COALESCE(da.‘actual_effort_hours‘, da.‘fix_effort‘) AS ‘fixeffort‘,
  da.‘fix_cost‘ AS ‘fixcost‘,
  da.‘resolution_description‘ AS ‘resodesc‘,
  COALESCE(da.‘resolution_completed_on‘, DATE(da.‘resolved_at‘)) AS ‘resocomplete‘,
  da.‘resolution_closed_on‘ AS ‘resoclosed‘,
  da.‘cost_of_defect‘ AS ‘costofdefect‘,
  da.‘activity_found‘ AS ‘actfound‘
FROM ‘defect_analyses‘ da
INNER JOIN ‘defect_reports‘ dr
  ON dr.‘id‘ = da.‘report_id‘
LEFT JOIN ‘fault_classifications‘ fc
  ON fc.‘id‘ = da.‘classification_id‘
LEFT JOIN (
  SELECT
    ap.‘report_id‘,
    GROUP_CONCAT(sp.‘name‘ ORDER BY sp.‘id‘ SEPARATOR ’,’) AS ‘phases_csv‘
  FROM ‘analysis_phases‘ ap
  INNER JOIN ‘sdlc_phases‘ sp
    ON sp.‘id‘ = ap.‘phase_id‘
  GROUP BY ap.‘report_id‘
) ph
  ON ph.‘report_id‘ = da.‘report_id‘;

SET FOREIGN_KEY_CHECKS = 1;


================================================================================
SECTION 2.3 Idempotent DML Migration Script (database/02_migrate_data.sql)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\database\02_migrate_data.sql
================================================================================

-- ============================================================================
-- File: 02_migrate_data.sql
-- Description: Idempotent, Zero-Data-Loss DML Migration from Legacy Tables
--              (‘product‘, ‘users‘, ‘user_sessions‘, ‘raport‘, ‘analyse‘)
--              into the 3NF Schema (‘products‘, ‘companies‘, ‘users‘,
--              ‘user_sessions‘, ‘password_resets‘, ‘sdlc_phases‘,
--              ‘fault_classifications‘, ‘defect_reports‘, ‘defect_analyses‘,
--              ‘analysis_phases‘) with IEEE Std 1044-2009 Anomaly Classification
--              and DORA/Flow Engineering Telemetry columns.
-- ============================================================================

SET NAMES utf8mb4;

-- ============================================================================
-- PRE-FLIGHT DDL PREPARATION (for in-place migration on an existing database)
-- Note: MySQL implicitly commits on DDL (CREATE/ALTER/DROP/RENAME), so schema
-- preparation runs prior to the atomic DML transaction block below.
-- ============================================================================
SET FOREIGN_KEY_CHECKS = 0;

CREATE TABLE IF NOT EXISTS ‘products‘ (
  ‘id‘ SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘name‘ VARCHAR(100) NOT NULL,
  ‘created_at‘ TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uk_products_name‘ (‘name‘)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
CREATE TABLE IF NOT EXISTS `companies` (
  `id` SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(100) NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_companies_name` (`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Add `company_id` to `users` if migrating in-place from legacy `users`
SET @col_exists := (
  SELECT COUNT(*)
  FROM INFORMATION_SCHEMA.COLUMNS
  WHERE TABLE_SCHEMA = DATABASE()
    AND TABLE_NAME = 'users'
    AND COLUMN_NAME = 'company_id'
);
SET @sql_add_company_id := IF(
  @col_exists = 0,
  'ALTER TABLE `users` ADD COLUMN `company_id` SMALLINT UNSIGNED NULL DEFAULT NULL AFTER
    `userlevel`, ADD KEY `idx_users_company_id` (`company_id`), ADD CONSTRAINT
    `fk_users_company` FOREIGN KEY (`company_id`) REFERENCES `companies` (`id`) ON DELETE
    SET NULL ON UPDATE CASCADE',
  'SELECT 1'
);
PREPARE stmt FROM @sql_add_company_id;
EXECUTE stmt;
DEALLOCATE PREPARE stmt;

CREATE TABLE IF NOT EXISTS `password_resets` (
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT,
  `user_id` SMALLINT UNSIGNED NOT NULL,
  `token_hash` CHAR(64) NOT NULL,
  `expires_at` DATETIME NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_password_resets_token_hash` (`token_hash`),
  KEY `idx_password_resets_user_expires` (`user_id`, `expires_at`),
  CONSTRAINT `fk_password_resets_user`
    FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS `sdlc_phases` (
  `id` TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(64) NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_sdlc_phases_name` (`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS `fault_classifications` (
  `id` SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `code` VARCHAR(100) NOT NULL,
  `description` VARCHAR(255) NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_fault_classifications_code` (`code`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS `defect_reports` (
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT,
  `product_id` SMALLINT UNSIGNED NOT NULL,
  `environment` TEXT NULL DEFAULT NULL,
  `detected_environment` ENUM('DEV', 'CI', 'STAGING', 'PROD') NOT NULL DEFAULT 'PROD',
  `symptom_category` VARCHAR(100) NULL DEFAULT NULL,
  `is_regression` BOOLEAN NOT NULL DEFAULT FALSE,
  `reporter_user_id` SMALLINT UNSIGNED NULL DEFAULT NULL,
  `guest_name` VARCHAR(100) NULL DEFAULT NULL,
```

```
  `guest_email` VARCHAR(255) NULL DEFAULT NULL,
  `reported_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `title` VARCHAR(255) NOT NULL,
  `description` TEXT NOT NULL,
  `discovered_by` VARCHAR(100) NULL DEFAULT NULL,
  `person_type` ENUM('Customer', 'Developer', 'Tester') NOT NULL DEFAULT 'Customer',
  `occurred_on` DATE NOT NULL,
  `location` VARCHAR(255) NULL DEFAULT NULL,
  `initial_severity` ENUM('minor', 'major', 'critical') NOT NULL DEFAULT 'minor',
  `initial_priority` ENUM('most_urgent', 'average', 'less_urgent') NOT NULL DEFAULT
    'average',
  `additional_info` TEXT NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_defect_reports_product_id` (`product_id`),
  KEY `idx_defect_reports_reporter_user_id` (`reporter_user_id`),
  KEY `idx_defect_reports_reported_at` (`reported_at` DESC),
  KEY `idx_defect_reports_detected_env` (`detected_environment`, `reported_at` DESC),
  KEY `idx_defect_reports_severity_reported` (`initial_severity`, `reported_at` DESC),
  KEY `idx_defect_reports_priority_reported` (`initial_priority`, `reported_at` DESC),
  CONSTRAINT `fk_defect_reports_product`
    FOREIGN KEY (`product_id`) REFERENCES `products` (`id`)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT `fk_defect_reports_reporter`
    FOREIGN KEY (`reporter_user_id`) REFERENCES `users` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT `chk_defect_reports_reporter_identity`
    CHECK (
      `reporter_user_id` IS NOT NULL
      OR (`guest_name` IS NOT NULL AND `guest_email` IS NOT NULL)
    )
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS `defect_analyses` (
  `report_id` INT UNSIGNED NOT NULL,
  `developer_description` TEXT NULL DEFAULT NULL,
  `classification_id` SMALLINT UNSIGNED NULL DEFAULT NULL,
  `symptom_category` VARCHAR(100) NULL DEFAULT NULL,
  `root_cause_type` VARCHAR(100) NULL DEFAULT NULL,
  `detected_environment` ENUM('DEV', 'CI', 'STAGING', 'PROD') NOT NULL DEFAULT 'PROD',
  `is_regression` BOOLEAN NOT NULL DEFAULT FALSE,
  `introducing_ref` VARCHAR(128) NULL DEFAULT NULL,
  `resolution_ref` VARCHAR(128) NULL DEFAULT NULL,
  `actual_effort_hours` DECIMAL(8,2) UNSIGNED NULL DEFAULT NULL,
  `triaged_at` DATETIME NULL DEFAULT NULL,
  `resolved_at` DATETIME NULL DEFAULT NULL,
  `date_originated` DATE NULL DEFAULT NULL,
  `activity_originated` TEXT NULL DEFAULT NULL,
  `deliverable` VARCHAR(255) NULL DEFAULT NULL,
  `what_to_do` TEXT NULL DEFAULT NULL,
  `assigned_on` DATE NULL DEFAULT NULL,
  `uniqueness` ENUM('new', 'duplicate') NOT NULL DEFAULT 'new',
  `duplicate_of_report_id` INT UNSIGNED NULL DEFAULT NULL,
  `priority` ENUM('most_urgent', 'average', 'less_urgent') NOT NULL DEFAULT 'average',
  `severity` ENUM('minor', 'major', 'critical') NOT NULL DEFAULT 'minor',
  `workaround` TEXT NULL DEFAULT NULL,
  `proposed_fix` TEXT NULL DEFAULT NULL,
  `status` ENUM('submitted', 'assigned', 'postponed', 'opened', 'fixed', 'resolved',
    'closed') NOT NULL DEFAULT 'submitted',
  `fix_description` TEXT NULL DEFAULT NULL,
  `fix_started_on` DATE NULL DEFAULT NULL,
  `fix_completed_on` DATE NULL DEFAULT NULL,
  `fix_impact` TEXT NULL DEFAULT NULL,
  `fix_effort` DECIMAL(8,2) UNSIGNED NULL DEFAULT NULL,
  `fix_cost` DECIMAL(10,2) UNSIGNED NULL DEFAULT NULL,
  `resolution_description` TEXT NULL DEFAULT NULL,
  `resolution_completed_on` DATE NULL DEFAULT NULL,
```

```
  `resolution_closed_on` DATE NULL DEFAULT NULL,
  `cost_of_defect` DECIMAL(10,2) UNSIGNED NULL DEFAULT NULL,
  `activity_found` TEXT NULL DEFAULT NULL,
  PRIMARY KEY (`report_id`),
  KEY `idx_defect_analyses_classification_id` (`classification_id`),
  KEY `idx_defect_analyses_root_cause` (`root_cause_type`),
  KEY `idx_defect_analyses_detected_env` (`detected_environment`),
  KEY `idx_defect_analyses_regression` (`is_regression`),
  KEY `idx_defect_analyses_triaged_resolved` (`triaged_at`, `resolved_at`),
  KEY `idx_defect_analyses_duplicate_of` (`duplicate_of_report_id`),
  KEY `idx_defect_analyses_status_priority` (`status`, `priority`, `severity`),
  CONSTRAINT `fk_defect_analyses_report`
    FOREIGN KEY (`report_id`) REFERENCES `defect_reports` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_defect_analyses_classification`
    FOREIGN KEY (`classification_id`) REFERENCES `fault_classifications` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT `fk_defect_analyses_duplicate_of`
    FOREIGN KEY (`duplicate_of_report_id`) REFERENCES `defect_reports` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS `analysis_phases` (
  `report_id` INT UNSIGNED NOT NULL,
  `phase_id` TINYINT UNSIGNED NOT NULL,
  PRIMARY KEY (`report_id`, `phase_id`),
  KEY `idx_analysis_phases_phase_id` (`phase_id`),
  CONSTRAINT `fk_analysis_phases_analysis`
    FOREIGN KEY (`report_id`) REFERENCES `defect_analyses` (`report_id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_analysis_phases_phase`
    FOREIGN KEY (`phase_id`) REFERENCES `sdlc_phases` (`id`)
    ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

SET FOREIGN_KEY_CHECKS = 1;

-- ============================================================================
-- ATOMIC DML DATA MIGRATION (START TRANSACTION ... COMMIT)
-- ============================================================================
START TRANSACTION;

-- ----------------------------------------------------------------------------
-- 1. Migrate `product` -> `products` and backfill uncataloged `raport.Product`
-- ----------------------------------------------------------------------------
INSERT INTO `products` (`id`, `name`, `created_at`)
SELECT
  p.`id`,
  TRIM(p.`name`),
  p.`created_at`
FROM `product` p
ON DUPLICATE KEY UPDATE
  `name` = VALUES(`name`);

INSERT IGNORE INTO `products` (`name`)
SELECT DISTINCT
  COALESCE(NULLIF(TRIM(r.`Product`), ''), 'Unknown Product')
FROM `raport` r
LEFT JOIN `products` p
  ON p.`name` = COALESCE(NULLIF(TRIM(r.`Product`), ''), 'Unknown Product')
WHERE p.`id` IS NULL;

-- Backfill NULL `raport.product_id` by matching `raport.Product = products.name`
UPDATE `raport` r
INNER JOIN `products` p
  ON p.`name` = COALESCE(NULLIF(TRIM(r.`Product`), ''), 'Unknown Product')
```

```
SET r.‘product_id‘ = p.‘id‘
WHERE r.‘product_id‘ IS NULL;

-- ----------------------------------------------------------------------------
-- 2. Extract distinct non-null ‘users.company‘ into ‘companies‘ & link ‘company_id‘
-- ----------------------------------------------------------------------------
INSERT IGNORE INTO ‘companies‘ (‘name‘)
SELECT DISTINCT
  TRIM(u.‘company‘)
FROM ‘users‘ u
WHERE u.‘company‘ IS NOT NULL
  AND TRIM(u.‘company‘) <> ’’;

UPDATE ‘users‘ u
INNER JOIN ‘companies‘ c
  ON c.‘name‘ = TRIM(u.‘company‘)
SET u.‘company_id‘ = c.‘id‘
WHERE u.‘company‘ IS NOT NULL
  AND TRIM(u.‘company‘) <> ’’;

-- ----------------------------------------------------------------------------
-- 3. Migrate active non-null ‘reset_token_hash‘ rows from ‘users‘ to ‘password_resets‘
-- ----------------------------------------------------------------------------
INSERT IGNORE INTO ‘password_resets‘ (‘user_id‘, ‘token_hash‘, ‘expires_at‘, ‘created_at‘)
SELECT
  u.‘id‘,
  u.‘reset_token_hash‘,
  COALESCE(u.‘reset_token_expires_at‘, DATE_ADD(NOW(), INTERVAL 1 HOUR)),
  u.‘updated_at‘
FROM ‘users‘ u
WHERE u.‘reset_token_hash‘ IS NOT NULL
  AND TRIM(u.‘reset_token_hash‘) <> ’’
  AND (u.‘reset_token_expires_at‘ IS NULL OR u.‘reset_token_expires_at‘ > NOW());

-- ----------------------------------------------------------------------------
-- 4. Migrate ‘raport‘ -> ‘defect_reports‘
--    - Converts ‘Whenreported‘ VARCHAR(32) to DATETIME ‘reported_at‘ via STR_TO_DATE()
--    - Derives ‘detected_environment‘ (’DEV’, ’CI’, ’STAGING’, ’PROD’) & ‘symptom_category‘
--    - Normalizes reporter identity (‘reporter_user_id‘ vs ‘guest_name‘/‘guest_email‘)
--    - Standardizes ‘Priority‘ (’m_urgent’ -> ’most_urgent’, ’l_urgent’ -> ’less_urgent’)
-- ----------------------------------------------------------------------------
INSERT INTO ‘defect_reports‘ (
  ‘id‘,
  ‘product_id‘,
  ‘environment‘,
  ‘detected_environment‘,
  ‘symptom_category‘,
  ‘is_regression‘,
  ‘reporter_user_id‘,
  ‘guest_name‘,
  ‘guest_email‘,
  ‘reported_at‘,
  ‘title‘,
  ‘description‘,
  ‘discovered_by‘,
  ‘person_type‘,
  ‘occurred_on‘,
  ‘location‘,
  ‘initial_severity‘,
  ‘initial_priority‘,
  ‘additional_info‘
)
SELECT
  r.‘ID‘ AS ‘id‘,
  COALESCE(p_id.‘id‘, p_name.‘id‘) AS ‘product_id‘,
  r.‘Environment‘ AS ‘environment‘,
```

```
  CASE
    WHEN r.‘Persontype‘ = ’Developer’ THEN ’DEV’
    WHEN r.‘Persontype‘ = ’Tester’    THEN ’STAGING’
    ELSE ’PROD’
  END AS ‘detected_environment‘,
  CASE
    WHEN r.‘Title‘ LIKE ’%Crash%’ OR r.‘Description‘ LIKE ’%close%’ THEN ’Application Crash
    / Hang’
    WHEN r.‘Title‘ LIKE ’%Precision%’ OR r.‘Description‘ LIKE ’%truncat%’ THEN ’Incorrect
    Calculation / Output’
    ELSE ’General Anomaly’
  END AS ‘symptom_category‘,
  FALSE AS ‘is_regression‘,
  u.‘id‘ AS ‘reporter_user_id‘,
  CASE
    WHEN u.‘id‘ IS NOT NULL THEN NULL
    ELSE COALESCE(NULLIF(TRIM(r.‘Who‘), ’’), ’Anonymous Guest’)
  END AS ‘guest_name‘,
  CASE
    WHEN u.‘id‘ IS NOT NULL THEN NULL
    ELSE COALESCE(NULLIF(TRIM(r.‘Contactemail‘), ’’), ’unknown@example.invalid’)
  END AS ‘guest_email‘,
  COALESCE(
    STR_TO_DATE(TRIM(r.‘Whenreported‘), ’%Y-%m-%d %H:%i:%s’),
    STR_TO_DATE(TRIM(r.‘Whenreported‘), ’%Y-%m-%d %H:%i’),
    STR_TO_DATE(TRIM(r.‘Whenreported‘), ’%Y-%m-%d’),
    CAST(r.‘Submitted‘ AS DATETIME),
    NOW()
  ) AS ‘reported_at‘,
  r.‘Title‘ AS ‘title‘,
  r.‘Description‘ AS ‘description‘,
  r.‘Discoveredby‘ AS ‘discovered_by‘,
  r.‘Persontype‘ AS ‘person_type‘,
  r.‘Occured‘ AS ‘occurred_on‘,
  r.‘Location‘ AS ‘location‘,
  r.‘Severity‘ AS ‘initial_severity‘,
  CASE r.‘Priority‘
    WHEN ’m_urgent’ THEN ’most_urgent’
    WHEN ’average’  THEN ’average’
    WHEN ’l_urgent’ THEN ’less_urgent’
    ELSE ’average’
  END AS ‘initial_priority‘,
  r.‘Addinfo‘ AS ‘additional_info‘
FROM ‘raport‘ r
LEFT JOIN ‘products‘ p_id
  ON p_id.‘id‘ = r.‘product_id‘
LEFT JOIN ‘products‘ p_name
  ON p_name.‘name‘ = COALESCE(NULLIF(TRIM(r.‘Product‘), ’’), ’Unknown Product’)
LEFT JOIN ‘users‘ u
  ON u.‘id‘ = r.‘CustomerID‘
ON DUPLICATE KEY UPDATE
  ‘product_id‘           = VALUES(‘product_id‘),
  ‘detected_environment‘ = VALUES(‘detected_environment‘),
  ‘symptom_category‘     = VALUES(‘symptom_category‘),
  ‘reporter_user_id‘     = VALUES(‘reporter_user_id‘),
  ‘guest_name‘           = VALUES(‘guest_name‘),
  ‘guest_email‘          = VALUES(‘guest_email‘),
  ‘reported_at‘          = VALUES(‘reported_at‘),
  ‘initial_priority‘     = VALUES(‘initial_priority‘);

-- ------------------------------------------------------------------------
-- 5. Migrate ‘analyse.classification‘ -> ‘fault_classifications‘
-- ------------------------------------------------------------------------
INSERT IGNORE INTO ‘fault_classifications‘ (‘code‘)
SELECT DISTINCT
  TRIM(a.‘classification‘)
```

```
FROM ‘analyse‘ a
WHERE a.‘classification‘ IS NOT NULL
  AND TRIM(a.‘classification‘) <> ’’;

-- ----------------------------------------------------------------------------
-- 6. Migrate ‘analyse‘ -> ‘defect_analyses‘
--    - Maps legacy 1992 classifications to IEEE Std 1044-2009 ‘root_cause_type‘
--      and ‘symptom_category‘
--    - Populates DORA timestamps (‘triaged_at‘, ‘resolved_at‘), ‘actual_effort_hours‘,
--      ‘detected_environment‘, and ‘is_regression‘
-- ----------------------------------------------------------------------------
INSERT INTO ‘defect_analyses‘ (
  ‘report_id‘,
  ‘developer_description‘,
  ‘classification_id‘,
  ‘symptom_category‘,
  ‘root_cause_type‘,
  ‘detected_environment‘,
  ‘is_regression‘,
  ‘introducing_ref‘,
  ‘resolution_ref‘,
  ‘actual_effort_hours‘,
  ‘triaged_at‘,
  ‘resolved_at‘,
  ‘date_originated‘,
  ‘activity_originated‘,
  ‘deliverable‘,
  ‘what_to_do‘,
  ‘assigned_on‘,
  ‘uniqueness‘,
  ‘duplicate_of_report_id‘,
  ‘priority‘,
  ‘severity‘,
  ‘workaround‘,
  ‘proposed_fix‘,
  ‘status‘,
  ‘fix_description‘,
  ‘fix_started_on‘,
  ‘fix_completed_on‘,
  ‘fix_impact‘,
  ‘fix_effort‘,
  ‘fix_cost‘,
  ‘resolution_description‘,
  ‘resolution_completed_on‘,
  ‘resolution_closed_on‘,
  ‘cost_of_defect‘,
  ‘activity_found‘
)
SELECT
  a.‘id‘ AS ‘report_id‘,
  a.‘devdesc‘ AS ‘developer_description‘,
  fc.‘id‘ AS ‘classification_id‘,
  COALESCE(dr.‘symptom_category‘, ’General Anomaly’) AS ‘symptom_category‘,
  CASE TRIM(COALESCE(a.‘classification‘, ’Other’))
    WHEN ’Logic’                  THEN ’Code / Logic’
    WHEN ’Computational’          THEN ’Computational’
    WHEN ’Interface’              THEN ’Interface / Integration’
    WHEN ’Data handling’          THEN ’Data / Schema’
    WHEN ’Data’                   THEN ’Data / Schema’
    WHEN ’Documentation’          THEN ’Documentation’
    WHEN ’Document quality’       THEN ’Documentation’
    WHEN ’Enhancement’            THEN ’Requirements / Specification’
    WHEN ’Caused by a prev. Fix’  THEN ’Regression’
    WHEN ’Caused by a prev. fix’  THEN ’Regression’
    ELSE TRIM(COALESCE(a.‘classification‘, ’Other’))
  END AS ‘root_cause_type‘,
```

```
  dr.‘detected_environment‘ AS ‘detected_environment‘,
  CASE
    WHEN TRIM(COALESCE(a.‘classification‘, ’’)) LIKE ’Caused by a prev%’ THEN TRUE
    ELSE FALSE
  END AS ‘is_regression‘,
  NULL AS ‘introducing_ref‘,
  NULL AS ‘resolution_ref‘,
  a.‘fixeffort‘ AS ‘actual_effort_hours‘,
  CASE
    WHEN a.‘assigned‘ IS NOT NULL THEN CAST(CONCAT(a.‘assigned‘, ’ 10:00:00’) AS DATETIME)
    ELSE NULL
  END AS ‘triaged_at‘,
  CASE
    WHEN a.‘resocomplete‘ IS NOT NULL THEN CAST(CONCAT(a.‘resocomplete‘, ’ 16:00:00’) AS
    DATETIME)
    WHEN a.‘fixcomplete‘ IS NOT NULL  THEN CAST(CONCAT(a.‘fixcomplete‘, ’ 16:00:00’) AS
    DATETIME)
    ELSE NULL
  END AS ‘resolved_at‘,
  a.‘dateoriginated‘ AS ‘date_originated‘,
  a.‘actoriginated‘ AS ‘activity_originated‘,
  a.‘deliverable‘ AS ‘deliverable‘,
  a.‘whattodo‘ AS ‘what_to_do‘,
  a.‘assigned‘ AS ‘assigned_on‘,
  CASE a.‘uniqueness‘
    WHEN ’Already submitted’ THEN ’duplicate’
    WHEN ’New defect’        THEN ’new’
    WHEN ’Critical’          THEN ’new’
    ELSE ’new’
  END AS ‘uniqueness‘,
  NULL AS ‘duplicate_of_report_id‘,
  CASE a.‘priority‘
    WHEN ’most urgent’ THEN ’most_urgent’
    WHEN ’average’     THEN ’average’
    WHEN ’less urgent’ THEN ’less_urgent’
    ELSE ’average’
  END AS ‘priority‘,
  CASE
    WHEN a.‘uniqueness‘ = ’Critical’ THEN ’critical’
    ELSE a.‘severity‘
  END AS ‘severity‘,
  a.‘workaround‘ AS ‘workaround‘,
  a.‘proposedfix‘ AS ‘proposed_fix‘,
  a.‘status‘ AS ‘status‘,
  a.‘fixdesc‘ AS ‘fix_description‘,
  a.‘fixstarted‘ AS ‘fix_started_on‘,
  a.‘fixcomplete‘ AS ‘fix_completed_on‘,
  a.‘fiximpact‘ AS ‘fix_impact‘,
  a.‘fixeffort‘ AS ‘fix_effort‘,
  a.‘fixcost‘ AS ‘fix_cost‘,
  a.‘resodesc‘ AS ‘resolution_description‘,
  a.‘resocomplete‘ AS ‘resolution_completed_on‘,
  a.‘resoclosed‘ AS ‘resolution_closed_on‘,
  a.‘costofdefect‘ AS ‘cost_of_defect‘,
  a.‘actfound‘ AS ‘activity_found‘
FROM ‘analyse‘ a
INNER JOIN ‘defect_reports‘ dr
  ON dr.‘id‘ = a.‘id‘
LEFT JOIN ‘fault_classifications‘ fc
  ON fc.‘code‘ = TRIM(a.‘classification‘)
ON DUPLICATE KEY UPDATE
  ‘classification_id‘    = VALUES(‘classification_id‘),
  ‘symptom_category‘     = VALUES(‘symptom_category‘),
  ‘root_cause_type‘      = VALUES(‘root_cause_type‘),
  ‘detected_environment‘ = VALUES(‘detected_environment‘),
  ‘is_regression‘        = VALUES(‘is_regression‘),
```

```
  `actual_effort_hours`  = VALUES(`actual_effort_hours`),
  `triaged_at`           = VALUES(`triaged_at`),
  `resolved_at`          = VALUES(`resolved_at`),
  `uniqueness`           = VALUES(`uniqueness`),
  `priority`             = VALUES(`priority`),
  `severity`             = VALUES(`severity`);

-- ------------------------------------------------------------------------
-- 7. Split comma-delimited `analyse.phase` into `sdlc_phases` and `analysis_phases`
-- ------------------------------------------------------------------------
INSERT IGNORE INTO `sdlc_phases` (`name`)
WITH RECURSIVE `phase_splitter` (`report_id`, `phase_token`, `remainder`) AS (
  SELECT
    a.`id` AS `report_id`,
    TRIM(SUBSTRING_INDEX(a.`phase`, ',', 1)) AS `phase_token`,
    SUBSTRING(a.`phase`, CHAR_LENGTH(SUBSTRING_INDEX(a.`phase`, ',', 1)) + 2) AS `remainder`
  FROM `analyse` a
  WHERE a.`phase` IS NOT NULL
    AND TRIM(a.`phase`) <> ''
  UNION ALL
  SELECT
    ps.`report_id`,
    TRIM(SUBSTRING_INDEX(ps.`remainder`, ',', 1)) AS `phase_token`,
    SUBSTRING(ps.`remainder`, CHAR_LENGTH(SUBSTRING_INDEX(ps.`remainder`, ',', 1)) + 2) AS
    `remainder`
  FROM `phase_splitter` ps
  WHERE ps.`remainder` <> ''
)
SELECT DISTINCT
  ps.`phase_token`
FROM `phase_splitter` ps
WHERE ps.`phase_token` <> '';

INSERT IGNORE INTO `analysis_phases` (`report_id`, `phase_id`)
WITH RECURSIVE `phase_splitter` (`report_id`, `phase_token`, `remainder`) AS (
  SELECT
    a.`id` AS `report_id`,
    TRIM(SUBSTRING_INDEX(a.`phase`, ',', 1)) AS `phase_token`,
    SUBSTRING(a.`phase`, CHAR_LENGTH(SUBSTRING_INDEX(a.`phase`, ',', 1)) + 2) AS `remainder`
  FROM `analyse` a
  INNER JOIN `defect_analyses` da
    ON da.`report_id` = a.`id`
  WHERE a.`phase` IS NOT NULL
    AND TRIM(a.`phase`) <> ''
  UNION ALL
  SELECT
    ps.`report_id`,
    TRIM(SUBSTRING_INDEX(ps.`remainder`, ',', 1)) AS `phase_token`,
    SUBSTRING(ps.`remainder`, CHAR_LENGTH(SUBSTRING_INDEX(ps.`remainder`, ',', 1)) + 2) AS
    `remainder`
  FROM `phase_splitter` ps
  WHERE ps.`remainder` <> ''
)
SELECT
  ps.`report_id`,
  sp.`id` AS `phase_id`
FROM `phase_splitter` ps
INNER JOIN `sdlc_phases` sp
  ON sp.`name` = ps.`phase_token`
WHERE ps.`phase_token` <> '';

COMMIT;

-- ========================================================================
-- POST-MIGRATION CLEANUP (Drop denormalized columns & legacy tables)
-- ========================================================================
```

```
SET FOREIGN_KEY_CHECKS = 0;

ALTER TABLE ‘users‘
  DROP COLUMN IF EXISTS ‘company‘,
  DROP COLUMN IF EXISTS ‘logged‘,
  DROP COLUMN IF EXISTS ‘reset_token_hash‘,
  DROP COLUMN IF EXISTS ‘reset_token_expires_at‘;

DROP TABLE IF EXISTS ‘analyse‘;
DROP TABLE IF EXISTS ‘raport‘;
DROP TABLE IF EXISTS ‘product‘;

SET FOREIGN_KEY_CHECKS = 1;


================================================================================
SECTION 2.4 Backward-Compatibility Read Views (database/03_legacy_views.sql)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\database\03_legacy_views.sql
================================================================================

-- ============================================================================
-- File: 03_legacy_views.sql
-- Description: Backward-Compatibility Read Views (‘raport‘, ‘analyse‘, ‘product‘)
--              Reconstructs exact legacy 2004 column names and formats from 3NF
--              IEEE Std 1044-2009 and DORA telemetry tables so legacy queries
--              still resolve without error.
-- ============================================================================

SET NAMES utf8mb4;

-- ----------------------------------------------------------------------------
-- 1. View: ‘product‘ (compatibility alias for ‘products‘)
-- ----------------------------------------------------------------------------
CREATE OR REPLACE VIEW ‘product‘ AS
SELECT
  p.‘id‘,
  p.‘name‘,
  p.‘created_at‘
FROM ‘products‘ p;

-- ----------------------------------------------------------------------------
-- 2. View: ‘raport‘ (reconstructs legacy ‘raport‘ format from ‘defect_reports‘)
-- ----------------------------------------------------------------------------
CREATE OR REPLACE VIEW ‘raport‘ AS
SELECT
  dr.‘id‘ AS ‘ID‘,
  p.‘name‘ AS ‘Product‘,
  dr.‘product_id‘ AS ‘product_id‘,
  dr.‘environment‘ AS ‘Environment‘,
  dr.‘reporter_user_id‘ AS ‘CustomerID‘,
  COALESCE(u.‘login‘, dr.‘guest_name‘, ’’) AS ‘Who‘,
  COALESCE(u.‘email‘, dr.‘guest_email‘, ’’) AS ‘Contactemail‘,
  DATE_FORMAT(dr.‘reported_at‘, ’%Y-%m-%d %H:%i’) AS ‘Whenreported‘,
  dr.‘title‘ AS ‘Title‘,
  dr.‘description‘ AS ‘Description‘,
  dr.‘discovered_by‘ AS ‘Discoveredby‘,
  dr.‘person_type‘ AS ‘Persontype‘,
  dr.‘occurred_on‘ AS ‘Occured‘,
  dr.‘location‘ AS ‘Location‘,
  dr.‘initial_severity‘ AS ‘Severity‘,
  CASE dr.‘initial_priority‘
    WHEN ’most_urgent’ THEN ’m_urgent’
    WHEN ’average’     THEN ’average’
    WHEN ’less_urgent’ THEN ’l_urgent’
  END AS ‘Priority‘,
  DATE(dr.‘reported_at‘) AS ‘Submitted‘,
```

```
  dr.‘additional_info‘ AS ‘Addinfo‘
FROM ‘defect_reports‘ dr
INNER JOIN ‘products‘ p
  ON p.‘id‘ = dr.‘product_id‘
LEFT JOIN ‘users‘ u
  ON u.‘id‘ = dr.‘reporter_user_id‘;

-- ------------------------------------------------------------------------
-- 3. View: ‘analyse‘ (reconstructs legacy ‘analyse‘ format from ‘defect_analyses‘)
--    Maps IEEE 1044-2009 ‘root_cause_type‘, ‘actual_effort_hours‘, ‘triaged_at‘,
--    and ‘resolved_at‘ seamlessly back to legacy 2004 column aliases.
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW ‘analyse‘ AS
SELECT
  da.‘report_id‘ AS ‘id‘,
  da.‘developer_description‘ AS ‘devdesc‘,
  COALESCE(da.‘root_cause_type‘, fc.‘code‘) AS ‘classification‘,
  da.‘date_originated‘ AS ‘dateoriginated‘,
  ph.‘phases_csv‘ AS ‘phase‘,
  da.‘activity_originated‘ AS ‘actoriginated‘,
  da.‘deliverable‘ AS ‘deliverable‘,
  da.‘what_to_do‘ AS ‘whattodo‘,
  DATE(dr.‘reported_at‘) AS ‘submitted‘,
  COALESCE(da.‘assigned_on‘, DATE(da.‘triaged_at‘)) AS ‘assigned‘,
  CASE da.‘uniqueness‘
    WHEN ’duplicate’ THEN ’Already submitted’
    WHEN ’new’       THEN ’New defect’
  END AS ‘uniqueness‘,
  CASE da.‘priority‘
    WHEN ’most_urgent’ THEN ’most urgent’
    WHEN ’average’     THEN ’average’
    WHEN ’less_urgent’ THEN ’less urgent’
  END AS ‘priority‘,
  da.‘severity‘ AS ‘severity‘,
  da.‘workaround‘ AS ‘workaround‘,
  da.‘proposed_fix‘ AS ‘proposedfix‘,
  da.‘status‘ AS ‘status‘,
  da.‘fix_description‘ AS ‘fixdesc‘,
  da.‘fix_started_on‘ AS ‘fixstarted‘,
  COALESCE(da.‘fix_completed_on‘, DATE(da.‘resolved_at‘)) AS ‘fixcomplete‘,
  da.‘fix_impact‘ AS ‘fiximpact‘,
  COALESCE(da.‘actual_effort_hours‘, da.‘fix_effort‘) AS ‘fixeffort‘,
  da.‘fix_cost‘ AS ‘fixcost‘,
  da.‘resolution_description‘ AS ‘resodesc‘,
  COALESCE(da.‘resolution_completed_on‘, DATE(da.‘resolved_at‘)) AS ‘resocomplete‘,
  da.‘resolution_closed_on‘ AS ‘resoclosed‘,
  da.‘cost_of_defect‘ AS ‘costofdefect‘,
  da.‘activity_found‘ AS ‘actfound‘
FROM ‘defect_analyses‘ da
INNER JOIN ‘defect_reports‘ dr
  ON dr.‘id‘ = da.‘report_id‘
LEFT JOIN ‘fault_classifications‘ fc
  ON fc.‘id‘ = da.‘classification_id‘
LEFT JOIN (
  SELECT
    ap.‘report_id‘,
    GROUP_CONCAT(sp.‘name‘ ORDER BY sp.‘id‘ SEPARATOR ’,’) AS ‘phases_csv‘
  FROM ‘analysis_phases‘ ap
  INNER JOIN ‘sdlc_phases‘ sp
    ON sp.‘id‘ = ap.‘phase_id‘
  GROUP BY ap.‘report_id‘
) ph
  ON ph.‘report_id‘ = da.‘report_id‘;


==========================================================================
```

SECTION 3.1 Domain Enums (backend/src/Domain/Enum/DomainEnums.php)
File:
C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Domain\Enum\DomainEnums.php
==========

```
<?php
declare(strict_types=1);

namespace DefMan\Domain\Enum;

/**
 * RBAC User Levels from menubar.php, form_usermanagement.php, form_apply.php
 */
enum UserLevel: int
{
    case APPLYING = 0; // Unapproved applicant (form_apply.php)
    case PUBLIC   = 1; // Public defect reporter (form_raportpub.php)
    case CUSTOMER = 2; // Authenticated customer (form_raport.php)
    case ANALYZER = 3; // Developer / QA triage & IEEE 1044-2009 / DORA telemetry
    case ADMIN    = 4; // Full administrator (form_usermanagement.php,
    form_productdetails.php)

    public function label(): string
    {
        return match ($this) {
            self::APPLYING => 'Applying',
            self::PUBLIC   => 'Public',
            self::CUSTOMER => 'Customer',
            self::ANALYZER => 'Analyzer',
            self::ADMIN    => 'Admin',
        };
    }
}

/**
 * Defect Severity from form_raport.php & form_analyze.php
 */
enum DefectSeverity: string
{
    case MINOR    = 'minor';
    case MAJOR    = 'major';
    case CRITICAL = 'critical';
}

/**
 * Defect Priority normalized across 'defect_reports.initial_priority' and
    'defect_analyses.priority'
 * Supports 3NF DB enum ('most_urgent', 'average', 'less_urgent') as well as legacy UI
    values
 */
enum DefectPriority: string
{
    case MOST_URGENT = 'm_urgent';
    case AVERAGE     = 'average';
    case LESS_URGENT = 'l_urgent';

    public function dbValue(): string
    {
        return match ($this) {
            self::MOST_URGENT => 'most_urgent',
            self::AVERAGE     => 'average',
            self::LESS_URGENT => 'less_urgent',
        };
    }

    public function analysisLabel(): string
```

```
    {
        return match ($this) {
            self::MOST_URGENT => 'most urgent',
            self::AVERAGE     => 'average',
            self::LESS_URGENT => 'less urgent',
        };
    }

    public static function fromInput(string $input): self
    {
        return match (strtolower(trim($input))) {
            'most_urgent', 'm_urgent', 'most urgent' => self::MOST_URGENT,
            'less_urgent', 'l_urgent', 'less urgent' => self::LESS_URGENT,
            default                                  => self::AVERAGE,
        };
    }
}

/**
 * Defect Status from form_analyze.php & PixiJS v8 Triage Lanes
 */
enum DefectStatus: string
{
    case SUBMITTED = 'submitted';
    case ASSIGNED  = 'assigned';
    case POSTPONED = 'postponed';
    case OPENED    = 'opened';
    case FIXED     = 'fixed';
    case RESOLVED  = 'resolved';
    case CLOSED    = 'closed';
}

/**
 * Person Type who discovered the defect (form_raport.php)
 */
enum PersonType: string
{
    case CUSTOMER  = 'Customer';
    case DEVELOPER = 'Developer';
    case TESTER    = 'Tester';
}

/**
 * Defect Uniqueness (form_analyze.php)
 * Normalized 3NF column uses ENUM('new', 'duplicate')
 */
enum Uniqueness: string
{
    case NEW_DEFECT        = 'New defect';
    case ALREADY_SUBMITTED = 'Already submitted';
    case CRITICAL          = 'Critical';

    public function dbValue(): string
    {
        return match ($this) {
            self::ALREADY_SUBMITTED => 'duplicate',
            self::NEW_DEFECT, self::CRITICAL => 'new',
        };
    }

    public static function fromInput(string $input): self
    {
        return match (strtolower(trim($input))) {
            'duplicate', 'already submitted' => self::ALREADY_SUBMITTED,
            'critical'                       => self::CRITICAL,
            default                          => self::NEW_DEFECT,
```

```
        };
    }
}

/**
 * Lifecycle Phase in which the defect originated (‘sdlc_phases‘)
 */
enum OriginPhase: string
{
    case REQUIREMENTS   = ’Requirements’;
    case DESIGN         = ’Design’;
    case IMPLEMENTATION = ’Implementation’;
    case TESTING        = ’Testing’;
    case DEPLOYMENT     = ’Deployment’;
    case MAINTENANCE    = ’Maintenance’;
}

/**
 * DORA / Flow Engineering Detection Environment (‘detected_environment‘)
 * Tracks Production Escape Rate (’PROD’ vs pre-production ’DEV’, ’CI’, ’STAGING’)
 */
enum DetectedEnvironment: string
{
    case DEV     = ’DEV’;
    case CI      = ’CI’;
    case STAGING = ’STAGING’;
    case PROD    = ’PROD’;

    public static function fromInput(?string $input): self
    {
        return match (strtoupper(trim((string) $input))) {
            ’DEV’, ’DEVELOPMENT’, ’LOCAL’ => self::DEV,
            ’CI’, ’BUILD’, ’PIPELINE’     => self::CI,
            ’STAGING’, ’STAGE’, ’QA’, ’UAT’ => self::STAGING,
            default                       => self::PROD,
        };
    }
}

/**
 * IEEE Std 1044-2009 Observed Symptom / Failure Effect Category (‘symptom_category‘)
 */
enum SymptomCategory: string
{
    case CRASH_HANG              = ’Application Crash / Hang’;
    case DATA_CORRUPTION         = ’Data Corruption / Loss’;
    case INCORRECT_OUTPUT        = ’Incorrect Calculation / Output’;
    case AUTH_SECURITY           = ’Authentication / Security Failure’;
    case UI_RENDERING            = ’UI / Rendering Anomaly’;
    case PERFORMANCE_DEGRADATION = ’Performance / Latency Degradation’;
    case INTEGRATION_FAILURE     = ’API / Integration Failure’;
    case OTHER                   = ’Other Symptom’;

    public static function fromInput(?string $input): self
    {
        $trimmed = trim((string) $input);
        if ($trimmed === ’’) {
            return self::OTHER;
        }
        foreach (self::cases() as $case) {
            if (strcasecmp($case->value, $trimmed) === 0 || strcasecmp($case->name,
    $trimmed) === 0) {
                return $case;
            }
        }
        return self::OTHER;
```

```
    }
}
```

```
================================================================================
```

SECTION 3.2 IEEE Std 1044-2009 Anomaly Enum (backend/src/Domain/Enum/IeeeFaultType.php)

```
File:
    C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Domain\Enum\IeeeFaultType.ph
    p
================================================================================

<?php
declare(strict_types=1);

namespace DefMan\Domain\Enum;

/**
 * IEEE Std 1044-2009 Standard Classification for Software Anomalies (Root Cause & Defect
    Type)
 * Replaces obsolete 1992 IEEE Fault & Cost multiplier classification while preserving
 * backward-compatible mapping for legacy 2004 records.
 */
enum IeeeFaultType: string
{
    case CODE_LOGIC            = 'Code / Logic';
    case COMPUTATIONAL         = 'Computational';
    case INTERFACE_INTEGRATION = 'Interface / Integration';
    case DATA_SCHEMA           = 'Data / Schema';
    case CONCURRENCY_TIMING    = 'Concurrency / Timing';
    case CONFIGURATION_ENV     = 'Configuration / Environment';
    case SECURITY_ACCESS       = 'Security / Access Control';
    case REQUIREMENTS_SPEC     = 'Requirements / Specification';
    case ARCHITECTURE_DESIGN   = 'Architecture / Design';
    case DOCUMENTATION         = 'Documentation';
    case REGRESSION            = 'Regression';
    case OTHER                 = 'Other';

    // Backward-compatibility aliases for legacy 1992 case references
    public const LOGIC                  = self::CODE_LOGIC;
    public const INTERFACE              = self::INTERFACE_INTEGRATION;
    public const DATA_HANDLING          = self::DATA_SCHEMA;
    public const DATA                   = self::DATA_SCHEMA;
    public const DOCUMENT_QUALITY       = self::DOCUMENTATION;
    public const ENHANCEMENT            = self::REQUIREMENTS_SPEC;
    public const CAUSED_BY_PREVIOUS_FIX = self::REGRESSION;

    /**
     * Hex color palette used by PixiJS v8 WebGPU DefectTriageCanvas & UI telemetry badges
     */
    public function hexColor(): int
    {
        return match ($this) {
            self::CODE_LOGIC            => 0xEF4444, // Red-500
            self::COMPUTATIONAL         => 0xF97316, // Orange-500
            self::INTERFACE_INTEGRATION => 0xEC4899, // Pink-500
            self::DATA_SCHEMA           => 0x8B5CF6, // Violet-500
            self::CONCURRENCY_TIMING    => 0x6366F1, // Indigo-500
            self::CONFIGURATION_ENV     => 0x0EA5E9, // Sky-500
            self::SECURITY_ACCESS       => 0xE11D48, // Rose-600
            self::REQUIREMENTS_SPEC     => 0x10B981, // Emerald-500
            self::ARCHITECTURE_DESIGN   => 0x14B8A6, // Teal-500
            self::DOCUMENTATION         => 0x06B6D4, // Cyan-500
            self::REGRESSION            => 0xF59E0B, // Amber-500
            self::OTHER                 => 0x64748B, // Slate-500
        };
    }
```

```
    /**
     * Detailed IEEE Std 1044-2009 root-cause sub-classifications
     *
     * @return list<string>
     */
    public function subcategories(): array
    {
        return match ($this) {
            self::CODE_LOGIC => [
                'Control flow or branch condition error',
                'Null / uninitialized reference dereference',
                'Boundary or off-by-one loop condition',
                'Cyclic state or recursion termination fault',
                'Missing exception or error handler',
            ],
            self::COMPUTATIONAL => [
                'Precision loss, rounding, or numeric truncation fault',
                'Incorrect formula, operator, or operand precedence',
                'Unit conversion or scaling mismatch',
                'Overflow / underflow in arithmetic calculation',
            ],
            self::INTERFACE_INTEGRATION => [
                'API contract or payload schema mismatch',
                'Session / protocol state desynchronization',
                'Module call parameter or return-type mismatch',
                'Upstream / downstream service timeout handling',
            ],
            self::DATA_SCHEMA => [
                'Database schema constraint or column type mismatch',
                'Incorrect data serialization or encoding',
                'Stale cache or out-of-bounds buffer access',
                'Missing transaction isolation or migration rule',
            ],
            self::CONCURRENCY_TIMING => [
                'Race condition or unsynchronized shared state',
                'Deadlock or lock contention under load',
                'Asynchronous event ordering violation',
            ],
            self::CONFIGURATION_ENV => [
                'Environment variable or secret misconfiguration',
                'Container / runtime dependency drift',
                'Infrastructure or deployment manifest fault',
            ],
            self::SECURITY_ACCESS => [
                'Authentication token or session lifecycle flaw',
                'RBAC authorization bypass or privilege escalation',
                'Input sanitization / injection vulnerability',
            ],
            self::REQUIREMENTS_SPEC => [
                'Ambiguous or incomplete acceptance criteria',
                'Unaccounted edge case in domain specification',
                'Behavioral change in product requirements',
            ],
            self::ARCHITECTURE_DESIGN => [
                'Coupling or component boundary design flaw',
                'Scalability or throughput bottleneck in design',
                'Missing idempotency or fallback architecture',
            ],
            self::DOCUMENTATION => [
                'Inaccurate API or operational runbook documentation',
                'Outdated configuration or upgrade instructions',
            ],
            self::REGRESSION => [
                'Regression introduced by a prior defect fix or refactor',
                'Unintended side effect of dependency or feature merge',
```

```
            ],
            self::OTHER => [
                'Unclassified or external environment anomaly',
            ],
        };
    }

    /**
     * Normalizes both IEEE Std 1044-2009 strings and legacy 1992 strings
     */
    public static function fromLegacyString(string $value): self
    {
        $trimmed = trim($value);
        foreach (self::cases() as $case) {
            if (strcasecmp($case->value, $trimmed) === 0 || strcasecmp($case->name,
    $trimmed) === 0) {
                return $case;
            }
        }

        return match (strtolower($trimmed)) {
            'logic'                                          => self::CODE_LOGIC,
            'computational'                                  => self::COMPUTATIONAL,
            'interface'                                      =>
    self::INTERFACE_INTEGRATION,
            'data handling', 'data'                          => self::DATA_SCHEMA,
            'documentation', 'document quality'              => self::DOCUMENTATION,
            'enhancement'                                    => self::REQUIREMENTS_SPEC,
            'caused by a prev. fix', 'caused by a previous fix' => self::REGRESSION,
            default                                          => self::OTHER,
        };
    }
}


================================================================================
SECTION 3.3 DORA & IEEE 1044-2009 Analysis Repository
    (backend/src/Repository/AnalysisRepository.php)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Repository\AnalysisReposit
    ory.php
================================================================================

<?php
declare(strict_types=1);

namespace DefMan\Repository;

use DefMan\Domain\Enum\DefectPriority;
use DefMan\Domain\Enum\DefectSeverity;
use DefMan\Domain\Enum\DefectStatus;
use DefMan\Domain\Enum\DetectedEnvironment;
use DefMan\Domain\Enum\IeeeFaultType;
use DefMan\Domain\Enum\SymptomCategory;
use DefMan\Domain\Enum\Uniqueness;
use PDO;

/**
 * IEEE Std 1044-2009 Anomaly Classification & DORA Flow Engineering Repository
 * (‘defect_analyses‘, ‘fault_classifications‘, ‘sdlc_phases‘, ‘analysis_phases‘ -- 3NF)
 *
 * Replaces obsolete 1992 phase-cost multiplier math with real aggregations for:
 * - MTTR (resolved_at - reported_at)
 * - Triage Lead Time (triaged_at - reported_at)
 * - Production Escape Rate (detected_environment = 'PROD')
 * - Regression Rate (is_regression = TRUE)
 * - Actual Effort Hours by IEEE 1044-2009 Root Cause
```

```
 */
final readonly class AnalysisRepository
{
    public function __construct(private PDO $pdo) {}

    /**
     * Resolves an IEEE Std 1044-2009 root cause / classification code to
    ‘fault_classifications.id‘
     */
    private function resolveClassificationId(string $code): int
    {
        $trimmed = trim(strip_tags($code));
        if ($trimmed === ’’) {
            $trimmed = IeeeFaultType::OTHER->value;
        }

        $stmt = $this->pdo->prepare(
            ’INSERT INTO ‘fault_classifications‘ (‘code‘) VALUES (:code)
             ON DUPLICATE KEY UPDATE ‘id‘ = LAST_INSERT_ID(‘id‘)’
        );
        $stmt->execute([’:code’ => $trimmed]);
        return (int) $this->pdo->lastInsertId();
    }

    /**
     * Syncs the 1NF ‘analysis_phases‘ junction table for a given ‘report_id‘
     *
     * @param list<string> $phases
     */
    private function syncAnalysisPhases(int $reportId, array $phases): void
    {
        $deleteStmt = $this->pdo->prepare(’DELETE FROM ‘analysis_phases‘ WHERE ‘report_id‘ =
    :report_id’);
        $deleteStmt->execute([’:report_id’ => $reportId]);

        $phaseUpsertStmt = $this->pdo->prepare(
            ’INSERT INTO ‘sdlc_phases‘ (‘name‘) VALUES (:name)
             ON DUPLICATE KEY UPDATE ‘id‘ = LAST_INSERT_ID(‘id‘)’
        );
        $junctionStmt = $this->pdo->prepare(
            ’INSERT IGNORE INTO ‘analysis_phases‘ (‘report_id‘, ‘phase_id‘) VALUES
    (:report_id, :phase_id)’
        );

        foreach ($phases as $rawPhase) {
            $phaseName = trim(strip_tags((string) $rawPhase));
            if ($phaseName === ’’) {
                continue;
            }
            $phaseUpsertStmt->execute([’:name’ => $phaseName]);
            $phaseId = (int) $this->pdo->lastInsertId();
            $junctionStmt->execute([
                ’:report_id’ => $reportId,
                ’:phase_id’  => $phaseId,
            ]);
        }
    }

    /**
     * Normalizes a datetime or date input string to MySQL DATETIME (’Y-m-d H:i:s’)
     */
    private function normalizeDateTime(?string $value, ?string $fallbackTime = null):
    ?string
    {
        $trimmed = trim((string) $value);
        if ($trimmed === ’’) {
```

```
            return $fallbackTime;
        }
        $normalized = str_replace('T', ' ', $trimmed);
        if (preg_match('/^\d{4}-\d{2}-\d{2}$/', $normalized) === 1) {
            return $normalized . ' 12:00:00';
        }
        if (preg_match('/^\d{4}-\d{2}-\d{2} \d{2}:\d{2}$/', $normalized) === 1) {
            return $normalized . ':00';
        }
        return $normalized;
    }

    /**
     * Returns the full Analyzer Queue with IEEE 1044-2009 classification and DORA lead-time
    metrics:
     * - triage_lead_time_hours (triaged_at - reported_at)
     * - mttr_hours (resolved_at - reported_at)
     * - detected_environment, is_regression, symptom_category, root_cause_type,
     *   introducing_ref, resolution_ref, actual_effort_hours
     */
    public function getAnalyzerQueue(?string $sort = 'all'): array
    {
        $whereSql = '';
        $params   = [];

        if (in_array($sort, ['critical', 'major', 'minor'], true)) {
            $whereSql        = 'WHERE COALESCE(da.`severity`, dr.`initial_severity`) =
    :sort';
            $params[':sort'] = $sort;
        } elseif (in_array($sort, ['most urgent', 'average', 'less urgent', 'most_urgent',
    'less_urgent'], true)) {
            $whereSql        = 'WHERE COALESCE(da.`priority`, dr.`initial_priority`) =
    :sort';
            $params[':sort'] = DefectPriority::fromInput($sort)->dbValue();
        } elseif (in_array($sort, ['submitted', 'assigned', 'postponed', 'opened', 'fixed',
    'resolved', 'closed'], true)) {
            $whereSql        = "WHERE COALESCE(da.`status`, 'submitted') = :sort";
            $params[':sort'] = $sort;
        } elseif (in_array($sort, ['DEV', 'CI', 'STAGING', 'PROD'], true)) {
            $whereSql        = "WHERE COALESCE(da.`detected_environment`,
    dr.`detected_environment`, 'PROD') = :sort";
            $params[':sort'] = $sort;
        }

        $orderBySql = match ($sort) {
            'date'  => 'dr.`reported_at` DESC, dr.`id` DESC',
            default => "FIELD(COALESCE(da.`priority`, dr.`initial_priority`, 'average'),
    'most_urgent', 'average', 'less_urgent') ASC, dr.`reported_at` DESC",
        };

        $sql = "
            SELECT
                dr.`id` AS `id`,
                p.`name` AS `product`,
                dr.`title` AS `title`,
                dr.`description` AS `report_description`,
                dr.`environment` AS `environment`,
                COALESCE(da.`detected_environment`, dr.`detected_environment`, 'PROD') AS
    `detected_environment`,
                COALESCE(da.`symptom_category`, dr.`symptom_category`, 'Other Symptom') AS
    `symptom_category`,
                COALESCE(da.`root_cause_type`, fc.`code`, 'Other') AS `root_cause_type`,
                COALESCE(da.`root_cause_type`, fc.`code`, 'Other') AS `classification`,
                COALESCE(da.`is_regression`, dr.`is_regression`, 0) AS `is_regression`,
                da.`introducing_ref` AS `introducing_ref`,
                da.`resolution_ref` AS `resolution_ref`,
```

```
            COALESCE(da.‘actual_effort_hours‘, da.‘fix_effort‘, 0.00) AS
    ‘actual_effort_hours‘,
            dr.‘reported_at‘ AS ‘reported_at‘,
            da.‘triaged_at‘ AS ‘triaged_at‘,
            da.‘resolved_at‘ AS ‘resolved_at‘,
            dr.‘occurred_on‘ AS ‘occured‘,
            COALESCE(u.‘email‘, dr.‘guest_email‘, ’’) AS ‘contact_email‘,
            COALESCE(u.‘login‘, dr.‘guest_name‘, ’’) AS ‘who‘,
            dr.‘initial_severity‘ AS ‘report_severity‘,
            CASE dr.‘initial_priority‘
                WHEN ’most_urgent’ THEN ’m_urgent’
                WHEN ’less_urgent’ THEN ’l_urgent’
                ELSE ’average’
            END AS ‘report_priority‘,
            DATE(dr.‘reported_at‘) AS ‘report_submitted‘,
            da.‘developer_description‘ AS ‘devdesc‘,
            da.‘date_originated‘ AS ‘dateoriginated‘,
            ph.‘phases_csv‘ AS ‘phase‘,
            da.‘activity_originated‘ AS ‘actoriginated‘,
            da.‘deliverable‘,
            da.‘what_to_do‘ AS ‘whattodo‘,
            DATE(dr.‘reported_at‘) AS ‘submitted‘,
            COALESCE(da.‘assigned_on‘, DATE(da.‘triaged_at‘)) AS ‘assigned‘,
            CASE da.‘uniqueness‘
                WHEN ’duplicate’ THEN ’Already submitted’
                ELSE ’New defect’
            END AS ‘uniqueness‘,
            da.‘duplicate_of_report_id‘,
            COALESCE(da.‘severity‘, dr.‘initial_severity‘) AS ‘severity‘,
            CASE COALESCE(da.‘priority‘, dr.‘initial_priority‘)
                WHEN ’most_urgent’ THEN ’most urgent’
                WHEN ’less_urgent’ THEN ’less urgent’
                ELSE ’average’
            END AS ‘priority‘,
            da.‘workaround‘,
            da.‘proposed_fix‘ AS ‘proposedfix‘,
            COALESCE(da.‘status‘, ’submitted’) AS ‘status‘,
            da.‘fix_description‘ AS ‘fixdesc‘,
            da.‘fix_started_on‘ AS ‘fixstarted‘,
            da.‘fix_completed_on‘ AS ‘fixcomplete‘,
            da.‘fix_impact‘ AS ‘fiximpact‘,
            COALESCE(da.‘actual_effort_hours‘, da.‘fix_effort‘, 0.00) AS ‘fixeffort‘,
            COALESCE(da.‘fix_cost‘, 0.00) AS ‘fixcost‘,
            da.‘resolution_description‘ AS ‘resodesc‘,
            COALESCE(da.‘resolution_completed_on‘, DATE(da.‘resolved_at‘)) AS
    ‘resocomplete‘,
            da.‘resolution_closed_on‘ AS ‘resoclosed‘,
            COALESCE(da.‘cost_of_defect‘, 0.00) AS ‘costofdefect‘,
            da.‘activity_found‘ AS ‘actfound‘,
            DATEDIFF(CURDATE(), DATE(dr.‘reported_at‘)) AS ‘days_open‘,
            CASE
                WHEN da.‘triaged_at‘ IS NOT NULL
                THEN ROUND(TIMESTAMPDIFF(SECOND, dr.‘reported_at‘, da.‘triaged_at‘) /
    3600.0, 2)
                WHEN da.‘assigned_on‘ IS NOT NULL
                THEN ROUND(DATEDIFF(da.‘assigned_on‘, DATE(dr.‘reported_at‘)) * 24.0, 2)
                ELSE NULL
            END AS ‘triage_lead_time_hours‘,
            CASE
                WHEN da.‘resolved_at‘ IS NOT NULL
                THEN ROUND(TIMESTAMPDIFF(SECOND, dr.‘reported_at‘, da.‘resolved_at‘) /
    3600.0, 2)
                WHEN da.‘resolution_completed_on‘ IS NOT NULL
                THEN ROUND(DATEDIFF(da.‘resolution_completed_on‘,
    DATE(dr.‘reported_at‘)) * 24.0, 2)
                ELSE NULL
```

```
            END AS ‘mttr_hours‘,
            CASE WHEN COALESCE(da.‘assigned_on‘, DATE(da.‘triaged_at‘)) IS NOT NULL
                 THEN DATEDIFF(COALESCE(da.‘assigned_on‘, DATE(da.‘triaged_at‘)),
    DATE(dr.‘reported_at‘))
                 ELSE NULL END AS ‘submitted_to_assigned_days‘,
            CASE WHEN da.‘fix_completed_on‘ IS NOT NULL
                 THEN DATEDIFF(da.‘fix_completed_on‘, DATE(dr.‘reported_at‘))
                 ELSE NULL END AS ‘submitted_to_fixed_days‘,
            CASE WHEN COALESCE(da.‘resolution_completed_on‘, DATE(da.‘resolved_at‘)) IS
    NOT NULL
                 THEN DATEDIFF(COALESCE(da.‘resolution_completed_on‘,
    DATE(da.‘resolved_at‘)), DATE(dr.‘reported_at‘))
                 ELSE NULL END AS ‘submitted_to_resolved_days‘,
            CASE WHEN da.‘fix_completed_on‘ IS NOT NULL AND COALESCE(da.‘assigned_on‘,
    DATE(da.‘triaged_at‘)) IS NOT NULL
                 THEN DATEDIFF(da.‘fix_completed_on‘, COALESCE(da.‘assigned_on‘,
    DATE(da.‘triaged_at‘)))
                 ELSE NULL END AS ‘assigned_to_fixed_days‘,
            CASE WHEN COALESCE(da.‘resolution_completed_on‘, DATE(da.‘resolved_at‘)) IS
    NOT NULL AND COALESCE(da.‘assigned_on‘, DATE(da.‘triaged_at‘)) IS NOT NULL
                 THEN DATEDIFF(COALESCE(da.‘resolution_completed_on‘,
    DATE(da.‘resolved_at‘)), COALESCE(da.‘assigned_on‘, DATE(da.‘triaged_at‘)))
                 ELSE NULL END AS ‘assigned_to_resolved_days‘
        FROM ‘defect_reports‘ dr
        INNER JOIN ‘products‘ p ON p.‘id‘ = dr.‘product_id‘
        LEFT JOIN ‘users‘ u ON u.‘id‘ = dr.‘reporter_user_id‘
        LEFT JOIN ‘defect_analyses‘ da ON da.‘report_id‘ = dr.‘id‘
        LEFT JOIN ‘fault_classifications‘ fc ON fc.‘id‘ = da.‘classification_id‘
        LEFT JOIN (
            SELECT
                ap.‘report_id‘,
                GROUP_CONCAT(sp.‘name‘ ORDER BY sp.‘id‘ SEPARATOR ’,’) AS ‘phases_csv‘
            FROM ‘analysis_phases‘ ap
            INNER JOIN ‘sdlc_phases‘ sp ON sp.‘id‘ = ap.‘phase_id‘
            GROUP BY ap.‘report_id‘
        ) ph ON ph.‘report_id‘ = dr.‘id‘
        {$whereSql}
        ORDER BY {$orderBySql}
    ";

    $stmt = $this->pdo->prepare($sql);
    $stmt->execute($params);
    $rows = $stmt->fetchAll();

    foreach ($rows as &$row) {
        $faultEnum = IeeeFaultType::fromLegacyString((string) $row[’root_cause_type’]);
        $row[’classification_color’] = sprintf(’#%06X’, $faultEnum->hexColor());
        $row[’is_regression’] = (bool) $row[’is_regression’];
        $row[’actual_effort_hours’] = (float) $row[’actual_effort_hours’];
        $row[’triage_lead_time_hours’] = $row[’triage_lead_time_hours’] !== null ?
(float) $row[’triage_lead_time_hours’] : null;
        $row[’mttr_hours’] = $row[’mttr_hours’] !== null ? (float) $row[’mttr_hours’] :
null;
        $row[’phases’] = !empty($row[’phase’]) ? explode(’,’, (string) $row[’phase’]) :
[];
    }

    return $rows;
}

/**
 * Computes real DORA / Flow Engineering & IEEE Std 1044-2009 aggregations:
 * - MTTR (Mean Time To Resolution in hours: resolved_at - reported_at)
 * - Triage Lead Time (Mean Time To Triage in hours: triaged_at - reported_at)
 * - Production Escape Rate (% of anomalies detected in ’PROD’)
 * - Regression Rate (% of anomalies with is_regression = 1)
```

```
     * - Actual Effort Hours broken down by IEEE 1044-2009 Root Cause
     */
    public function getDoraTelemetrySummary(): array
    {
        $summaryStmt = $this->pdo->query("
            SELECT
                COUNT(dr.‘id‘) AS ‘total_defects‘,
                SUM(CASE WHEN da.‘triaged_at‘ IS NOT NULL THEN 1 ELSE 0 END) AS
    ‘triaged_count‘,
                SUM(CASE WHEN da.‘resolved_at‘ IS NOT NULL THEN 1 ELSE 0 END) AS
    ‘resolved_count‘,
                ROUND(AVG(
                    CASE WHEN da.‘resolved_at‘ IS NOT NULL
                         THEN TIMESTAMPDIFF(SECOND, dr.‘reported_at‘, da.‘resolved_at‘) /
    3600.0
                         ELSE NULL
                    END
                ), 2) AS ‘mttr_hours‘,
                ROUND(AVG(
                    CASE WHEN da.‘triaged_at‘ IS NOT NULL
                         THEN TIMESTAMPDIFF(SECOND, dr.‘reported_at‘, da.‘triaged_at‘) /
    3600.0
                         ELSE NULL
                    END
                ), 2) AS ‘triage_lead_time_hours‘,
                SUM(CASE WHEN COALESCE(da.‘detected_environment‘, dr.‘detected_environment‘,
    ’PROD’) = ’PROD’ THEN 1 ELSE 0 END) AS ‘prod_escape_count‘,
                SUM(CASE WHEN COALESCE(da.‘detected_environment‘, dr.‘detected_environment‘,
    ’PROD’) = ’STAGING’ THEN 1 ELSE 0 END) AS ‘staging_count‘,
                SUM(CASE WHEN COALESCE(da.‘detected_environment‘, dr.‘detected_environment‘,
    ’PROD’) = ’CI’ THEN 1 ELSE 0 END) AS ‘ci_count‘,
                SUM(CASE WHEN COALESCE(da.‘detected_environment‘, dr.‘detected_environment‘,
    ’PROD’) = ’DEV’ THEN 1 ELSE 0 END) AS ‘dev_count‘,
                SUM(CASE WHEN COALESCE(da.‘is_regression‘, dr.‘is_regression‘, 0) = 1 THEN 1
    ELSE 0 END) AS ‘regression_count‘,
                ROUND(COALESCE(SUM(COALESCE(da.‘actual_effort_hours‘, da.‘fix_effort‘, 0)),
    0), 2) AS ‘total_actual_effort_hours‘,
                ROUND(COALESCE(AVG(COALESCE(da.‘actual_effort_hours‘, da.‘fix_effort‘, 0)),
    0), 2) AS ‘avg_actual_effort_hours‘
            FROM ‘defect_reports‘ dr
            LEFT JOIN ‘defect_analyses‘ da ON da.‘report_id‘ = dr.‘id‘
        ");
        $summary = $summaryStmt->fetch() ?: [];

        $totalDefects    = (int) ($summary[’total_defects’] ?? 0);
        $prodEscapeCount = (int) ($summary[’prod_escape_count’] ?? 0);
        $regressionCount = (int) ($summary[’regression_count’] ?? 0);

        $escapeRatePct = $totalDefects > 0
            ? round(($prodEscapeCount * 100.0) / $totalDefects, 1)
            : 0.0;
        $regressionRatePct = $totalDefects > 0
            ? round(($regressionCount * 100.0) / $totalDefects, 1)
            : 0.0;

        $breakdownStmt = $this->pdo->query("
            SELECT
                COALESCE(da.‘root_cause_type‘, fc.‘code‘, ’Unclassified’) AS
    ‘root_cause_type‘,
                COUNT(dr.‘id‘) AS ‘defect_count‘,
                ROUND(COALESCE(SUM(COALESCE(da.‘actual_effort_hours‘, da.‘fix_effort‘, 0)),
    0), 2) AS ‘total_effort_hours‘,
                ROUND(COALESCE(AVG(COALESCE(da.‘actual_effort_hours‘, da.‘fix_effort‘, 0)),
    0), 2) AS ‘avg_effort_hours‘,
                ROUND(AVG(
                    CASE WHEN da.‘resolved_at‘ IS NOT NULL
```

```
                    THEN TIMESTAMPDIFF(SECOND, dr.‘reported_at‘, da.‘resolved_at‘) /
3600.0
                    ELSE NULL
                END
            ), 2) AS ‘avg_mttr_hours‘,
            SUM(CASE WHEN COALESCE(da.‘is_regression‘, dr.‘is_regression‘, 0) = 1 THEN 1
ELSE 0 END) AS ‘regression_count‘,
            SUM(CASE WHEN COALESCE(da.‘detected_environment‘, dr.‘detected_environment‘,
’PROD’) = ’PROD’ THEN 1 ELSE 0 END) AS ‘prod_escape_count‘
        FROM ‘defect_reports‘ dr
        LEFT JOIN ‘defect_analyses‘ da ON da.‘report_id‘ = dr.‘id‘
        LEFT JOIN ‘fault_classifications‘ fc ON fc.‘id‘ = da.‘classification_id‘
        GROUP BY COALESCE(da.‘root_cause_type‘, fc.‘code‘, ’Unclassified’)
        ORDER BY ‘total_effort_hours‘ DESC, ‘defect_count‘ DESC
    ");
    $breakdownRows = $breakdownStmt->fetchAll();

    $rootCauseBreakdown = [];
    foreach ($breakdownRows as $row) {
        $rootCauseBreakdown[] = [
            ’root_cause_type’    => (string) $row[’root_cause_type’],
            ’defect_count’       => (int) $row[’defect_count’],
            ’total_effort_hours’ => (float) $row[’total_effort_hours’],
            ’avg_effort_hours’   => (float) $row[’avg_effort_hours’],
            ’avg_mttr_hours’     => $row[’avg_mttr_hours’] !== null ? (float)
$row[’avg_mttr_hours’] : null,
            ’regression_count’   => (int) $row[’regression_count’],
            ’prod_escape_count’  => (int) $row[’prod_escape_count’],
        ];
    }

    return [
        ’total_defects’               => $totalDefects,
        ’triaged_count’               => (int) ($summary[’triaged_count’] ?? 0),
        ’resolved_count’              => (int) ($summary[’resolved_count’] ?? 0),
        ’mttr_hours’                  => isset($summary[’mttr_hours’]) &&
$summary[’mttr_hours’] !== null ? (float) $summary[’mttr_hours’] : null,
        ’triage_lead_time_hours’      => isset($summary[’triage_lead_time_hours’]) &&
$summary[’triage_lead_time_hours’] !== null ? (float) $summary[’triage_lead_time_hours’]
: null,
        ’production_escape_rate_pct’  => $escapeRatePct,
        ’prod_escape_count’           => $prodEscapeCount,
        ’regression_rate_pct’         => $regressionRatePct,
        ’regression_count’            => $regressionCount,
        ’total_actual_effort_hours’   => (float) ($summary[’total_actual_effort_hours’]
?? 0.0),
        ’avg_actual_effort_hours’     => (float) ($summary[’avg_actual_effort_hours’] ??
0.0),
        ’environment_counts’          => [
            ’DEV’     => (int) ($summary[’dev_count’] ?? 0),
            ’CI’      => (int) ($summary[’ci_count’] ?? 0),
            ’STAGING’ => (int) ($summary[’staging_count’] ?? 0),
            ’PROD’    => $prodEscapeCount,
        ],
        ’root_cause_effort_breakdown’ => $rootCauseBreakdown,
    ];
}

/**
 * Upserts developer triage analysis & IEEE Std 1044-2009 classification
(‘defect_analyses‘ + ‘analysis_phases‘)
 */
public function upsertAnalysis(int $defectId, array $data): void
{
    $phaseList = [];
    if (is_array($data[’phases’] ?? null)) {
```

```
            $phaseList = array_values(array_filter(array_map('strval', $data['phases'])));
        } elseif (!empty($data['phase'])) {
            $phaseList = array_values(array_filter(array_map('trim', explode(',', (string)
    $data['phase']))));
        }

        $rawRootCause = !empty($data['classification_other'])
            ? (string) $data['classification_other']
            : (string) ($data['root_cause_type'] ?? $data['classification'] ??
    IeeeFaultType::CODE_LOGIC->value);

        $rootCauseType = IeeeFaultType::fromLegacyString($rawRootCause)->value;
        if (!empty($data['classification_other'])) {
            $rootCauseType = trim(strip_tags((string) $data['classification_other']));
        } elseif (!empty($data['root_cause_type']) &&
    IeeeFaultType::fromLegacyString((string) $data['root_cause_type']) ===
    IeeeFaultType::OTHER && strcasecmp((string) $data['root_cause_type'], 'Other') !== 0) {
            $rootCauseType = trim(strip_tags((string) $data['root_cause_type']));
        }

        $symptomCategory = !empty($data['symptom_category'])
            ? trim(strip_tags((string) $data['symptom_category']))
            : SymptomCategory::OTHER->value;

        $detectedEnv = DetectedEnvironment::fromInput((string)
    ($data['detected_environment'] ?? 'PROD'))->value;
        $isRegression = !empty($data['is_regression']) || $rootCauseType ===
    IeeeFaultType::REGRESSION->value ? 1 : 0;

        $introducingRef = !empty($data['introducing_ref'])
            ? trim(strip_tags((string) $data['introducing_ref']))
            : null;
        $resolutionRef = !empty($data['resolution_ref'])
            ? trim(strip_tags((string) $data['resolution_ref']))
            : null;
        $actualEffortHours = isset($data['actual_effort_hours']) &&
    $data['actual_effort_hours'] !== ''
            ? (float) $data['actual_effort_hours']
            : (isset($data['fixeffort']) && $data['fixeffort'] !== '' ? (float)
    $data['fixeffort'] : null);

        $assignedOn = !empty($data['assigned']) ? (string) $data['assigned'] :
    date('Y-m-d');
        $triagedAt  = $this->normalizeDateTime(
            $data['triaged_at'] ?? null,
            !empty($data['assigned']) ? $data['assigned'] . ' ' . date('H:i:s') :
    date('Y-m-d H:i:s')
        );

        $uniquenessEnum = Uniqueness::fromInput((string) ($data['uniqueness'] ?? 'New
    defect'));
        $uniquenessDb   = $uniquenessEnum->dbValue();

        $severity = $uniquenessEnum === Uniqueness::CRITICAL
            ? DefectSeverity::CRITICAL->value
            : DefectSeverity::from($data['severity'] ?? 'minor')->value;

        $priorityDb = DefectPriority::fromInput((string) ($data['priority'] ??
    'average'))->dbValue();
        $status     = DefectStatus::from($data['status'] ?? 'opened')->value;

        $duplicateOfReportId = !empty($data['duplicate_of_report_id'])
            ? (int) $data['duplicate_of_report_id']
            : null;

        $this->pdo->beginTransaction();
```

```
        try {
            $classificationId = $this->resolveClassificationId($rootCauseType);

            $stmt = $this->pdo->prepare(
                'INSERT INTO `defect_analyses`
                 (`report_id`, `developer_description`, `classification_id`,
    `symptom_category`,
                  `root_cause_type`, `detected_environment`, `is_regression`,
    `introducing_ref`,
                  `resolution_ref`, `actual_effort_hours`, `triaged_at`, `date_originated`,
                  `activity_originated`, `deliverable`, `what_to_do`, `assigned_on`,
    `uniqueness`,
                  `duplicate_of_report_id`, `priority`, `severity`, `workaround`,
    `proposed_fix`,
                  `status`, `activity_found`)
                 VALUES
                 (:report_id, :developer_description, :classification_id, :symptom_category,
                  :root_cause_type, :detected_environment, :is_regression, :introducing_ref,
                  :resolution_ref, :actual_effort_hours, :triaged_at, :date_originated,
                  :activity_originated, :deliverable, :what_to_do, :assigned_on,
    :uniqueness,
                  :duplicate_of_report_id, :priority, :severity, :workaround, :proposed_fix,
                  :status, :activity_found)
                 ON DUPLICATE KEY UPDATE
                  `developer_description`  = VALUES(`developer_description`),
                  `classification_id`      = VALUES(`classification_id`),
                  `symptom_category`       = VALUES(`symptom_category`),
                  `root_cause_type`        = VALUES(`root_cause_type`),
                  `detected_environment`   = VALUES(`detected_environment`),
                  `is_regression`          = VALUES(`is_regression`),
                  `introducing_ref`        = COALESCE(VALUES(`introducing_ref`),
    `introducing_ref`),
                  `resolution_ref`         = COALESCE(VALUES(`resolution_ref`),
    `resolution_ref`),
                  `actual_effort_hours`    = COALESCE(VALUES(`actual_effort_hours`),
    `actual_effort_hours`),
                  `triaged_at`             = COALESCE(VALUES(`triaged_at`), `triaged_at`),
                  `date_originated`        = VALUES(`date_originated`),
                  `activity_originated`    = VALUES(`activity_originated`),
                  `deliverable`            = VALUES(`deliverable`),
                  `what_to_do`             = VALUES(`what_to_do`),
                  `assigned_on`            = VALUES(`assigned_on`),
                  `uniqueness`             = VALUES(`uniqueness`),
                  `duplicate_of_report_id` = VALUES(`duplicate_of_report_id`),
                  `priority`               = VALUES(`priority`),
                  `severity`               = VALUES(`severity`),
                  `workaround`             = VALUES(`workaround`),
                  `proposed_fix`           = VALUES(`proposed_fix`),
                  `status`                 = VALUES(`status`),
                  `activity_found`         = VALUES(`activity_found`)'
            );

            $stmt->execute([
                ':report_id'              => $defectId,
                ':developer_description'  => trim(strip_tags((string) ($data['devdesc'] ??
    '' ))),
                ':classification_id'      => $classificationId,
                ':symptom_category'       => $symptomCategory,
                ':root_cause_type'        => $rootCauseType,
                ':detected_environment'   => $detectedEnv,
                ':is_regression'          => $isRegression,
                ':introducing_ref'        => $introducingRef,
                ':resolution_ref'         => $resolutionRef,
                ':actual_effort_hours'    => $actualEffortHours,
                ':triaged_at'             => $triagedAt,
                ':date_originated'        => !empty($data['dateoriginated']) ?
```

```
    $data['dateoriginated'] : date('Y-m-d'),
            ':activity_originated'    => trim(strip_tags((string)
($data['actoriginated'] ?? 'Development'))),
            ':deliverable'            => trim(strip_tags((string) ($data['deliverable']
?? ''))),
            ':what_to_do'             => trim(strip_tags((string) ($data['whattodo'] ??
''))),
            ':assigned_on'            => $assignedOn,
            ':uniqueness'             => $uniquenessDb,
            ':duplicate_of_report_id' => $duplicateOfReportId,
            ':priority'               => $priorityDb,
            ':severity'               => $severity,
            ':workaround'             => trim(strip_tags((string) ($data['workaround']
?? ''))),
            ':proposed_fix'           => trim(strip_tags((string) ($data['proposedfix']
?? ''))),
            ':status'                 => $status,
            ':activity_found'         => trim(strip_tags((string) ($data['actfound'] ??
'Automated / QA Triage'))),
        ]);

        $reportSyncStmt = $this->pdo->prepare(
            'UPDATE `defect_reports`
             SET `detected_environment` = :detected_environment,
                 `symptom_category`     = :symptom_category,
                 `is_regression`        = :is_regression
             WHERE `id` = :id'
        );
        $reportSyncStmt->execute([
            ':detected_environment' => $detectedEnv,
            ':symptom_category'     => $symptomCategory,
            ':is_regression'        => $isRegression,
            ':id'                   => $defectId,
        ]);

        $this->syncAnalysisPhases($defectId, $phaseList);
        $this->pdo->commit();
    } catch (\Throwable $e) {
        if ($this->pdo->inTransaction()) {
            $this->pdo->rollBack();
        }
        throw $e;
    }
}

/**
 * Upserts IEEE Std 1044-2009 root cause, resolution commit/build ref, actual effort
hours,
 * and DORA resolution timestamp (`defect_analyses`)
 */
public function upsertResolutionAndCost(int $defectId, array $data): void
{
    $status = isset($data['status'])
        ? DefectStatus::from((string) $data['status'])->value
        : DefectStatus::RESOLVED->value;

    $rootCauseType = !empty($data['root_cause_type'])
        ? trim(strip_tags((string) $data['root_cause_type']))
        : (!empty($data['classification']) ? trim(strip_tags((string)
$data['classification'])) : null);

    $classificationId = $rootCauseType !== null
        ? $this->resolveClassificationId($rootCauseType)
        : null;

    $actualEffortHours = isset($data['actual_effort_hours']) &&
```

```
    $data['actual_effort_hours'] !== ''
            ? (float) $data['actual_effort_hours']
            : (isset($data['fixeffort']) && $data['fixeffort'] !== '' ? (float)
    $data['fixeffort'] : null);

        $resolutionRef = !empty($data['resolution_ref'])
            ? trim(strip_tags((string) $data['resolution_ref']))
            : null;
        $introducingRef = !empty($data['introducing_ref'])
            ? trim(strip_tags((string) $data['introducing_ref']))
            : null;

        $isResolvedState = in_array($status, [
            DefectStatus::FIXED->value,
            DefectStatus::RESOLVED->value,
            DefectStatus::CLOSED->value,
        ], true);

        $resolvedAt = $this->normalizeDateTime(
            $data['resolved_at'] ?? null,
            !empty($data['resocomplete'])
                ? $data['resocomplete'] . ' ' . date('H:i:s')
                : ($isResolvedState ? date('Y-m-d H:i:s') : null)
        );

        $stmt = $this->pdo->prepare(
            'INSERT INTO `defect_analyses`
             (`report_id`, `status`, `classification_id`, `root_cause_type`,
    `resolution_ref`,
              `introducing_ref`, `actual_effort_hours`, `resolved_at`, `fix_description`,
              `fix_started_on`, `fix_completed_on`, `fix_impact`, `fix_effort`,
              `resolution_description`, `resolution_completed_on`, `resolution_closed_on`)
             VALUES
             (:report_id, :status, :classification_id, :root_cause_type, :resolution_ref,
              :introducing_ref, :actual_effort_hours, :resolved_at, :fix_description,
              :fix_started_on, :fix_completed_on, :fix_impact, :fix_effort,
              :resolution_description, :resolution_completed_on, :resolution_closed_on)
             ON DUPLICATE KEY UPDATE
              `status`                = VALUES(`status`),
              `classification_id`     = COALESCE(VALUES(`classification_id`),
    `classification_id`),
              `root_cause_type`       = COALESCE(VALUES(`root_cause_type`),
    `root_cause_type`),
              `resolution_ref`        = COALESCE(VALUES(`resolution_ref`),
    `resolution_ref`),
              `introducing_ref`       = COALESCE(VALUES(`introducing_ref`),
    `introducing_ref`),
              `actual_effort_hours`   = COALESCE(VALUES(`actual_effort_hours`),
    `actual_effort_hours`),
              `resolved_at`           = COALESCE(VALUES(`resolved_at`), `resolved_at`),
              `fix_description`       = COALESCE(VALUES(`fix_description`),
    `fix_description`),
              `fix_started_on`        = COALESCE(VALUES(`fix_started_on`),
    `fix_started_on`),
              `fix_completed_on`      = COALESCE(VALUES(`fix_completed_on`),
    `fix_completed_on`),
              `fix_impact`            = COALESCE(VALUES(`fix_impact`), `fix_impact`),
              `fix_effort`            = COALESCE(VALUES(`fix_effort`), `fix_effort`),
              `resolution_description`  = COALESCE(VALUES(`resolution_description`),
    `resolution_description`),
              `resolution_completed_on` = COALESCE(VALUES(`resolution_completed_on`),
    `resolution_completed_on`),
              `resolution_closed_on`    = COALESCE(VALUES(`resolution_closed_on`),
    `resolution_closed_on`)'
        );
```

```
        $stmt->execute([
            ':report_id'              => $defectId,
            ':status'                 => $status,
            ':classification_id'      => $classificationId,
            ':root_cause_type'        => $rootCauseType,
            ':resolution_ref'         => $resolutionRef,
            ':introducing_ref'        => $introducingRef,
            ':actual_effort_hours'    => $actualEffortHours,
            ':resolved_at'            => $resolvedAt,
            ':fix_description'        => isset($data['fixdesc']) ? trim(strip_tags((string)
    $data['fixdesc'])) : null,
            ':fix_started_on'         => !empty($data['fixstarted']) ? $data['fixstarted']
    : null,
            ':fix_completed_on'       => !empty($data['fixcomplete']) ?
    $data['fixcomplete'] : null,
            ':fix_impact'             => isset($data['fiximpact']) ?
    trim(strip_tags((string) $data['fiximpact'])) : null,
            ':fix_effort'             => $actualEffortHours,
            ':resolution_description'  => isset($data['resodesc']) ?
    trim(strip_tags((string) $data['resodesc'])) : null,
            ':resolution_completed_on' => !empty($data['resocomplete']) ?
    $data['resocomplete'] : ($resolvedAt !== null ? substr($resolvedAt, 0, 10) : null),
            ':resolution_closed_on'    => !empty($data['resoclosed']) ? $data['resoclosed']
    : null,
        ]);

        if (isset($data['is_regression'])) {
            $regFlag = !empty($data['is_regression']) ? 1 : 0;
            $regStmt = $this->pdo->prepare(
                'UPDATE `defect_analyses` da
                 INNER JOIN `defect_reports` dr ON dr.`id` = da.`report_id`
                 SET da.`is_regression` = :reg1, dr.`is_regression` = :reg2
                 WHERE da.`report_id` = :id'
            );
            $regStmt->execute([':reg1' => $regFlag, ':reg2' => $regFlag, ':id' =>
    $defectId]);
        }
    }

    /**
     * Deletes analysis entry only (db.php?function=deleteanalyze)
     */
    public function delete(int $defectId): bool
    {
        $stmt = $this->pdo->prepare('DELETE FROM `defect_analyses` WHERE `report_id` =
    :report_id');
        $stmt->execute([':report_id' => $defectId]);
        return $stmt->rowCount() > 0;
    }
}
```

```
================================================================================
SECTION 3.4 Defect Report Repository (backend/src/Repository/DefectRepository.php)
File:
    C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Repository\DefectRepository.
    php
================================================================================
```

```
<?php
declare(strict_types=1);

namespace DefMan\Repository;

use DefMan\Domain\Enum\DefectPriority;
use DefMan\Domain\Enum\DefectSeverity;
```

```
use DefMan\Domain\Enum\DetectedEnvironment;
use DefMan\Domain\Enum\PersonType;
use PDO;

/**
 * Defect Report Repository (‘defect_reports‘ + 3NF IEEE 1044-2009 / DORA joins)
 * Refactored from: form_raport.php, form_raportpub.php, form_raportlist.php,
 *                  form_list.php, form_defrap.php, db.php (deleteraport)
 */
final readonly class DefectRepository
{
    public function __construct(private PDO $pdo) {}

    /**
     * Resolves a product name to ‘products.id‘ (inserting if not yet cataloged)
     */
    private function resolveProductId(string $productName): int
    {
        $trimmed = trim(strip_tags($productName));
        if ($trimmed === ’’) {
            $trimmed = ’Unknown Product’;
        }

        $stmt = $this->pdo->prepare(
            ’INSERT INTO ‘products‘ (‘name‘) VALUES (:name)
             ON DUPLICATE KEY UPDATE ‘id‘ = LAST_INSERT_ID(‘id‘)’
        );
        $stmt->execute([’:name’ => $trimmed]);
        return (int) $this->pdo->lastInsertId();
    }

    /**
     * Multi-column sorting & filtering by severity, priority, status, product, and
    customer.
     * Includes IEEE 1044-2009 anomaly classification and DORA telemetry fields.
     */
    public function findFiltered(
        ?string $severity = null,
        ?string $priority = null,
        ?string $status = null,
        ?string $product = null,
        ?int $customerId = null,
        string $sortBy = ’date’
    ): array {
        $conditions = [];
        $params     = [];

        if ($severity !== null && $severity !== ’’) {
            $conditions[]        = ’dr.‘initial_severity‘ = :severity’;
            $params[’:severity’] = $severity;
        }

        if ($priority !== null && $priority !== ’’) {
            $normalizedPriority  = DefectPriority::fromInput($priority)->dbValue();
            $conditions[]        = ’dr.‘initial_priority‘ = :priority’;
            $params[’:priority’] = $normalizedPriority;
        }

        if ($status !== null && $status !== ’’) {
            if ($status === ’unassigned’) {
                $conditions[] = ’da.‘status‘ IS NULL’;
            } else {
                $conditions[]      = ’da.‘status‘ = :status’;
                $params[’:status’] = $status;
            }
        }
```

```
        if ($product !== null && $product !== '') {
            $conditions[]       = 'p.`name` = :product';
            $params[':product'] = $product;
        }

        if ($customerId !== null) {
            $conditions[]           = 'dr.`reporter_user_id` = :customer_id';
            $params[':customer_id'] = $customerId;
        }

        $whereSql = count($conditions) > 0 ? 'WHERE ' . implode(' AND ', $conditions) : '';

        $orderBySql = match ($sortBy) {
            'product'  => 'p.`name` ASC, dr.`id` DESC',
            'contact'  => "COALESCE(u.`login`, dr.`guest_name`, '') DESC, dr.`id` DESC",
            'severity' => "FIELD(dr.`initial_severity`, 'critical', 'major', 'minor') ASC,
    dr.`reported_at` DESC",
            'priority', 'all' => "FIELD(dr.`initial_priority`, 'most_urgent', 'average',
    'less_urgent') ASC, dr.`reported_at` DESC",
            default    => 'dr.`reported_at` DESC, dr.`id` DESC',
        };

        $sql = "
            SELECT
                dr.`id` AS `ID`,
                p.`name` AS `Product`,
                dr.`product_id`,
                dr.`environment` AS `Environment`,
                COALESCE(da.`detected_environment`, dr.`detected_environment`, 'PROD') AS
    `detected_environment`,
                COALESCE(da.`symptom_category`, dr.`symptom_category`) AS
    `symptom_category`,
                COALESCE(da.`is_regression`, dr.`is_regression`, 0) AS `is_regression`,
                dr.`reporter_user_id` AS `CustomerID`,
                COALESCE(u.`login`, dr.`guest_name`, '') AS `Who`,
                COALESCE(u.`email`, dr.`guest_email`, '') AS `Contactemail`,
                DATE_FORMAT(dr.`reported_at`, '%Y-%m-%d %H:%i') AS `Whenreported`,
                dr.`reported_at`,
                dr.`title` AS `Title`,
                dr.`description` AS `Description`,
                dr.`discovered_by` AS `Discoveredby`,
                dr.`person_type` AS `Persontype`,
                dr.`occurred_on` AS `Occured`,
                dr.`location` AS `Location`,
                dr.`initial_severity` AS `Severity`,
                CASE dr.`initial_priority`
                    WHEN 'most_urgent' THEN 'm_urgent'
                    WHEN 'less_urgent' THEN 'l_urgent'
                    ELSE 'average'
                END AS `Priority`,
                CASE dr.`initial_priority`
                    WHEN 'most_urgent' THEN 'most urgent'
                    WHEN 'less_urgent' THEN 'less urgent'
                    ELSE 'average'
                END AS `PriorityLabel`,
                DATE(dr.`reported_at`) AS `Submitted`,
                DATEDIFF(CURDATE(), DATE(dr.`reported_at`)) AS `days_since_submitted`,
                dr.`additional_info` AS `Addinfo`,
                CASE
                    WHEN dr.`reporter_user_id` IS NULL AND dr.`person_type` = 'Customer' AND
    dr.`guest_name` IS NOT NULL
                         AND NOT EXISTS (SELECT 1 FROM `users` u2 WHERE u2.`login` =
    dr.`guest_name`)
                    THEN 0
                    WHEN dr.`reporter_user_id` IS NOT NULL AND u.`id` IS NULL
```

```
                    THEN 1
                    ELSE 0
                END AS ‘is_deleted_user‘,
                COALESCE(da.‘root_cause_type‘, fc.‘code‘) AS ‘root_cause_type‘,
                COALESCE(da.‘root_cause_type‘, fc.‘code‘) AS ‘classification‘,
                COALESCE(da.‘status‘, ’submitted’) AS ‘analysis_status‘,
                da.‘severity‘ AS ‘developer_severity‘,
                CASE da.‘priority‘
                    WHEN ’most_urgent’ THEN ’most urgent’
                    WHEN ’less_urgent’ THEN ’less urgent’
                    WHEN ’average’     THEN ’average’
                    ELSE NULL
                END AS ‘developer_priority‘,
                da.‘introducing_ref‘,
                da.‘resolution_ref‘,
                COALESCE(da.‘actual_effort_hours‘, da.‘fix_effort‘) AS
    ‘actual_effort_hours‘,
                COALESCE(da.‘actual_effort_hours‘, da.‘fix_effort‘) AS ‘fixeffort‘,
                da.‘fix_cost‘ AS ‘fixcost‘,
                da.‘cost_of_defect‘ AS ‘costofdefect‘,
                da.‘triaged_at‘,
                da.‘resolved_at‘,
                COALESCE(da.‘assigned_on‘, DATE(da.‘triaged_at‘)) AS ‘assigned‘,
                da.‘fix_completed_on‘ AS ‘fixcomplete‘,
                COALESCE(da.‘resolution_completed_on‘, DATE(da.‘resolved_at‘)) AS
    ‘resocomplete‘,
                da.‘resolution_closed_on‘ AS ‘resoclosed‘
            FROM ‘defect_reports‘ dr
            INNER JOIN ‘products‘ p ON p.‘id‘ = dr.‘product_id‘
            LEFT JOIN ‘users‘ u ON u.‘id‘ = dr.‘reporter_user_id‘
            LEFT JOIN ‘defect_analyses‘ da ON da.‘report_id‘ = dr.‘id‘
            LEFT JOIN ‘fault_classifications‘ fc ON fc.‘id‘ = da.‘classification_id‘
            {$whereSql}
            ORDER BY {$orderBySql}
        ";

        $stmt = $this->pdo->prepare($sql);
        $stmt->execute($params);
        return $stmt->fetchAll();
    }

    public function findById(int $id): ?array
    {
        $stmt = $this->pdo->prepare(
            "SELECT
                dr.‘id‘ AS ‘ID‘,
                p.‘name‘ AS ‘Product‘,
                dr.‘product_id‘,
                dr.‘environment‘ AS ‘Environment‘,
                dr.‘detected_environment‘,
                dr.‘symptom_category‘,
                dr.‘is_regression‘,
                dr.‘reporter_user_id‘ AS ‘CustomerID‘,
                COALESCE(u.‘login‘, dr.‘guest_name‘, ’’) AS ‘Who‘,
                COALESCE(u.‘email‘, dr.‘guest_email‘, ’’) AS ‘Contactemail‘,
                DATE_FORMAT(dr.‘reported_at‘, ’%Y-%m-%d %H:%i’) AS ‘Whenreported‘,
                dr.‘reported_at‘,
                dr.‘title‘ AS ‘Title‘,
                dr.‘description‘ AS ‘Description‘,
                dr.‘discovered_by‘ AS ‘Discoveredby‘,
                dr.‘person_type‘ AS ‘Persontype‘,
                dr.‘occurred_on‘ AS ‘Occured‘,
                dr.‘location‘ AS ‘Location‘,
                dr.‘initial_severity‘ AS ‘Severity‘,
                CASE dr.‘initial_priority‘
                    WHEN ’most_urgent’ THEN ’m_urgent’
```

```
                WHEN 'less_urgent' THEN 'l_urgent'
                ELSE 'average'
            END AS `Priority`,
            CASE dr.`initial_priority`
                WHEN 'most_urgent' THEN 'most urgent'
                WHEN 'less_urgent' THEN 'less urgent'
                ELSE 'average'
            END AS `PriorityLabel`,
            DATE(dr.`reported_at`) AS `Submitted`,
            DATEDIFF(CURDATE(), DATE(dr.`reported_at`)) AS `days_since_submitted`,
            dr.`additional_info` AS `Addinfo`
         FROM `defect_reports` dr
         INNER JOIN `products` p ON p.`id` = dr.`product_id`
         LEFT JOIN `users` u ON u.`id` = dr.`reporter_user_id`
         WHERE dr.`id` = :id
         LIMIT 1"
    );
    $stmt->execute([':id' => $id]);
    $report = $stmt->fetch();
    if (!$report) {
        return null;
    }
    $report['is_regression'] = (bool) $report['is_regression'];

    $analysisStmt = $this->pdo->prepare(
        "SELECT
            da.`report_id` AS `id`,
            da.`developer_description` AS `devdesc`,
            COALESCE(da.`symptom_category`, dr.`symptom_category`) AS
`symptom_category`,
            COALESCE(da.`root_cause_type`, fc.`code`) AS `root_cause_type`,
            COALESCE(da.`root_cause_type`, fc.`code`) AS `classification`,
            COALESCE(da.`detected_environment`, dr.`detected_environment`, 'PROD') AS
`detected_environment`,
            COALESCE(da.`is_regression`, dr.`is_regression`, 0) AS `is_regression`,
            da.`introducing_ref`,
            da.`resolution_ref`,
            COALESCE(da.`actual_effort_hours`, da.`fix_effort`) AS
`actual_effort_hours`,
            da.`triaged_at`,
            da.`resolved_at`,
            CASE
                WHEN da.`triaged_at` IS NOT NULL
                THEN ROUND(TIMESTAMPDIFF(SECOND, dr.`reported_at`, da.`triaged_at`) /
3600.0, 2)
                ELSE NULL
            END AS `triage_lead_time_hours`,
            CASE
                WHEN da.`resolved_at` IS NOT NULL
                THEN ROUND(TIMESTAMPDIFF(SECOND, dr.`reported_at`, da.`resolved_at`) /
3600.0, 2)
                ELSE NULL
            END AS `mttr_hours`,
            da.`date_originated` AS `dateoriginated`,
            ph.`phases_csv` AS `phase`,
            da.`activity_originated` AS `actoriginated`,
            da.`deliverable`,
            da.`what_to_do` AS `whattodo`,
            DATE(dr.`reported_at`) AS `submitted`,
            COALESCE(da.`assigned_on`, DATE(da.`triaged_at`)) AS `assigned`,
            CASE da.`uniqueness`
                WHEN 'duplicate' THEN 'Already submitted'
                ELSE 'New defect'
            END AS `uniqueness`,
            da.`duplicate_of_report_id`,
            CASE da.`priority`
```

```
                WHEN 'most_urgent' THEN 'most urgent'
                WHEN 'less_urgent' THEN 'less urgent'
                ELSE 'average'
            END AS `priority`,
            da.`severity`,
            da.`workaround`,
            da.`proposed_fix` AS `proposedfix`,
            da.`status`,
            da.`fix_description` AS `fixdesc`,
            da.`fix_started_on` AS `fixstarted`,
            da.`fix_completed_on` AS `fixcomplete`,
            da.`fix_impact` AS `fiximpact`,
            COALESCE(da.`actual_effort_hours`, da.`fix_effort`) AS `fixeffort`,
            da.`fix_cost` AS `fixcost`,
            da.`resolution_description` AS `resodesc`,
            COALESCE(da.`resolution_completed_on`, DATE(da.`resolved_at`)) AS
    `resocomplete`,
            da.`resolution_closed_on` AS `resoclosed`,
            da.`cost_of_defect` AS `costofdefect`,
            da.`activity_found` AS `actfound`
        FROM `defect_analyses` da
        INNER JOIN `defect_reports` dr ON dr.`id` = da.`report_id`
        LEFT JOIN `fault_classifications` fc ON fc.`id` = da.`classification_id`
        LEFT JOIN (
            SELECT
                ap.`report_id`,
                GROUP_CONCAT(sp.`name` ORDER BY sp.`id` SEPARATOR ',') AS `phases_csv`
            FROM `analysis_phases` ap
            INNER JOIN `sdlc_phases` sp ON sp.`id` = ap.`phase_id`
            WHERE ap.`report_id` = :report_id
            GROUP BY ap.`report_id`
        ) ph ON ph.`report_id` = da.`report_id`
        WHERE da.`report_id` = :id
        LIMIT 1"
    );
    $analysisStmt->execute([':id' => $id, ':report_id' => $id]);
    $analysis = $analysisStmt->fetch();
    if ($analysis) {
        $analysis['is_regression'] = (bool) $analysis['is_regression'];
        $analysis['actual_effort_hours'] = $analysis['actual_effort_hours'] !== null ?
    (float) $analysis['actual_effort_hours'] : null;
        $analysis['triage_lead_time_hours'] = $analysis['triage_lead_time_hours'] !==
    null ? (float) $analysis['triage_lead_time_hours'] : null;
        $analysis['mttr_hours'] = $analysis['mttr_hours'] !== null ? (float)
    $analysis['mttr_hours'] : null;
        $analysis['phases'] = !empty($analysis['phase'])
            ? explode(',', (string) $analysis['phase'])
            : [];
        $report['analysis'] = $analysis;
    } else {
        $report['analysis'] = null;
    }

    return $report;
}

/**
 * Creates a new normalized defect report (`defect_reports`)
 */
public function create(array $data): int
{
    $severity    = DefectSeverity::from($data['severity'] ?? 'minor');
    $priority    = DefectPriority::fromInput($data['priority'] ?? 'average');
    $personType  = PersonType::from($data['persontype'] ?? 'Customer');
    $detectedEnv = DetectedEnvironment::fromInput($data['detected_environment'] ??
    null);
```

```
        $productId   = $this->resolveProductId((string) $data['product']);

        $reporterUserId = isset($data['customer_id']) && $data['customer_id'] !== null
            ? (int) $data['customer_id']
            : null;

        $guestName  = $reporterUserId === null
            ? substr(trim(strip_tags((string) ($data['who'] ?? 'Anonymous Guest'))), 0, 100)
            : null;
        $guestEmail = $reporterUserId === null
            ? trim(strip_tags((string) ($data['contact_email'] ?? '')))
            : null;

        $symptomCategory = !empty($data['symptom_category'])
            ? trim(strip_tags((string) $data['symptom_category']))
            : null;
        $isRegression = !empty($data['is_regression']) ? 1 : 0;

        $stmt = $this->pdo->prepare(
            'INSERT INTO `defect_reports`
             (`product_id`, `environment`, `detected_environment`, `symptom_category`,
    `is_regression`,
              `reporter_user_id`, `guest_name`, `guest_email`, `reported_at`,
              `title`, `description`, `discovered_by`, `person_type`, `occurred_on`,
    `location`,
              `initial_severity`, `initial_priority`, `additional_info`)
             VALUES
             (:product_id, :environment, :detected_environment, :symptom_category,
    :is_regression,
              :reporter_user_id, :guest_name, :guest_email, :reported_at,
              :title, :description, :discovered_by, :person_type, :occurred_on, :location,
              :initial_severity, :initial_priority, :additional_info)'
        );

        $stmt->execute([
            ':product_id'           => $productId,
            ':environment'          => trim(strip_tags((string) ($data['environment'] ??
    ''))),
            ':detected_environment' => $detectedEnv->value,
            ':symptom_category'     => $symptomCategory,
            ':is_regression'        => $isRegression,
            ':reporter_user_id'     => $reporterUserId,
            ':guest_name'           => $guestName,
            ':guest_email'          => $guestEmail,
            ':reported_at'          => date('Y-m-d H:i:s'),
            ':title'                => trim(strip_tags((string) $data['title'])),
            ':description'          => trim(strip_tags((string) $data['description'])),
            ':discovered_by'        => trim(strip_tags((string) ($data['discovered_by'] ??
    ''))),
            ':person_type'          => $personType->value,
            ':occurred_on'          => $data['occured'],
            ':location'             => trim(strip_tags((string) ($data['location'] ?? ''))),
            ':initial_severity'     => $severity->value,
            ':initial_priority'     => $priority->dbValue(),
            ':additional_info'      => trim(strip_tags((string) ($data['addinfo'] ?? ''))),
        ]);

        return (int) $this->pdo->lastInsertId();
    }

    /**
     * Updates an existing defect report (`defect_reports`)
     */
    public function update(int $id, array $data): bool
    {
        $severity    = DefectSeverity::from($data['severity']);
```

```
        $priority    = DefectPriority::fromInput($data['priority']);
        $personType  = PersonType::from($data['persontype']);
        $detectedEnv = DetectedEnvironment::fromInput($data['detected_environment'] ??
    null);
        $productId   = $this->resolveProductId((string) $data['product']);

        $symptomCategory = isset($data['symptom_category']) && $data['symptom_category'] !==
    ''
            ? trim(strip_tags((string) $data['symptom_category']))
            : null;
        $isRegression = !empty($data['is_regression']) ? 1 : 0;

        $stmt = $this->pdo->prepare(
            'UPDATE `defect_reports` SET
                `product_id`           = :product_id,
                `environment`          = :environment,
                `detected_environment` = :detected_environment,
                `symptom_category`     = COALESCE(:symptom_category, `symptom_category`),
                `is_regression`        = :is_regression,
                `title`                = :title,
                `description`          = :description,
                `discovered_by`        = :discovered_by,
                `person_type`          = :person_type,
                `occurred_on`          = :occurred_on,
                `location`             = :location,
                `initial_severity`     = :initial_severity,
                `initial_priority`     = :initial_priority,
                `additional_info`      = :additional_info
             WHERE `id` = :id'
        );

        $stmt->execute([
            ':product_id'           => $productId,
            ':environment'          => trim(strip_tags((string) ($data['environment'] ??
    '')),
            ':detected_environment' => $detectedEnv->value,
            ':symptom_category'     => $symptomCategory,
            ':is_regression'        => $isRegression,
            ':title'                => trim(strip_tags((string) $data['title'])),
            ':description'          => trim(strip_tags((string) $data['description'])),
            ':discovered_by'        => trim(strip_tags((string) ($data['discovered_by'] ??
    '')),
            ':person_type'          => $personType->value,
            ':occurred_on'          => $data['occured'],
            ':location'             => trim(strip_tags((string) ($data['location'] ?? ''))),
            ':initial_severity'     => $severity->value,
            ':initial_priority'     => $priority->dbValue(),
            ':additional_info'      => trim(strip_tags((string) ($data['addinfo'] ?? ''))),
            ':id'                   => $id,
        ]);

        return $stmt->rowCount() >= 0;
    }

    /**
     * Deletes a defect report and cascades to `defect_analyses` & `analysis_phases`
     */
    public function delete(int $id): bool
    {
        $stmt = $this->pdo->prepare('DELETE FROM `defect_reports` WHERE `id` = :id');
        $stmt->execute([':id' => $id]);
        return $stmt->rowCount() > 0;
    }
}
```

```
================================================================================
SECTION 3.5 Defect & Telemetry Controller (backend/src/Http/Controller/DefectController.php)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Http\Controller\DefectCont
    roller.php
================================================================================

<?php
declare(strict_types=1);

namespace DefMan\Http\Controller;

use DefMan\Domain\Enum\UserLevel;
use DefMan\Http\Middleware\RbacMiddleware;
use DefMan\Repository\AnalysisRepository;
use DefMan\Repository\DefectRepository;

/**
 * Defect & IEEE Std 1044-2009 / DORA Telemetry Controller
 * Refactored from: form_raportpub.php, form_raport.php, form_raportlist.php,
 *                  form_list.php, form_defrap.php, form_alist.php,
 *                  form_analyze.php, form_analyzed.php, form_deffix.php, form_defreso.php
 */
final readonly class DefectController
{
    public function __construct(
        private DefectRepository $defectRepository,
        private AnalysisRepository $analysisRepository,
        private RbacMiddleware $rbac
    ) {}

    /**
     * POST /api/defects/public (Refactored from form_raportpub.php -- Level 1 Public)
     */
    public function createPublicReport(array $body): array
    {
        $this->rbac->requireLevel(UserLevel::PUBLIC);

        $errors = $this->validateReportFields($body, isPublic: true);
        if (count($errors) > 0) {
            http_response_code(422);
            return ['errors' => $errors];
        }

        $id = $this->defectRepository->create([
            'product'              => $body['product'],
            'environment'          => $body['environment'] ?? '',
            'detected_environment' => $body['detected_environment'] ?? 'PROD',
            'symptom_category'     => $body['symptom_category'] ?? null,
            'is_regression'        => !empty($body['is_regression']),
            'customer_id'          => null,
            'who'                  => $body['who'],
            'contact_email'        => $body['contact_email'],
            'title'                => $body['title'],
            'description'          => $body['description'],
            'discovered_by'        => $body['discovered_by'] ?? '',
            'persontype'           => $body['persontype'],
            'occured'              => $body['occured'],
            'location'             => $body['location'] ?? '',
            'severity'             => $body['severity'],
            'priority'             => $body['priority'],
            'addinfo'              => $body['addinfo'] ?? '',
        ]);

        http_response_code(201);
        return [
            'id'      => $id,
```

```
            'message' => 'Report has been stored to the database',
        ];
    }

    /**
     * POST /api/defects (Refactored from form_raport.php -- Level 2+ Customer)
     */
    public function createCustomerReport(array $body): array
    {
        $user = $this->rbac->requireLevel(UserLevel::CUSTOMER);

        $errors = $this->validateReportFields($body, isPublic: false);
        if (count($errors) > 0) {
            http_response_code(422);
            return ['errors' => $errors];
        }

        $id = $this->defectRepository->create([
            'product'              => $body['product'],
            'environment'          => $body['environment'] ?? '',
            'detected_environment' => $body['detected_environment'] ?? 'PROD',
            'symptom_category'     => $body['symptom_category'] ?? null,
            'is_regression'        => !empty($body['is_regression']),
            'customer_id'          => (int) $user['id'],
            'who'                  => (string) $user['login'],
            'contact_email'        => (string) $user['email'],
            'title'                => $body['title'],
            'description'          => $body['description'],
            'discovered_by'        => $body['discovered_by'] ?? ($user['name'] ??
    $user['login']),
            'persontype'           => $body['persontype'],
            'occured'              => $body['occured'],
            'location'             => $body['location'] ?? '',
            'severity'             => $body['severity'],
            'priority'             => $body['priority'],
            'addinfo'              => $body['addinfo'] ?? '',
        ]);

        http_response_code(201);
        return [
            'id'      => $id,
            'message' => 'Report has been stored into the database',
        ];
    }

    /**
     * GET /api/defects (Refactored from form_raportlist.php & form_list.php)
     */
    public function listDefects(array $query): array
    {
        $user = $this->rbac->requireLevel(UserLevel::CUSTOMER);

        // Customers (Level 2) default to viewing their own defects unless Level 3+
    Analyzer/Admin
        $customerId = isset($query['customer_id']) && $query['customer_id'] !== ''
            ? (int) $query['customer_id']
            : ((int) $user['userlevel'] === UserLevel::CUSTOMER->value ? (int) $user['id'] :
    null);

        $defects = $this->defectRepository->findFiltered(
            severity: $query['severity'] ?? null,
            priority: $query['priority'] ?? null,
            status: $query['status'] ?? null,
            product: $query['product'] ?? null,
            customerId: $customerId,
            sortBy: (string) ($query['sort'] ?? 'date')
```

```
        );

        return [
            'count'   => count($defects),
            'defects' => $defects,
        ];
    }

    /**
     * GET /api/defects/{id} (Refactored from form_defrap.php + DORA telemetry summary)
     */
    public function getDefect(int $id): array
    {
        $this->rbac->requireLevel(UserLevel::CUSTOMER);
        $defect = $this->defectRepository->findById($id);

        if ($defect === null) {
            http_response_code(404);
            return ['error' => "Defect report #{$id} not found."];
        }

        $defect['telemetry'] = $this->analysisRepository->getDoraTelemetrySummary();

        return $defect;
    }

    /**
     * PUT /api/defects/{id} (Refactored from form_defrap.php update)
     */
    public function updateDefectReport(int $id, array $body): array
    {
        $this->rbac->requireLevel(UserLevel::ANALYZER);

        $errors = $this->validateReportFields($body, isPublic: false);
        if (count($errors) > 0) {
            http_response_code(422);
            return ['errors' => $errors];
        }

        $this->defectRepository->update($id, [
            'product'              => $body['product'],
            'environment'          => $body['environment'] ?? '',
            'detected_environment' => $body['detected_environment'] ?? 'PROD',
            'symptom_category'     => $body['symptom_category'] ?? null,
            'is_regression'        => !empty($body['is_regression']),
            'title'                => $body['title'],
            'description'          => $body['description'],
            'discovered_by'        => $body['discovered_by'] ?? '',
            'persontype'           => $body['persontype'],
            'occured'              => $body['occured'],
            'location'             => $body['location'] ?? '',
            'severity'             => $body['severity'],
            'priority'             => $body['priority'],
            'submitted'            => $body['submitted'] ?? date('Y-m-d'),
            'addinfo'              => $body['addinfo'] ?? '',
        ]);

        return ['message' => "You updated the defect raport #{$id} in the database."];
    }

    /**
     * DELETE /api/defects/{id} (Refactored from db.php?function=deleteraport)
     */
    public function deleteDefect(int $id): array
    {
        $this->rbac->requireLevel(UserLevel::ANALYZER);
```

```
        $this->defectRepository->delete($id);
        return ['message' => "Defect #{$id} and its analysis deleted."];
    }

    /**
     * GET /api/analyzer/queue (Refactored from form_alist.php & form_analyzed.php + DORA
    Telemetry)
     */
    public function getAnalyzerQueue(array $query): array
    {
        $this->rbac->requireLevel(UserLevel::ANALYZER);
        $items = $this->analysisRepository->getAnalyzerQueue($query['sort'] ?? 'all');

        return [
            'count'     => count($items),
            'queue'     => $items,
            'telemetry' => $this->analysisRepository->getDoraTelemetrySummary(),
        ];
    }

    /**
     * PUT /api/defects/{id}/analyze (IEEE 1044-2009 Anomaly Triage & Classification)
     */
    public function analyzeDefect(int $id, array $body): array
    {
        $this->rbac->requireLevel(UserLevel::ANALYZER);

        $errors = [];
        if (empty(trim((string) ($body['devdesc'] ?? '')))) {
            $errors[] = 'Empty Defect description!';
        }
        if (
            empty($body['root_cause_type']) &&
            empty($body['classification']) &&
            empty($body['classification_other'])
        ) {
            $errors[] = 'Empty IEEE 1044-2009 Root Cause / Classification!';
        }
        if (!empty($body['dateoriginated']) && !$this->isValidDate((string)
    $body['dateoriginated'])) {
            $errors[] = 'Invalid Date defect originated!';
        }
        if (!empty($body['assigned']) && !$this->isValidDate((string) $body['assigned'])) {
            $errors[] = 'Invalid Date assigned!';
        }
        if (empty(trim((string) ($body['proposedfix'] ?? '')))) {
            $errors[] = 'Empty Proposed fix!';
        }

        if (count($errors) > 0) {
            http_response_code(422);
            return ['errors' => $errors];
        }

        $this->analysisRepository->upsertAnalysis($id, $body);
        return [
            'message'   => "You updated analyze report #{$id} in the database.",
            'telemetry' => $this->analysisRepository->getDoraTelemetrySummary(),
        ];
    }

    /**
     * PUT /api/defects/{id}/resolve (IEEE 1044-2009 Root Cause, Resolution Ref & Actual
    Effort Hours)
     */
    public function resolveDefect(int $id, array $body): array
```

```
    {
        $this->rbac->requireLevel(UserLevel::ANALYZER);

        $this->analysisRepository->upsertResolutionAndCost($id, $body);
        return [
            'id'        => $id,
            'status'    => $body['status'] ?? 'resolved',
            'message'   => "You updated the analyze report #{$id} in the database.",
            'telemetry' => $this->analysisRepository->getDoraTelemetrySummary(),
        ];
    }

    /**
     * Validates report fields per form_raport.php & form_raportpub.php
     *
     * @return list<string>
     */
    private function validateReportFields(array $body, bool $isPublic): array
    {
        $errors = [];

        if ($isPublic) {
            $who   = trim((string) ($body['who'] ?? ''));
            $email = trim((string) ($body['contact_email'] ?? ''));
            if ($who === '') {
                $errors[] = 'Empty Who reported the problem!';
            } elseif (strlen($who) > 64) {
                $errors[] = 'Who reported the problem is too long!';
            }
            if ($email === '') {
                $errors[] = 'Empty Contact email!';
            } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
                $errors[] = 'Bad Contact email address!';
            }
        }

        if (empty(trim((string) ($body['title'] ?? '')))) {
            $errors[] = 'Empty Title!';
        }
        if (empty(trim((string) ($body['description'] ?? '')))) {
            $errors[] = 'Empty Description!';
        }
        if (empty($body['occured']) || !$this->isValidDate((string) $body['occured'])) {
            $errors[] = 'Invalid Date Problem occurred!';
        }
        if (empty($body['product'])) {
            $errors[] = 'Empty Product!';
        }
        if (empty($body['severity'])) {
            $errors[] = 'Empty Severity!';
        }
        if (empty($body['priority'])) {
            $errors[] = 'Empty Priority!';
        }
        if (empty($body['persontype'])) {
            $errors[] = 'Empty Type of person!';
        }

        return $errors;
    }

    /**
     * Preserves 2004 date bounds validation (1980 < year < 2100) from form_raport.php
     */
    private function isValidDate(string $dateStr): bool
    {
```

```
        $parts = explode('-', $dateStr);
        if (count($parts) !== 3) {
            return false;
        }
        [$year, $month, $day] = array_map('intval', $parts);
        return $year > 1980 && $year < 2100 && checkdate($month, $day, $year);
    }
}
```

==========

SECTION 3.6 Database Seeder (backend/src/Database/DatabaseSeeder.php)

File:
C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Database\DatabaseSeeder.php

==========

```
<?php
declare(strict_types=1);

namespace DefMan\Database;

use DefMan\Domain\Enum\DefectPriority;
use DefMan\Domain\Enum\DefectSeverity;
use DefMan\Domain\Enum\DefectStatus;
use DefMan\Domain\Enum\DetectedEnvironment;
use DefMan\Domain\Enum\IeeeFaultType;
use DefMan\Domain\Enum\OriginPhase;
use DefMan\Domain\Enum\PersonType;
use DefMan\Domain\Enum\SymptomCategory;
use DefMan\Domain\Enum\Uniqueness;
use DefMan\Domain\Enum\UserLevel;
use PDO;

/**
 * PDO Database Seeder (3NF Normalized Schema + IEEE Std 1044-2009 & DORA Telemetry)
 *
 * Test/Seed Account Rule:
 * - Admin / System placeholders: zuckerberg@fb.com, billg@microsoft.com
 * - Customer / Analyzer accounts: Sabrina (sabrina@hotmail.com), Julia (julia@hotmail.com)
 */
final readonly class DatabaseSeeder
{
    public function __construct(private PDO $pdo) {}

    public function run(): void
    {
        $this->pdo->beginTransaction();

        try {
            // 1. Seed Products (`products`)
            $products = [
                'MMF 1.5',
                'DefMan 2004 Core',
                'Katiska Auth Gateway 2.0',
                'Telemetry Engine v4',
            ];

            $productStmt = $this->pdo->prepare(
                'INSERT INTO `products` (`name`) VALUES (:name)
                 ON DUPLICATE KEY UPDATE `name` = VALUES(`name`)'
            );
            foreach ($products as $productName) {
                $productStmt->execute([':name' => $productName]);
            }

            $productMap = [];
```

```
        $rows = $this->pdo->query('SELECT `id`, `name` FROM `products`')->fetchAll();
        foreach ($rows as $row) {
            $productMap[$row['name']] = (int) $row['id'];
        }

        // 2. Seed SDLC Phases (`sdlc_phases` -- 1NF lookup table)
        $phaseStmt = $this->pdo->prepare(
            'INSERT INTO `sdlc_phases` (`name`) VALUES (:name)
             ON DUPLICATE KEY UPDATE `name` = VALUES(`name`)'
        );
        foreach (OriginPhase::cases() as $phaseCase) {
            $phaseStmt->execute([':name' => $phaseCase->value]);
        }

        $phaseMap = [];
        $phaseRows = $this->pdo->query('SELECT `id`, `name` FROM
    `sdlc_phases`')->fetchAll();
        foreach ($phaseRows as $row) {
            $phaseMap[$row['name']] = (int) $row['id'];
        }

        // 3. Seed IEEE Std 1044-2009 Anomaly & Root Cause Classifications
    (`fault_classifications`)
        $classificationStmt = $this->pdo->prepare(
            'INSERT INTO `fault_classifications` (`code`, `description`)
             VALUES (:code, :description)
             ON DUPLICATE KEY UPDATE `description` = VALUES(`description`)'
        );
        foreach (IeeeFaultType::cases() as $faultCase) {
            $subcategories = $faultCase->subcategories();
            $classificationStmt->execute([
                ':code'        => $faultCase->value,
                ':description' => $subcategories[0] ?? null,
            ]);
        }

        $classificationMap = [];
        $classRows = $this->pdo->query('SELECT `id`, `code` FROM
    `fault_classifications`')->fetchAll();
        foreach ($classRows as $row) {
            $classificationMap[$row['code']] = (int) $row['id'];
        }

        // 4. Seed Companies (`companies` -- 3NF lookup table) & Users (`users`)
        $users = [
            [
                'login'     => 'zuck',
                'password'  => password_hash('Admin#2026!Zuck', PASSWORD_ARGON2ID),
                'email'     => 'zuckerberg@fb.com',
                'userlevel' => UserLevel::ADMIN->value,
                'company'   => 'Meta Platforms',
                'phone'     => '+1-650-543-4800',
                'name'      => 'Mark Zuckerberg',
                'activate'  => 1,
                'online'    => true,
            ],
            [
                'login'     => 'billg',
                'password'  => password_hash('System#2026!Bill', PASSWORD_ARGON2ID),
                'email'     => 'billg@microsoft.com',
                'userlevel' => UserLevel::ADMIN->value,
                'company'   => 'Microsoft Corp',
                'phone'     => '+1-425-882-8080',
                'name'      => 'Bill Gates',
                'activate'  => 1,
                'online'    => false,
```

```
            ],
            [
                'login'     => 'julia',
                'password'  => password_hash('Analyze#2026!Julia', PASSWORD_ARGON2ID),
                'email'     => 'julia@hotmail.com',
                'userlevel' => UserLevel::ANALYZER->value,
                'company'   => 'DefMan QA Labs',
                'phone'     => '+358-40-555-0103',
                'name'      => 'Julia',
                'activate'  => 1,
                'online'    => true,
            ],
            [
                'login'     => 'sabrin',
                'password'  => password_hash('Cust#2026!Sabrina', PASSWORD_ARGON2ID),
                'email'     => 'sabrina@hotmail.com',
                'userlevel' => UserLevel::CUSTOMER->value,
                'company'   => 'Nordic Enterprise Oy',
                'phone'     => '+358-40-555-0102',
                'name'      => 'Sabrina',
                'activate'  => 1,
                'online'    => false,
            ],
        ];

        $companyStmt = $this->pdo->prepare(
            'INSERT INTO `companies` (`name`) VALUES (:name)
             ON DUPLICATE KEY UPDATE `id` = LAST_INSERT_ID(`id`)'
        );

        $userStmt = $this->pdo->prepare(
            'INSERT INTO `users` (`login`, `password`, `email`, `userlevel`,
`company_id`, `phone`, `name`, `activate`)
             VALUES (:login, :password, :email, :userlevel, :company_id, :phone, :name,
:activate)
             ON DUPLICATE KEY UPDATE
               `id`         = LAST_INSERT_ID(`id`),
               `userlevel`  = VALUES(`userlevel`),
               `company_id` = VALUES(`company_id`),
               `name`       = VALUES(`name`),
               `activate`   = VALUES(`activate`)'
        );

        $sessionStmt = $this->pdo->prepare(
            'INSERT IGNORE INTO `user_sessions` (`user_id`, `token_hash`, `ip_address`,
`user_agent`, `expires_at`)
             VALUES (:user_id, :token_hash, :ip_address, :user_agent, :expires_at)'
        );

        foreach ($users as $u) {
            $companyStmt->execute([':name' => $u['company']]);
            $companyId = (int) $this->pdo->lastInsertId();

            $userStmt->execute([
                ':login'      => $u['login'],
                ':password'   => $u['password'],
                ':email'      => $u['email'],
                ':userlevel'  => $u['userlevel'],
                ':company_id' => $companyId,
                ':phone'      => $u['phone'],
                ':name'       => $u['name'],
                ':activate'   => $u['activate'],
            ]);

            $userId = (int) $this->pdo->lastInsertId();
```

```
            if ($u['online']) {
                $sessionStmt->execute([
                    ':user_id'    => $userId,
                    ':token_hash' => hash('sha256', 'seed-session-' . $u['login']),
                    ':ip_address' => '127.0.0.1',
                    ':user_agent' => 'DefMan-2026-Seed-Session/1.0',
                    ':expires_at' => (new \DateTimeImmutable('+7 days'))->format('Y-m-d
H:i:s'),
                ]);
            }
        }

        $sabrinaId = (int) $this->pdo->query(
            "SELECT `id` FROM `users` WHERE `email` = 'sabrina@hotmail.com' LIMIT 1"
        )->fetchColumn();

        // 5. Seed Sample Defects & Analyses with IEEE Std 1044-2009 & DORA Telemetry
        $sampleDefects = [
            [
                'product_id'           => $productMap['MMF 1.5'],
                'environment'          => 'AMD Athlon XP 2000, WinXP / Production
Desktop Runtime',
                'detected_environment' => DetectedEnvironment::PROD->value,
                'symptom_category'     => SymptomCategory::CRASH_HANG->value,
                'is_regression'        => 1,
                'reporter_user_id'     => $sabrinaId,
                'guest_name'           => null,
                'guest_email'          => null,
                'reported_at'          => '2026-10-14 08:20:00',
                'title'                => 'PopUp Menu Cyclic Drag-and-Drop Crash',
                'description'          => 'When dragging a parent node onto its own
child entry in the popup menu tree, the client terminates abruptly without an error
dialog.',
                'discovered_by'        => 'Sabrina',
                'person_type'          => PersonType::CUSTOMER->value,
                'occurred_on'          => '2026-10-14',
                'location'             => 'settings form / popup.cpp',
                'initial_severity'     => DefectSeverity::CRITICAL->value,
                'initial_priority'     => DefectPriority::MOST_URGENT->dbValue(),
                'additional_info'      => 'Regression introduced after tree-reorder
patch in v1.5.2.',
                'analysis'             => [
                    'developer_description'   => 'Cyclic parent-to-child pointer
assignment triggers unbounded recursion in popup tree traversal.',
                    'classification_id'       =>
$classificationMap[IeeeFaultType::CODE_LOGIC->value],
                    'symptom_category'        => SymptomCategory::CRASH_HANG->value,
                    'root_cause_type'         => IeeeFaultType::CODE_LOGIC->value,
                    'detected_environment'    => DetectedEnvironment::PROD->value,
                    'is_regression'           => 1,
                    'introducing_ref'         => 'git:a4f9c21 (build-1502)',
                    'resolution_ref'          => 'git:e83a11f (pr-412 / v1.5.3)',
                    'actual_effort_hours'     => 3.50,
                    'triaged_at'              => '2026-10-14 09:35:00',
                    'resolved_at'             => '2026-10-14 15:50:00',
                    'date_originated'         => '2026-10-10',
                    'phases'                  => [OriginPhase::IMPLEMENTATION->value,
OriginPhase::TESTING->value],
                    'activity_originated'     => 'Tree node drag-and-drop refactoring',
                    'deliverable'             => 'settings.cpp, popup.cpp',
                    'what_to_do'              => 'Add ancestor cycle check before
committing node reparenting.',
                    'assigned_on'             => '2026-10-14',
                    'uniqueness'              => Uniqueness::CRITICAL->dbValue(),
                    'priority'                => DefectPriority::MOST_URGENT->dbValue(),
                    'severity'                => DefectSeverity::CRITICAL->value,
```

```
                    'workaround'             => 'Avoid dragging top-level menu groups
    into nested submenus.',
                    'proposed_fix'           => 'Guard reparenting call with
    isDescendantOf() check and unit test.',
                    'status'                 => DefectStatus::RESOLVED->value,
                    'fix_description'        => 'Added ancestor cycle guard and
    regression test suite for popup tree.',
                    'fix_started_on'         => '2026-10-14',
                    'fix_completed_on'       => '2026-10-14',
                    'fix_impact'             => 'MMF 1.5 settings.cpp, popup.cpp',
                    'fix_effort'             => 3.50,
                    'fix_cost'               => null,
                    'resolution_description' => 'Verified in staging and deployed
    hotfix v1.5.3.',
                    'resolution_completed_on' => '2026-10-14',
                    'resolution_closed_on'   => '2026-10-15',
                    'cost_of_defect'         => null,
                    'activity_found'         => 'Customer production telemetry',
                ],
            ],
            [
                'product_id'           => $productMap['DefMan 2004 Core'],
                'environment'          => 'MySQL 8.0 / PHP 8.3 Container Pipeline',
                'detected_environment' => DetectedEnvironment::CI->value,
                'symptom_category'     => SymptomCategory::DATA_CORRUPTION->value,
                'is_regression'        => 0,
                'reporter_user_id'     => $sabrinaId,
                'guest_name'           => null,
                'guest_email'          => null,
                'reported_at'          => '2026-10-18 09:15:00',
                'title'                => 'Fractional Effort Hours Truncated in Legacy
    MyISAM Column',
                'description'          => 'Fractional engineering effort hours get
    truncated when stored in legacy integer columns during data migration.',
                'discovered_by'        => 'Julia',
                'person_type'          => PersonType::DEVELOPER->value,
                'occurred_on'          => '2026-10-18',
                'location'             => 'database/02_migrate_data.sql',
                'initial_severity'     => DefectSeverity::MAJOR->value,
                'initial_priority'     => DefectPriority::MOST_URGENT->dbValue(),
                'additional_info'      => 'Detected by automated migration schema
    assertion.',
                'analysis'             => [
                    'developer_description'   => 'Legacy SMALLINT column definition
    truncates sub-hour DORA effort telemetry.',
                    'classification_id'       =>
    $classificationMap[IeeeFaultType::DATA_SCHEMA->value],
                    'symptom_category'        =>
    SymptomCategory::DATA_CORRUPTION->value,
                    'root_cause_type'         => IeeeFaultType::DATA_SCHEMA->value,
                    'detected_environment'    => DetectedEnvironment::CI->value,
                    'is_regression'           => 0,
                    'introducing_ref'         => 'git:3c08b12 (legacy-schema)',
                    'resolution_ref'          => 'git:7f19d04 (pr-419 / schema-3nf)',
                    'actual_effort_hours'     => 4.25,
                    'triaged_at'              => '2026-10-18 10:00:00',
                    'resolved_at'             => '2026-10-18 18:30:00',
                    'date_originated'         => '2026-10-15',
                    'phases'                  => [OriginPhase::DESIGN->value,
    OriginPhase::IMPLEMENTATION->value],
                    'activity_originated'     => 'Database schema definition',
                    'deliverable'             => 'database/01_schema_3nf.sql',
                    'what_to_do'              => 'Migrate effort tracking to
    DECIMAL(8,2) actual_effort_hours.',
                    'assigned_on'             => '2026-10-18',
                    'uniqueness'              => Uniqueness::NEW_DEFECT->dbValue(),
```

```
                    'priority'               => DefectPriority::MOST_URGENT->dbValue(),
                    'severity'               => DefectSeverity::MAJOR->value,
                    'workaround'             => 'None required after 3NF DDL
migration.',
                    'proposed_fix'           => 'Use DECIMAL(8,2) UNSIGNED for
actual_effort_hours in defect_analyses.',
                    'status'                 => DefectStatus::RESOLVED->value,
                    'fix_description'        => 'Migrated schema to InnoDB DECIMAL(8,2)
actual_effort_hours.',
                    'fix_started_on'         => '2026-10-18',
                    'fix_completed_on'       => '2026-10-18',
                    'fix_impact'             => '01_schema_3nf.sql,
AnalysisRepository.php',
                    'fix_effort'             => 4.25,
                    'fix_cost'               => null,
                    'resolution_description'  => 'CI migration suite verified zero
precision loss.',
                    'resolution_completed_on' => '2026-10-18',
                    'resolution_closed_on'    => '2026-10-19',
                    'cost_of_defect'         => null,
                    'activity_found'         => 'CI schema integration pipeline',
                ],
            ],
            [
                'product_id'           => $productMap['Katiska Auth Gateway 2.0'],
                'environment'          => 'Kubernetes Staging Cluster / Envoy 1.31',
                'detected_environment' => DetectedEnvironment::STAGING->value,
                'symptom_category'     => SymptomCategory::AUTH_SECURITY->value,
                'is_regression'        => 0,
                'reporter_user_id'     => $sabrinaId,
                'guest_name'           => null,
                'guest_email'          => null,
                'reported_at'          => '2026-10-21 11:40:00',
                'title'                => 'Stale Bearer Session Presence After Tab
Closure',
                'description'          => 'User session remains marked active if browser
tab closes without invoking explicit logout endpoint.',
                'discovered_by'        => 'Sabrina',
                'person_type'          => PersonType::TESTER->value,
                'occurred_on'          => '2026-10-21',
                'location'             => 'AuthController.php / user_sessions',
                'initial_severity'     => DefectSeverity::MINOR->value,
                'initial_priority'     => DefectPriority::AVERAGE->dbValue(),
                'additional_info'      => 'Observed during staging RBAC soak test.',
                'analysis'             => [
                    'developer_description'   => 'Session query lacked strict expires_at
window filter when computing online status.',
                    'classification_id'       =>
$classificationMap[IeeeFaultType::INTERFACE_INTEGRATION->value],
                    'symptom_category'        => SymptomCategory::AUTH_SECURITY->value,
                    'root_cause_type'         =>
IeeeFaultType::INTERFACE_INTEGRATION->value,
                    'detected_environment'    => DetectedEnvironment::STAGING->value,
                    'is_regression'           => 0,
                    'introducing_ref'         => 'git:5b82e10 (build-1810)',
                    'resolution_ref'          => 'git:9d04b7c (pr-427 / auth-ttl)',
                    'actual_effort_hours'     => 2.75,
                    'triaged_at'              => '2026-10-21 13:10:00',
                    'resolved_at'             => null,
                    'date_originated'         => '2026-10-12',
                    'phases'                  => [OriginPhase::DESIGN->value,
OriginPhase::IMPLEMENTATION->value],
                    'activity_originated'     => 'Session lifecycle implementation',
                    'deliverable'             => 'AuthController.php,
UserRepository.php',
                    'what_to_do'              => 'Enforce expires_at > NOW() check and
```

```
    sliding TTL cleanup.',
                    'assigned_on'             => '2026-10-21',
                    'uniqueness'              => Uniqueness::NEW_DEFECT->dbValue(),
                    'priority'                => DefectPriority::AVERAGE->dbValue(),
                    'severity'                => DefectSeverity::MINOR->value,
                    'workaround'              => 'Use explicit Logout action in top
    menubar.',
                    'proposed_fix'            => 'Prune expired rows in user_sessions
    and enforce short bearer TTL.',
                    'status'                  => DefectStatus::FIXED->value,
                    'fix_description'         => 'Added indexed expires_at filter and
    automatic token pruning.',
                    'fix_started_on'          => '2026-10-22',
                    'fix_completed_on'        => '2026-10-22',
                    'fix_impact'              => 'AuthController.php, user_sessions',
                    'fix_effort'              => 2.75,
                    'fix_cost'                => null,
                    'resolution_description'  => 'Awaiting final staging verification.',
                    'resolution_completed_on' => null,
                    'resolution_closed_on'    => null,
                    'cost_of_defect'          => null,
                    'activity_found'          => 'Staging integration testing',
                ],
            ],
            [
                'product_id'           => $productMap['Telemetry Engine v4'],
                'environment'          => 'WebGPU Chrome 130 / Linux Collector Node',
                'detected_environment' => DetectedEnvironment::PROD->value,
                'symptom_category'     =>
    SymptomCategory::PERFORMANCE_DEGRADATION->value,
                'is_regression'        => 1,
                'reporter_user_id'     => $sabrinaId,
                'guest_name'           => null,
                'guest_email'          => null,
                'reported_at'          => '2026-10-24 14:00:00',
                'title'                => 'Lock Contention Spike in Concurrent DORA
    Event Ingestion',
                'description'          => 'Simultaneous deployment webhook bursts cause
    450ms lock wait spikes on telemetry rollup rows.',
                'discovered_by'        => 'Julia',
                'person_type'          => PersonType::DEVELOPER->value,
                'occurred_on'          => '2026-10-24',
                'location'             => 'TelemetryIngestor.php',
                'initial_severity'     => DefectSeverity::MAJOR->value,
                'initial_priority'     => DefectPriority::MOST_URGENT->dbValue(),
                'additional_info'      => 'Triggered during multi-service release
    train.',
                'analysis'             => [
                    'developer_description'   => 'Unbatched row-level upserts contend on
    shared product counter index.',
                    'classification_id'       =>
    $classificationMap[IeeeFaultType::CONCURRENCY_TIMING->value],
                    'symptom_category'        =>
    SymptomCategory::PERFORMANCE_DEGRADATION->value,
                    'root_cause_type'         =>
    IeeeFaultType::CONCURRENCY_TIMING->value,
                    'detected_environment'    => DetectedEnvironment::PROD->value,
                    'is_regression'           => 1,
                    'introducing_ref'         => 'git:c91d4e8 (v4.1.0-rc2)',
                    'resolution_ref'          => 'git:f042a8b (pr-435 / v4.1.1)',
                    'actual_effort_hours'     => 5.50,
                    'triaged_at'              => '2026-10-24 14:45:00',
                    'resolved_at'             => '2026-10-25 02:15:00',
                    'date_originated'         => '2026-10-20',
                    'phases'                  => [OriginPhase::IMPLEMENTATION->value,
    OriginPhase::DEPLOYMENT->value],
```

```
                    'activity_originated'     => 'Webhook batch consumer refactor',
                    'deliverable'             => 'AnalysisRepository.php',
                    'what_to_do'              => 'Eliminate mutable rollup counters and
    compute DORA aggregates directly from indexed timestamps.',
                    'assigned_on'             => '2026-10-24',
                    'uniqueness'              => Uniqueness::NEW_DEFECT->dbValue(),
                    'priority'                => DefectPriority::MOST_URGENT->dbValue(),
                    'severity'                => DefectSeverity::MAJOR->value,
                    'workaround'              => 'Serialize webhook workers to
    concurrency=2.',
                    'proposed_fix'            => 'Use lock-free append-only timestamps
    with indexed TIMESTAMPDIFF aggregation.',
                    'status'                  => DefectStatus::OPENED->value,
                    'fix_description'         => 'Replaced counter table lock with
    indexed read aggregation.',
                    'fix_started_on'          => '2026-10-24',
                    'fix_completed_on'        => null,
                    'fix_impact'              => 'AnalysisRepository.php,
    01_schema_3nf.sql',
                    'fix_effort'              => 5.50,
                    'fix_cost'                => null,
                    'resolution_description'  => null,
                    'resolution_completed_on' => null,
                    'resolution_closed_on'    => null,
                    'cost_of_defect'          => null,
                    'activity_found'          => 'Production APM latency alert',
                ],
            ],
        ];

        $reportStmt = $this->pdo->prepare(
            'INSERT INTO `defect_reports`
             (`product_id`, `environment`, `detected_environment`, `symptom_category`,
    `is_regression`,
              `reporter_user_id`, `guest_name`, `guest_email`, `reported_at`,
              `title`, `description`, `discovered_by`, `person_type`, `occurred_on`,
    `location`,
              `initial_severity`, `initial_priority`, `additional_info`)
             VALUES
             (:product_id, :environment, :detected_environment, :symptom_category,
    :is_regression,
              :reporter_user_id, :guest_name, :guest_email, :reported_at,
              :title, :description, :discovered_by, :person_type, :occurred_on,
    :location,
              :initial_severity, :initial_priority, :additional_info)'
        );

        $analysisStmt = $this->pdo->prepare(
            'INSERT INTO `defect_analyses`
             (`report_id`, `developer_description`, `classification_id`,
    `symptom_category`,
              `root_cause_type`, `detected_environment`, `is_regression`,
    `introducing_ref`,
              `resolution_ref`, `actual_effort_hours`, `triaged_at`, `resolved_at`,
              `date_originated`, `activity_originated`, `deliverable`, `what_to_do`,
              `assigned_on`, `uniqueness`, `priority`, `severity`, `workaround`,
              `proposed_fix`, `status`, `fix_description`, `fix_started_on`,
    `fix_completed_on`,
              `fix_impact`, `fix_effort`, `fix_cost`, `resolution_description`,
              `resolution_completed_on`, `resolution_closed_on`, `cost_of_defect`,
    `activity_found`)
             VALUES
             (:report_id, :developer_description, :classification_id, :symptom_category,
              :root_cause_type, :detected_environment, :is_regression, :introducing_ref,
              :resolution_ref, :actual_effort_hours, :triaged_at, :resolved_at,
              :date_originated, :activity_originated, :deliverable, :what_to_do,
```

```
                :assigned_on, :uniqueness, :priority, :severity, :workaround,
                :proposed_fix, :status, :fix_description, :fix_started_on,
    :fix_completed_on,
                :fix_impact, :fix_effort, :fix_cost, :resolution_description,
                :resolution_completed_on, :resolution_closed_on, :cost_of_defect,
    :activity_found)'
            );

            $analysisPhaseStmt = $this->pdo->prepare(
                'INSERT IGNORE INTO `analysis_phases` (`report_id`, `phase_id`)
                 VALUES (:report_id, :phase_id)'
            );

            foreach ($sampleDefects as $defect) {
                $analysis = $defect['analysis'];
                unset($defect['analysis']);

                $reportStmt->execute([
                    ':product_id'           => $defect['product_id'],
                    ':environment'          => $defect['environment'],
                    ':detected_environment' => $defect['detected_environment'],
                    ':symptom_category'     => $defect['symptom_category'],
                    ':is_regression'        => $defect['is_regression'],
                    ':reporter_user_id'     => $defect['reporter_user_id'],
                    ':guest_name'           => $defect['guest_name'],
                    ':guest_email'          => $defect['guest_email'],
                    ':reported_at'          => $defect['reported_at'],
                    ':title'                => $defect['title'],
                    ':description'          => $defect['description'],
                    ':discovered_by'        => $defect['discovered_by'],
                    ':person_type'          => $defect['person_type'],
                    ':occurred_on'          => $defect['occurred_on'],
                    ':location'             => $defect['location'],
                    ':initial_severity'     => $defect['initial_severity'],
                    ':initial_priority'     => $defect['initial_priority'],
                    ':additional_info'      => $defect['additional_info'],
                ]);

                $reportId = (int) $this->pdo->lastInsertId();

                $analysisStmt->execute([
                    ':report_id'               => $reportId,
                    ':developer_description'   => $analysis['developer_description'],
                    ':classification_id'       => $analysis['classification_id'],
                    ':symptom_category'        => $analysis['symptom_category'],
                    ':root_cause_type'         => $analysis['root_cause_type'],
                    ':detected_environment'    => $analysis['detected_environment'],
                    ':is_regression'           => $analysis['is_regression'],
                    ':introducing_ref'         => $analysis['introducing_ref'],
                    ':resolution_ref'          => $analysis['resolution_ref'],
                    ':actual_effort_hours'     => $analysis['actual_effort_hours'],
                    ':triaged_at'              => $analysis['triaged_at'],
                    ':resolved_at'             => $analysis['resolved_at'],
                    ':date_originated'         => $analysis['date_originated'],
                    ':activity_originated'     => $analysis['activity_originated'],
                    ':deliverable'             => $analysis['deliverable'],
                    ':what_to_do'              => $analysis['what_to_do'],
                    ':assigned_on'             => $analysis['assigned_on'],
                    ':uniqueness'              => $analysis['uniqueness'],
                    ':priority'                => $analysis['priority'],
                    ':severity'                => $analysis['severity'],
                    ':workaround'              => $analysis['workaround'],
                    ':proposed_fix'            => $analysis['proposed_fix'],
                    ':status'                  => $analysis['status'],
                    ':fix_description'         => $analysis['fix_description'],
                    ':fix_started_on'          => $analysis['fix_started_on'],
```

```
                    ':fix_completed_on'        => $analysis['fix_completed_on'],
                    ':fix_impact'              => $analysis['fix_impact'],
                    ':fix_effort'              => $analysis['fix_effort'],
                    ':fix_cost'                => $analysis['fix_cost'],
                    ':resolution_description'  => $analysis['resolution_description'],
                    ':resolution_completed_on' => $analysis['resolution_completed_on'],
                    ':resolution_closed_on'    => $analysis['resolution_closed_on'],
                    ':cost_of_defect'          => $analysis['cost_of_defect'],
                    ':activity_found'          => $analysis['activity_found'],
                ]);

                foreach ($analysis['phases'] as $phaseName) {
                    if (isset($phaseMap[$phaseName])) {
                        $analysisPhaseStmt->execute([
                            ':report_id' => $reportId,
                            ':phase_id'  => $phaseMap[$phaseName],
                        ]);
                    }
                }
            }

            $this->pdo->commit();
        } catch (\Throwable $e) {
            if ($this->pdo->inTransaction()) {
                $this->pdo->rollBack();
            }
            throw $e;
        }
    }
}
```

```
================================================================================
SECTION 4.1 Frontend TypeScript Definitions (frontend/src/types/defman.ts)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\types\defman.ts
================================================================================
```

```
/**
 * IEEE Std 1044-2009 Standard Classification for Software Anomalies (Root Cause & Defect
    Type)
 * Replaces obsolete 1992 IEEE Fault & Cost Analytics while preserving legacy key
    compatibility
 */
export enum IeeeFaultType {
  CODE_LOGIC            = 'Code / Logic',
  COMPUTATIONAL         = 'Computational',
  INTERFACE_INTEGRATION = 'Interface / Integration',
  DATA_SCHEMA           = 'Data / Schema',
  CONCURRENCY_TIMING    = 'Concurrency / Timing',
  CONFIGURATION_ENV     = 'Configuration / Environment',
  SECURITY_ACCESS       = 'Security / Access Control',
  REQUIREMENTS_SPEC     = 'Requirements / Specification',
  ARCHITECTURE_DESIGN   = 'Architecture / Design',
  DOCUMENTATION         = 'Documentation',
  REGRESSION            = 'Regression',
  OTHER                 = 'Other',
}

export type Ieee1044RootCause = IeeeFaultType;

/**
 * IEEE Std 1044-2009 Observed Symptom / Failure Effect Classification
 */
export enum SymptomCategory {
  CRASH_HANG              = 'Application Crash / Hang',
  DATA_CORRUPTION         = 'Data Corruption / Loss',
```

```
  INCORRECT_OUTPUT        = 'Incorrect Calculation / Output',
  AUTH_SECURITY           = 'Authentication / Security Failure',
  UI_RENDERING            = 'UI / Rendering Anomaly',
  PERFORMANCE_DEGRADATION = 'Performance / Latency Degradation',
  INTEGRATION_FAILURE     = 'API / Integration Failure',
  OTHER                   = 'Other Symptom',
}

/**
 * DORA / Flow Engineering Detection Environment ('detected_environment')
 */
export enum DetectedEnvironment {
  DEV     = 'DEV',
  CI      = 'CI',
  STAGING = 'STAGING',
  PROD    = 'PROD',
}

export const IEEE_FAULT_HEX_COLORS: Record<string, number> = {
  [IeeeFaultType.CODE_LOGIC]:            0xef4444, // Red-500
  [IeeeFaultType.COMPUTATIONAL]:         0xf97316, // Orange-500
  [IeeeFaultType.INTERFACE_INTEGRATION]: 0xec4899, // Pink-500
  [IeeeFaultType.DATA_SCHEMA]:           0x8b5cf6, // Violet-500
  [IeeeFaultType.CONCURRENCY_TIMING]:    0x6366f1, // Indigo-500
  [IeeeFaultType.CONFIGURATION_ENV]:     0x0ea5e9, // Sky-500
  [IeeeFaultType.SECURITY_ACCESS]:       0xe11d48, // Rose-600
  [IeeeFaultType.REQUIREMENTS_SPEC]:     0x10b981, // Emerald-500
  [IeeeFaultType.ARCHITECTURE_DESIGN]:   0x14b8a6, // Teal-500
  [IeeeFaultType.DOCUMENTATION]:         0x06b6d4, // Cyan-500
  [IeeeFaultType.REGRESSION]:            0xf59e0b, // Amber-500
  [IeeeFaultType.OTHER]:                 0x64748b, // Slate-500
  // Legacy 1992 color fallbacks
  Logic:                                 0xef4444,
  Interface:                             0xec4899,
  'Data handling':                       0x8b5cf6,
  Data:                                  0x6366f1,
  'Document quality':                    0x14b8a6,
  Enhancement:                           0x10b981,
  'Caused by a prev. Fix':               0xf59e0b,
};

/**
 * IEEE Std 1044-2009 Root Cause Taxonomy
 */
export const IEEE_1044_2009_TAXONOMY: Record<IeeeFaultType, readonly string[]> = {
  [IeeeFaultType.CODE_LOGIC]: [
    'Control flow or branch condition error',
    'Null / uninitialized reference dereference',
    'Boundary or off-by-one loop condition',
    'Cyclic state or recursion termination fault',
    'Missing exception or error handler',
  ],
  [IeeeFaultType.COMPUTATIONAL]: [
    'Precision loss, rounding, or numeric truncation fault',
    'Incorrect formula, operator, or operand precedence',
    'Unit conversion or scaling mismatch',
    'Overflow / underflow in arithmetic calculation',
  ],
  [IeeeFaultType.INTERFACE_INTEGRATION]: [
    'API contract or payload schema mismatch',
    'Session / protocol state desynchronization',
    'Module call parameter or return-type mismatch',
    'Upstream / downstream service timeout handling',
  ],
  [IeeeFaultType.DATA_SCHEMA]: [
    'Database schema constraint or column type mismatch',
```

```
    'Incorrect data serialization or encoding',
    'Stale cache or out-of-bounds buffer access',
    'Missing transaction isolation or migration rule',
  ],
  [IeeeFaultType.CONCURRENCY_TIMING]: [
    'Race condition or unsynchronized shared state',
    'Deadlock or lock contention under load',
    'Asynchronous event ordering violation',
  ],
  [IeeeFaultType.CONFIGURATION_ENV]: [
    'Environment variable or secret misconfiguration',
    'Container / runtime dependency drift',
    'Infrastructure or deployment manifest fault',
  ],
  [IeeeFaultType.SECURITY_ACCESS]: [
    'Authentication token or session lifecycle flaw',
    'RBAC authorization bypass or privilege escalation',
    'Input sanitization / injection vulnerability',
  ],
  [IeeeFaultType.REQUIREMENTS_SPEC]: [
    'Ambiguous or incomplete acceptance criteria',
    'Unaccounted edge case in domain specification',
    'Behavioral change in product requirements',
  ],
  [IeeeFaultType.ARCHITECTURE_DESIGN]: [
    'Coupling or component boundary design flaw',
    'Scalability or throughput bottleneck in design',
    'Missing idempotency or fallback architecture',
  ],
  [IeeeFaultType.DOCUMENTATION]: [
    'Inaccurate API or operational runbook documentation',
    'Outdated configuration or upgrade instructions',
  ],
  [IeeeFaultType.REGRESSION]: [
    'Regression introduced by a prior defect fix or refactor',
    'Unintended side effect of dependency or feature merge',
  ],
  [IeeeFaultType.OTHER]: [
    'Unclassified or external environment anomaly',
  ],
};

// Backward-compatible export alias
export const IEEE_1992_TAXONOMY = IEEE_1044_2009_TAXONOMY;

export enum UserLevel {
  APPLYING = 0,
  PUBLIC   = 1,
  CUSTOMER = 2,
  ANALYZER = 3,
  ADMIN    = 4,
}

export enum DefectSeverity {
  MINOR    = 'minor',
  MAJOR    = 'major',
  CRITICAL = 'critical',
}

export enum DefectPriority {
  MOST_URGENT = 'm_urgent',
  AVERAGE     = 'average',
  LESS_URGENT = 'l_urgent',
}

export enum DefectStatus {
```

```
  SUBMITTED = 'submitted',
  ASSIGNED  = 'assigned',
  POSTPONED = 'postponed',
  OPENED    = 'opened',
  FIXED     = 'fixed',
  RESOLVED  = 'resolved',
  CLOSED    = 'closed',
}

export enum PersonType {
  CUSTOMER  = 'Customer',
  DEVELOPER = 'Developer',
  TESTER    = 'Tester',
}

export enum Uniqueness {
  NEW_DEFECT        = 'New defect',
  ALREADY_SUBMITTED = 'Already submitted',
  CRITICAL          = 'Critical',
}

export enum OriginPhase {
  REQUIREMENTS   = 'Requirements',
  DESIGN         = 'Design',
  IMPLEMENTATION = 'Implementation',
  TESTING        = 'Testing',
  DEPLOYMENT     = 'Deployment',
  MAINTENANCE    = 'Maintenance',
}

export interface AuthUser {
  id: number;
  login: string;
  email: string;
  userlevel: UserLevel;
  role?: string;
  name?: string | null;
  company?: string | null;
  phone?: string | null;
  activate?: number;
  logged?: number;
}

export interface Product {
  id: number;
  name: string;
  report_count?: number;
}

export interface RootCauseEffortBreakdownItem {
  root_cause_type: string;
  defect_count: number;
  total_effort_hours: number;
  avg_effort_hours: number;
  avg_mttr_hours: number | null;
  regression_count: number;
  prod_escape_count: number;
}

export interface DoraTelemetrySummary {
  total_defects: number;
  triaged_count: number;
  resolved_count: number;
  mttr_hours: number | null;
  triage_lead_time_hours: number | null;
  production_escape_rate_pct: number;
```

```
  prod_escape_count: number;
  regression_rate_pct: number;
  regression_count: number;
  total_actual_effort_hours: number;
  avg_actual_effort_hours: number;
  environment_counts: Record<string, number>;
  root_cause_effort_breakdown: RootCauseEffortBreakdownItem[];
}

export interface DefectAnalysis {
  id: number;
  devdesc: string | null;
  symptom_category?: SymptomCategory | string | null;
  root_cause_type?: IeeeFaultType | string | null;
  classification: IeeeFaultType | string;
  detected_environment?: DetectedEnvironment | string;
  is_regression?: boolean | number;
  introducing_ref?: string | null;
  resolution_ref?: string | null;
  actual_effort_hours?: number | null;
  triaged_at?: string | null;
  resolved_at?: string | null;
  triage_lead_time_hours?: number | null;
  mttr_hours?: number | null;
  dateoriginated: string | null;
  phase: string | null;
  phases?: string[];
  actoriginated: string | null;
  deliverable: string | null;
  whattodo: string | null;
  submitted: string | null;
  assigned: string | null;
  uniqueness: Uniqueness | string;
  priority: 'most urgent' | 'average' | 'less urgent';
  severity: DefectSeverity;
  workaround: string | null;
  proposedfix: string | null;
  status: DefectStatus;
  fixdesc: string | null;
  fixstarted: string | null;
  fixcomplete: string | null;
  fiximpact: string | null;
  fixeffort: number;
  fixcost?: number | null;
  resodesc: string | null;
  resocomplete: string | null;
  resoclosed: string | null;
  costofdefect?: number | null;
  actfound: string | null;
}

export interface DefectReport {
  ID: number;
  Product: string;
  product_id: number | null;
  Environment: string | null;
  detected_environment?: DetectedEnvironment | string;
  symptom_category?: SymptomCategory | string | null;
  is_regression?: boolean | number;
  CustomerID: number | null;
  Who: string;
  Contactemail: string;
  Whenreported: string;
  reported_at?: string;
  Title: string;
  Description: string;
```

```
  Discoveredby: string | null;
  Persontype: PersonType;
  Occured: string;
  Location: string | null;
  Severity: DefectSeverity;
  Priority: DefectPriority;
  PriorityLabel?: string;
  Submitted: string;
  days_since_submitted?: number;
  Addinfo: string | null;
  is_deleted_user?: number;
  root_cause_type?: string | null;
  classification?: string | null;
  analysis_status?: DefectStatus;
  introducing_ref?: string | null;
  resolution_ref?: string | null;
  actual_effort_hours?: number | null;
  triaged_at?: string | null;
  resolved_at?: string | null;
  fixeffort?: number | null;
  fixcost?: number | null;
  costofdefect?: number | null;
  analysis?: DefectAnalysis | null;
  telemetry?: DoraTelemetrySummary | null;
}

export interface AnalyzerQueueItem extends DefectAnalysis {
  product: string;
  title: string;
  report_description: string;
  environment: string | null;
  occured: string;
  contact_email: string;
  who: string;
  classification_color: string;
  days_open: number;
  submitted_to_assigned_days: number | null;
  submitted_to_fixed_days: number | null;
  submitted_to_resolved_days: number | null;
  assigned_to_fixed_days: number | null;
  assigned_to_resolved_days: number | null;
}
```

==========

SECTION 4.2 IEEE 1044-2009 & DORA Defect Telemetry Workbench
(frontend/src/components/DefectAnalyzerWorkbench.tsx)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\components\DefectAnalyzer Workbench.tsx
==========

```
import React, { useCallback, useEffect, useState } from 'react';
import { defmanApi } from '../api/client';
import {
  DefectSeverity,
  DefectStatus,
  DetectedEnvironment,
  type DoraTelemetrySummary,
  IEEE_1044_2009_TAXONOMY,
  IeeeFaultType,
  OriginPhase,
  SymptomCategory,
  Uniqueness,
} from '../types/defman';
import { DefectResolutionForm } from './DefectResolutionForm';
```

```
export interface DefectAnalyzerWorkbenchProps {
  defectId: number;
  onClose?: () => void;
  onSaved?: () => void;
}

function formatHoursMetric(hours: number | null | undefined): string {
  if (hours === null || hours === undefined || Number.isNaN(Number(hours))) {
    return '--';
  }
  const numeric = Number(hours);
  if (numeric < 24) {
    return `${numeric.toFixed(1)}h`;
  }
  const days = (numeric / 24).toFixed(1);
  return `${numeric.toFixed(1)}h (${days}d)`;
}

/**
 * <DefectAnalyzerWorkbench />
 * IEEE 1044-2009 & DORA Defect Telemetry Workbench
 * Replaces obsolete 1992 IEEE Fault & Cost Analytics and phase-cost multipliers with:
 * - MTTR (resolved_at - reported_at)
 * - Triage Lead Time (triaged_at - reported_at)
 * - Production Escape Rate ('PROD' vs 'DEV' / 'CI' / 'STAGING')
 * - Regression Ratio (is_regression)
 * - Root Cause Effort Breakdown (IEEE Std 1044-2009)
 * Endpoint: PUT /api/defects/{id}/analyze
 */
export function DefectAnalyzerWorkbench({
  defectId,
  onClose,
  onSaved,
}: DefectAnalyzerWorkbenchProps): React.ReactElement {
  const [activeTab, setActiveTab] = useState<'analyze' | 'resolve'>('analyze');
  const [showTaxonomyModal, setShowTaxonomyModal] = useState<boolean>(false);
  const [statusMessage, setStatusMessage] = useState<string | null>(null);
  const [errorMessage, setErrorMessage] = useState<string | null>(null);
  const [telemetry, setTelemetry] = useState<DoraTelemetrySummary | null>(null);
  const [defectTimings, setDefectTimings] = useState<{
    triageLeadTimeHours: number | null;
    mttrHours: number | null;
  }>({
    triageLeadTimeHours: null,
    mttrHours: null,
  });

  const [form, setForm] = useState({
    devdesc: '',
    symptom_category: SymptomCategory.CRASH_HANG as string,
    root_cause_type: IeeeFaultType.CODE_LOGIC as string,
    classification: IeeeFaultType.CODE_LOGIC as string,
    classification_other: '',
    detected_environment: DetectedEnvironment.PROD as string,
    is_regression: false,
    introducing_ref: '',
    triaged_at: new Date().toISOString().slice(0, 16),
    dateoriginated: new Date().toISOString().slice(0, 10),
    phases: [OriginPhase.IMPLEMENTATION] as string[],
    actoriginated: '',
    deliverable: '',
    actfound: '',
    submitted: new Date().toISOString().slice(0, 10),
    assigned: new Date().toISOString().slice(0, 10),
    uniqueness: Uniqueness.NEW_DEFECT as string,
    severity: DefectSeverity.MAJOR,
```

```
    priority: 'most urgent',
    workaround: '',
    proposedfix: '',
    status: DefectStatus.OPENED,
  });

  const loadDefectTelemetryAndAnalysis = useCallback(async function fetchWorkbenchState():
    Promise<void> {
    const res = await defmanApi.getDefectById(defectId);
    if (res.telemetry) {
      setTelemetry(res.telemetry);
    }
    if (res.analysis) {
      const a = res.analysis;
      const candidateRootCause = (a.root_cause_type || a.classification ||
  IeeeFaultType.CODE_LOGIC) as string;
      const isStandardEnum = Object.values(IeeeFaultType).includes(
        candidateRootCause as IeeeFaultType
      );
      setDefectTimings({
        triageLeadTimeHours: a.triage_lead_time_hours ?? null,
        mttrHours: a.mttr_hours ?? null,
      });
      setForm({
        devdesc: a.devdesc || '',
        symptom_category: (a.symptom_category || res.symptom_category ||
  SymptomCategory.CRASH_HANG) as string,
        root_cause_type: isStandardEnum ? candidateRootCause : IeeeFaultType.OTHER,
        classification: isStandardEnum ? candidateRootCause : IeeeFaultType.OTHER,
        classification_other: isStandardEnum ? '' : candidateRootCause,
        detected_environment: (a.detected_environment || res.detected_environment ||
  DetectedEnvironment.PROD) as string,
        is_regression: Boolean(a.is_regression ?? res.is_regression),
        introducing_ref: a.introducing_ref || '',
        triaged_at: a.triaged_at
          ? a.triaged_at.replace(' ', 'T').slice(0, 16)
          : new Date().toISOString().slice(0, 16),
        dateoriginated: a.dateoriginated || res.Occured,
        phases: a.phase ? a.phase.split(',').filter(Boolean) : [],
        actoriginated: a.actoriginated || '',
        deliverable: a.deliverable || '',
        actfound: a.actfound || '',
        submitted: a.submitted || res.Submitted,
        assigned: a.assigned || new Date().toISOString().slice(0, 10),
        uniqueness: (a.uniqueness || Uniqueness.NEW_DEFECT) as string,
        severity: a.severity || res.Severity,
        priority: a.priority || 'most urgent',
        workaround: a.workaround || '',
        proposedfix: a.proposedfix || '',
        status: a.status || DefectStatus.OPENED,
      });
    } else {
      setForm((prev) => ({
        ...prev,
        symptom_category: (res.symptom_category || SymptomCategory.CRASH_HANG) as string,
        detected_environment: (res.detected_environment || DetectedEnvironment.PROD) as
  string,
        is_regression: Boolean(res.is_regression),
        dateoriginated: res.Occured,
        submitted: res.Submitted,
        severity: res.Severity,
      }));
    }
  }, [defectId]);

  useEffect(function initWorkbenchEffect() {
```

```
    loadDefectTelemetryAndAnalysis();
  }, [loadDefectTelemetryAndAnalysis]);

  function togglePhase(phase: string): void {
    setForm(function updatePhases(prev) {
      return {
        ...prev,
        phases: prev.phases.includes(phase)
          ? prev.phases.filter(function filterPhase(p) {
              return p !== phase;
            })
          : [...prev.phases, phase],
      };
    });
  }

  function handleToggleTaxonomyModal(): void {
    setShowTaxonomyModal(function flipModal(prev) {
      return !prev;
    });
  }

  async function handleAnalyzeSubmit(e: React.FormEvent): Promise<void> {
    e.preventDefault();
    setErrorMessage(null);
    setStatusMessage(null);
    try {
      const payload = {
        ...form,
        classification:
          form.classification_other.trim().length > 0
            ? form.classification_other.trim()
            : form.root_cause_type,
      };
      const res = await defmanApi.analyzeDefect(defectId, payload);
      setStatusMessage(res.message);
      await loadDefectTelemetryAndAnalysis();
      if (onSaved) {
        onSaved();
      }
    } catch (err) {
      setErrorMessage(err instanceof Error ? err.message : 'Failed to save IEEE 1044-2009
    analysis');
    }
  }

  async function handleResolutionSaved(): Promise<void> {
    await loadDefectTelemetryAndAnalysis();
    if (onSaved) {
      onSaved();
    }
  }

  return (
    <div className="bg-slate-900 border border-slate-700 rounded-xl p-6 text-slate-100
    shadow-xl space-y-5">
      {/* Header & Navigation Bar */}
      <div className="flex flex-wrap items-center justify-between gap-3 border-b
    border-slate-700 pb-3">
        <div>
          <div className="flex items-center gap-2">
            <h2 className="text-lg font-bold text-amber-400">
              IEEE 1044-2009 &amp;amp; DORA Defect Telemetry -- Defect ID #{defectId}
            </h2>
            <span className="text-[11px] px-2 py-0.5 rounded bg-emerald-950 text-emerald-300
    border border-emerald-700 font-mono">
```

```
            {form.detected_environment}
          </span>
          {form.is_regression && (
            <span className="text-[11px] px-2 py-0.5 rounded bg-amber-950 text-amber-300
    border border-amber-600 font-semibold">
              REGRESSION
            </span>
          )}
        </div>
        <div className="flex items-center gap-2 mt-2 text-xs">
          <button
            type="button"
            onClick={function selectAnalyzeTab() {
              setActiveTab('analyze');
            }}
            className={`px-3 py-1 rounded font-semibold ${
              activeTab === 'analyze'
                ? 'bg-amber-500 text-slate-950'
                : 'bg-slate-800 text-slate-300'
            }`}
          >
            1. IEEE 1044-2009 Anomaly Triage &amp;amp; Classification
          </button>
          <button
            type="button"
            onClick={function selectResolveTab() {
              setActiveTab('resolve');
            }}
            className={`px-3 py-1 rounded font-semibold ${
              activeTab === 'resolve'
                ? 'bg-amber-500 text-slate-950'
                : 'bg-slate-800 text-slate-300'
            }`}
          >
            2. Resolution Ref, Root Cause &amp;amp; Actual Effort Hours
          </button>
        </div>
      </div>
      <div className="flex items-center gap-2">
        <button
          type="button"
          onClick={handleToggleTaxonomyModal}
          className="text-xs px-3 py-1.5 rounded bg-slate-800 hover:bg-slate-700
    text-amber-300 border border-slate-600"
        >
          {showTaxonomyModal
            ? 'Hide IEEE 1044-2009 Taxonomy'
            : 'IEEE 1044-2009 Anomaly Reference'}
        </button>
        {onClose && (
          <button
            type="button"
            onClick={onClose}
            className="text-xs px-3 py-1.5 rounded bg-slate-800 hover:bg-slate-700
    text-slate-300"
          >
            Close
          </button>
        )}
      </div>
    </div>

    {/* IEEE 1044-2009 & DORA Defect Telemetry Summary Cards */}
    {telemetry && (
      <div className="space-y-3 bg-slate-950/80 border border-slate-800 rounded-xl p-4">
        <div className="flex flex-wrap items-center justify-between gap-2">
```

```
          <span className="text-xs font-bold uppercase tracking-wider text-amber-400">
            IEEE 1044-2009 &amp;amp; DORA Defect Telemetry
          </span>
          <span className="text-[11px] text-slate-400 font-mono">
            This Defect Triage Lead:
    {formatHoursMetric(defectTimings.triageLeadTimeHours)} * This Defect MTTR:
    {formatHoursMetric(defectTimings.mttrHours)}
          </span>
        </div>

        <div className="grid grid-cols-2 md:grid-cols-4 gap-3 text-xs">
          <div className="p-3 rounded-lg bg-slate-900 border border-slate-800">
            <div className="text-slate-400">MTTR (Resolved - Reported)</div>
            <div className="text-lg font-bold text-emerald-400 font-mono mt-0.5">
              {formatHoursMetric(telemetry.mttr_hours)}
            </div>
            <div className="text-[11px] text-slate-500">
               Triage Lead Time: {formatHoursMetric(telemetry.triage_lead_time_hours)}
            </div>
          </div>

          <div className="p-3 rounded-lg bg-slate-900 border border-slate-800">
            <div className="text-slate-400">Production Escape Rate</div>
            <div className="text-lg font-bold text-rose-400 font-mono mt-0.5">
              {telemetry.production_escape_rate_pct.toFixed(1)}%
            </div>
            <div className="text-[11px] text-slate-500">
              PROD: {telemetry.environment_counts?.PROD ?? telemetry.prod_escape_count} *
    STAGING: {telemetry.environment_counts?.STAGING ?? 0} * CI:
    {telemetry.environment_counts?.CI ?? 0} * DEV: {telemetry.environment_counts?.DEV ?? 0}
            </div>
          </div>

          <div className="p-3 rounded-lg bg-slate-900 border border-slate-800">
            <div className="text-slate-400">Regression Ratio</div>
            <div className="text-lg font-bold text-amber-400 font-mono mt-0.5">
              {telemetry.regression_rate_pct.toFixed(1)}%
            </div>
            <div className="text-[11px] text-slate-500">
              {telemetry.regression_count} regressions of {telemetry.total_defects}
    anomalies
            </div>
          </div>

          <div className="p-3 rounded-lg bg-slate-900 border border-slate-800">
            <div className="text-slate-400">Actual Engineering Effort</div>
            <div className="text-lg font-bold text-sky-400 font-mono mt-0.5">
              {telemetry.total_actual_effort_hours.toFixed(2)}h
            </div>
            <div className="text-[11px] text-slate-500">
              Avg {telemetry.avg_actual_effort_hours.toFixed(2)}h / anomaly
    ({telemetry.resolved_count} resolved)
            </div>
          </div>
        </div>

        {/* Root Cause Effort Breakdown Table */}
        {telemetry.root_cause_effort_breakdown.length > 0 && (
          <div className="pt-1">
            <div className="text-[11px] font-semibold text-slate-300 mb-1.5">
              Root Cause Effort Breakdown (IEEE Std 1044-2009)
            </div>
            <div className="overflow-x-auto">
              <table className="w-full text-left text-[11px] border-collapse">
                <thead>
                  <tr className="border-b border-slate-800 text-slate-400">
```

```
                    <th className="py-1 pr-3">IEEE 1044-2009 Root Cause</th>
                    <th className="py-1 px-2">Anomalies</th>
                    <th className="py-1 px-2">Total Effort (h)</th>
                    <th className="py-1 px-2">Avg Effort (h)</th>
                    <th className="py-1 px-2">Avg MTTR</th>
                    <th className="py-1 px-2">PROD Escapes</th>
                    <th className="py-1 pl-2">Regressions</th>
                  </tr>
                </thead>
                <tbody className="divide-y divide-slate-900">
                  {telemetry.root_cause_effort_breakdown.map(function
renderBreakdownRow(item) {
                    return (
                      <tr key={item.root_cause_type} className="text-slate-200">
                        <td className="py-1 pr-3 font-medium text-amber-300">
                          {item.root_cause_type}
                        </td>
                        <td className="py-1 px-2 font-mono">{item.defect_count}</td>
                        <td className="py-1 px-2
font-mono">{item.total_effort_hours.toFixed(2)}h</td>
                        <td className="py-1 px-2
font-mono">{item.avg_effort_hours.toFixed(2)}h</td>
                        <td className="py-1 px-2
font-mono">{formatHoursMetric(item.avg_mttr_hours)}</td>
                        <td className="py-1 px-2 font-mono">{item.prod_escape_count}</td>
                        <td className="py-1 pl-2 font-mono">{item.regression_count}</td>
                      </tr>
                    );
                  })}
                </tbody>
              </table>
            </div>
          </div>
        )}
      </div>
    )}

    {showTaxonomyModal && (
      <div className="p-4 rounded-lg bg-slate-950 border border-amber-500/40 text-xs grid
grid-cols-1 md:grid-cols-2 gap-3 max-h-60 overflow-y-auto">
        {Object.entries(IEEE_1044_2009_TAXONOMY).map(function
renderTaxonomyGroup([category, subs]) {
          return (
            <div key={category} className="space-y-1">
              <div className="font-bold text-amber-400">{category}</div>
              <ul className="list-disc list-inside text-slate-400 space-y-0.5">
                {subs.map(function renderSubcategory(s) {
                  return <li key={s}>{s}</li>;
                })}
              </ul>
            </div>
          );
        })}
      </div>
    )}

    {activeTab === 'analyze' ? (
      <form onSubmit={handleAnalyzeSubmit} className="space-y-4 text-xs">
        {errorMessage && (
          <div className="p-3 rounded bg-red-950 border border-red-600 text-amber-300">
            {errorMessage}
          </div>
        )}
        {statusMessage && (
          <div className="p-3 rounded bg-emerald-950 border border-emerald-600
text-emerald-300">
```

```
            {statusMessage}
          </div>
        )}

        <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
          <label className="block md:col-span-2">
            <span className="text-slate-300">
              Engineering anomaly analysis <span className="text-red-400">*</span>
            </span>
            <input
              type="text"
              required
              value={form.devdesc}
              onChange={function onDevDescChange(e) {
                setForm({ ...form, devdesc: e.target.value });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
py-1.5"
            />
          </label>

          <label className="block">
            <span className="text-slate-300">
              Workflow status <span className="text-red-400">*</span>
            </span>
            <select
              value={form.status}
              onChange={function onStatusChange(e) {
                setForm({ ...form, status: e.target.value as DefectStatus });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
py-1.5"
            >
              {Object.values(DefectStatus).map(function renderStatusOption(st) {
                return (
                  <option key={st} value={st}>
                    {st}
                  </option>
                );
              })}
            </select>
          </label>
        </div>

        {/* IEEE Std 1044-2009 Symptom Category, Root Cause & Detection Environment */}
        <div className="grid grid-cols-1 md:grid-cols-3 gap-3 bg-slate-950/60 p-3
rounded-lg border border-slate-800">
          <label className="block">
            <span className="text-slate-300">
              Observed Symptom Category (IEEE 1044-2009) <span
className="text-red-400">*</span>
            </span>
            <select
              value={form.symptom_category}
              onChange={function onSymptomChange(e) {
                setForm({ ...form, symptom_category: e.target.value });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
py-1.5"
            >
              {Object.values(SymptomCategory).map(function renderSymptomOption(sc) {
                return (
                  <option key={sc} value={sc}>
                    {sc}
                  </option>
                );
```

```
              ))}
            </select>
          </label>

          <label className="block">
            <span className="text-slate-300">
              Root Cause Type (IEEE 1044-2009) <span className="text-red-400">*</span>
            </span>
            <select
              disabled={form.classification_other.trim().length > 0}
              value={
                form.classification_other.trim().length > 0
                  ? IeeeFaultType.OTHER
                  : form.root_cause_type
              }
              onChange={function onRootCauseChange(e) {
                setForm({
                  ...form,
                  root_cause_type: e.target.value,
                  classification: e.target.value,
                });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
    py-1.5 disabled:opacity-50"
            >
              {Object.values(IeeeFaultType).map(function renderRootCauseOption(ft) {
                return (
                  <option key={ft} value={ft}>
                    {ft}
                  </option>
                );
              })}
            </select>
          </label>

          <label className="block">
            <span className="text-slate-300">
              Custom Root Cause Override (optional)
            </span>
            <input
              type="text"
              value={form.classification_other}
              onChange={function onCustomClassificationChange(e) {
                setForm({
                  ...form,
                  classification_other: e.target.value,
                  root_cause_type:
                    e.target.value.length > 0
                      ? IeeeFaultType.OTHER
                      : form.root_cause_type,
                });
              }}
              placeholder="Custom IEEE 1044-2009 root cause..."
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
    py-1.5"
            />
          </label>
        </div>

        {/* DORA Environment Escape, Regression Flag, Introducing Commit/Build Ref, Triage
    Timestamp */}
        <div className="grid grid-cols-1 md:grid-cols-4 gap-3 bg-slate-950/60 p-3
    rounded-lg border border-slate-800">
          <label className="block">
            <span className="text-slate-300">
              Detected Environment <span className="text-red-400">*</span>
```

```
            </span>
            <select
              value={form.detected_environment}
              onChange={function onDetectedEnvChange(e) {
                setForm({ ...form, detected_environment: e.target.value });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
    py-1.5 font-mono"
            >
              {Object.values(DetectedEnvironment).map(function renderEnvOption(env) {
                return (
                  <option key={env} value={env}>
                    {env}
                  </option>
                );
              })}
            </select>
          </label>

          <label className="block">
            <span className="text-slate-300">Introducing Commit / Build Ref</span>
            <input
              type="text"
              value={form.introducing_ref}
              onChange={function onIntroducingRefChange(e) {
                setForm({ ...form, introducing_ref: e.target.value });
              }}
              placeholder="e.g. git:a4f9c21 (build-1502)"
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
    py-1.5 font-mono"
            />
          </label>

          <label className="block">
            <span className="text-slate-300">
              Triaged Timestamp (DORA Lead Time)
            </span>
            <input
              type="datetime-local"
              value={form.triaged_at}
              onChange={function onTriagedAtChange(e) {
                setForm({ ...form, triaged_at: e.target.value });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5
    py-1.5 font-mono"
            />
          </label>

          <label className="flex items-center gap-2 pt-5 cursor-pointer">
            <input
              type="checkbox"
              checked={form.is_regression}
              onChange={function onRegressionToggle(e) {
                setForm({ ...form, is_regression: e.target.checked });
              }}
              className="rounded border-slate-600"
            />
            <span className="text-slate-200 font-semibold">
              Regression Defect (is_regression)
            </span>
          </label>
        </div>

        {/* Phase Originated Checkboxes */}
        <div>
          <span className="text-slate-300 block mb-1.5">
```

```
            SDLC Phase Originated <span className="text-red-400">*</span>
          </span>
          <div className="grid grid-cols-2 md:grid-cols-6 gap-2 bg-slate-950/60 p-3
    rounded-lg border border-slate-800">
            {Object.values(OriginPhase).map(function renderPhaseCheckbox(phase) {
              return (
                <label
                  key={phase}
                  className="flex items-center gap-2 cursor-pointer"
                >
                  <input
                    type="checkbox"
                    checked={form.phases.includes(phase)}
                    onChange={function onPhaseCheckboxChange() {
                      togglePhase(phase);
                    }}
                    className="rounded border-slate-600"
                  />
                  <span>{phase}</span>
                </label>
              );
            })}
          </div>
        </div>

        <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
          <label className="block">
            <span className="text-slate-300">
              Deliverable / Module Originated <span className="text-red-400">*</span>
            </span>
            <input
              type="text"
              required
              value={form.deliverable}
              onChange={function onDeliverableChange(e) {
                setForm({ ...form, deliverable: e.target.value });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
    py-1.5"
            />
          </label>
          <label className="block">
            <span className="text-slate-300">
              Originating Engineering Activity <span className="text-red-400">*</span>
            </span>
            <input
              type="text"
              required
              value={form.actoriginated}
              onChange={function onActOriginatedChange(e) {
                setForm({ ...form, actoriginated: e.target.value });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
    py-1.5"
            />
          </label>
          <label className="block">
            <span className="text-slate-300">
              Target Verification / Detection Gate <span className="text-red-400">*</span>
            </span>
            <input
              type="text"
              required
              value={form.actfound}
              onChange={function onActFoundChange(e) {
                setForm({ ...form, actfound: e.target.value });
```

```
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
py-1.5"
          />
        </label>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
        <label className="block">
          <span className="text-slate-300">
            Uniqueness <span className="text-red-400">*</span>
          </span>
          <select
            value={form.uniqueness}
            onChange={function onUniquenessChange(e) {
              setForm({ ...form, uniqueness: e.target.value });
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
py-1.5"
          >
            {Object.values(Uniqueness).map(function renderUniquenessOption(u) {
              return (
                <option key={u} value={u}>
                  {u}
                </option>
              );
            })}
          </select>
        </label>
        <label className="block">
          <span className="text-slate-300">
            Triaged Severity <span className="text-red-400">*</span>
          </span>
          <select
            value={form.severity}
            onChange={function onSeverityChange(e) {
              setForm({
                ...form,
                severity: e.target.value as DefectSeverity,
              });
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
py-1.5"
          >
            <option value={DefectSeverity.MINOR}>minor</option>
            <option value={DefectSeverity.MAJOR}>major</option>
            <option value={DefectSeverity.CRITICAL}>critical</option>
          </select>
        </label>
        <label className="block">
          <span className="text-slate-300">
            Triaged Priority <span className="text-red-400">*</span>
          </span>
          <select
            value={form.priority}
            onChange={function onPriorityChange(e) {
              setForm({ ...form, priority: e.target.value });
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
py-1.5"
          >
            <option value="most urgent">most urgent</option>
            <option value="average">average</option>
            <option value="less urgent">less urgent</option>
          </select>
        </label>
```

```
          </div>

          <div className="grid grid-cols-1 md:grid-cols-2 gap-3">
            <label className="block">
              <span className="text-slate-300">Mitigation / Workaround</span>
              <textarea
                rows={2}
                value={form.workaround}
                onChange={function onWorkaroundChange(e) {
                  setForm({ ...form, workaround: e.target.value });
                }}
                className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
    py-1.5"
              />
            </label>
            <label className="block">
              <span className="text-slate-300">
                Proposed Engineering Remediation <span className="text-red-400">*</span>
              </span>
              <textarea
                rows={2}
                required
                value={form.proposedfix}
                onChange={function onProposedFixChange(e) {
                  setForm({ ...form, proposedfix: e.target.value });
                }}
                className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
    py-1.5"
              />
            </label>
          </div>

          <div className="flex items-center justify-end gap-3 pt-2">
            <button
              type="submit"
              className="px-4 py-2 rounded bg-amber-500 hover:bg-amber-400 text-slate-950
    font-semibold"
            >
              Save IEEE 1044-2009 Triage &amp;amp; Telemetry
            </button>
          </div>
        </form>
      ) : (
        <DefectResolutionForm defectId={defectId} onSaved={handleResolutionSaved} />
      )}
    </div>
  );
}


================================================================================
SECTION 4.3 Defect Resolution & Actual Effort Form
    (frontend/src/components/DefectResolutionForm.tsx)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\components\DefectResoluti
    onForm.tsx
================================================================================

import React, { useEffect, useState } from 'react';
import { defmanApi } from '../api/client';
import { DefectStatus, IeeeFaultType } from '../types/defman';

export interface DefectResolutionFormProps {
  defectId: number;
  onSaved?: () => void;
}
```

```
/**
 * <DefectResolutionForm />
 * Captures IEEE Std 1044-2009 ‘root_cause_type‘, ‘resolution_ref‘ (commit/build/PR ref),
 * ‘actual_effort_hours‘, and ‘resolved_at‘ for DORA MTTR and Root Cause Effort telemetry.
 * Endpoint: PUT /api/defects/{id}/resolve
 */
export function DefectResolutionForm({
  defectId,
  onSaved,
}: DefectResolutionFormProps): React.ReactElement {
  const [statusMessage, setStatusMessage] = useState<string | null>(null);
  const [errorMessage, setErrorMessage] = useState<string | null>(null);

  const [form, setForm] = useState({
    status: DefectStatus.RESOLVED,
    root_cause_type: IeeeFaultType.CODE_LOGIC as string,
    introducing_ref: ’’,
    resolution_ref: ’’,
    actual_effort_hours: 2.5,
    is_regression: false,
    resolved_at: new Date().toISOString().slice(0, 16),
    fixdesc: ’’,
    fixstarted: new Date().toISOString().slice(0, 10),
    fixcomplete: new Date().toISOString().slice(0, 10),
    fiximpact: ’’,
    resodesc: ’’,
    resocomplete: new Date().toISOString().slice(0, 10),
    resoclosed: new Date().toISOString().slice(0, 10),
  });

  useEffect(function loadResolutionDetailsEffect() {
    defmanApi.getDefectById(defectId).then(function handleDefectResponse(res) {
      if (res.analysis) {
        const a = res.analysis;
        const effortHours = Number(a.actual_effort_hours ?? a.fixeffort ?? 2.5);
        setForm({
          status: a.status || DefectStatus.RESOLVED,
          root_cause_type: (a.root_cause_type || a.classification ||
    IeeeFaultType.CODE_LOGIC) as string,
          introducing_ref: a.introducing_ref || ’’,
          resolution_ref: a.resolution_ref || ’’,
          actual_effort_hours: effortHours,
          is_regression: Boolean(a.is_regression ?? res.is_regression),
          resolved_at: a.resolved_at
            ? a.resolved_at.replace(’ ’, ’T’).slice(0, 16)
            : new Date().toISOString().slice(0, 16),
          fixdesc: a.fixdesc || ’’,
          fixstarted: a.fixstarted || new Date().toISOString().slice(0, 10),
          fixcomplete: a.fixcomplete || new Date().toISOString().slice(0, 10),
          fiximpact: a.fiximpact || ’’,
          resodesc: a.resodesc || ’’,
          resocomplete: a.resocomplete || new Date().toISOString().slice(0, 10),
          resoclosed: a.resoclosed || new Date().toISOString().slice(0, 10),
        });
      }
    });
  }, [defectId]);

  function handleEffortChange(effortHours: number): void {
    setForm(function updateEffortState(prev) {
      return {
        ...prev,
        actual_effort_hours: effortHours,
      };
    });
  }
```

```
  async function handleSubmit(e: React.FormEvent): Promise<void> {
    e.preventDefault();
    setErrorMessage(null);
    setStatusMessage(null);
    try {
      const res = await defmanApi.resolveDefect(defectId, {
        ...form,
        fixeffort: form.actual_effort_hours,
      });
      setStatusMessage(res.message);
      if (onSaved) {
        onSaved();
      }
    } catch (err) {
      setErrorMessage(err instanceof Error ? err.message : 'Resolution save failed');
    }
  }

  return (
    <form onSubmit={handleSubmit} className="space-y-4 text-xs">
      {errorMessage && (
        <div className="p-3 rounded bg-red-950 border border-red-600 text-amber-300">
          {errorMessage}
        </div>
      )}
      {statusMessage && (
        <div className="p-3 rounded bg-emerald-950 border border-emerald-600
    text-emerald-300">
          {statusMessage}
        </div>
      )}

      {/* Section 1: IEEE 1044-2009 Root Cause, Build/Commit Ref & Actual Effort Hours */}
      <div className="p-4 rounded-lg bg-slate-950/70 border border-slate-800 space-y-3">
        <div className="font-bold text-amber-400">
          Part A -- IEEE 1044-2009 Root Cause, Resolution Ref &amp;amp; Actual Effort Telemetry
        </div>

        <div className="grid grid-cols-1 md:grid-cols-4 gap-3">
          <label className="block">
            <span className="text-slate-300">
              IEEE 1044-2009 Root Cause Type <span className="text-red-400">*</span>
            </span>
            <select
              value={form.root_cause_type}
              onChange={function onRootCauseChange(e) {
                setForm({ ...form, root_cause_type: e.target.value });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5
    py-1.5"
            >
              {Object.values(IeeeFaultType).map(function renderRootCauseOption(rc) {
                return (
                  <option key={rc} value={rc}>
                    {rc}
                  </option>
                );
              })}
            </select>
          </label>

          <label className="block">
            <span className="text-slate-300">
              Resolution Commit / PR / Release Ref <span className="text-red-400">*</span>
            </span>
```

```
          <input
            type="text"
            required
            value={form.resolution_ref}
            onChange={function onResolutionRefChange(e) {
              setForm({ ...form, resolution_ref: e.target.value });
            }}
            placeholder="e.g. git:e83a11f (pr-412 / v1.5.3)"
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5
    py-1.5 font-mono"
          />
        </label>

        <label className="block">
          <span className="text-slate-300">
            Introducing Commit / Build Ref
          </span>
          <input
            type="text"
            value={form.introducing_ref}
            onChange={function onIntroducingRefChange(e) {
              setForm({ ...form, introducing_ref: e.target.value });
            }}
            placeholder="e.g. git:a4f9c21 (build-1502)"
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5
    py-1.5 font-mono"
          />
        </label>

        <label className="block">
          <span className="text-slate-300">
            Actual Effort Hours <span className="text-red-400">*</span>
          </span>
          <input
            type="number"
            step="0.25"
            min="0"
            required
            value={form.actual_effort_hours}
            onChange={function onActualEffortInput(e) {
              handleEffortChange(Number(e.target.value));
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5
    py-1.5 font-mono"
          />
        </label>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2 gap-3">
        <label className="block">
          <span className="text-slate-300">Description of engineering fix *</span>
          <textarea
            rows={2}
            required
            value={form.fixdesc}
            onChange={function onFixDescChange(e) {
              setForm({ ...form, fixdesc: e.target.value });
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
    py-1.5"
          />
        </label>
        <label className="block">
          <span className="text-slate-300">Modules / Deliverables impacted by fix</span>
          <textarea
            rows={2}
```

```
            value={form.fiximpact}
            onChange={function onFixImpactChange(e) {
              setForm({ ...form, fiximpact: e.target.value });
            }}
            placeholder="e.g. settings.cpp, popup.cpp, AnalysisRepository.php"
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3
    py-1.5"
          />
        </label>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
        <label className="block">
          <span className="text-slate-300">Date work started *</span>
          <input
            type="date"
            required
            value={form.fixstarted}
            onChange={function onFixStartedChange(e) {
              setForm({ ...form, fixstarted: e.target.value });
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5
    py-1.5"
          />
        </label>
        <label className="block">
          <span className="text-slate-300">Date work completed *</span>
          <input
            type="date"
            required
            value={form.fixcomplete}
            onChange={function onFixCompleteChange(e) {
              setForm({ ...form, fixcomplete: e.target.value });
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5
    py-1.5"
          />
        </label>
        <label className="flex items-center gap-2 pt-5 cursor-pointer">
          <input
            type="checkbox"
            checked={form.is_regression}
            onChange={function onRegressionChange(e) {
              setForm({ ...form, is_regression: e.target.checked });
            }}
            className="rounded border-slate-600"
          />
          <span className="text-slate-200 font-semibold">
            Regression Anomaly (is_regression)
          </span>
        </label>
      </div>
    </div>

    {/* Section 2: DORA Resolution Timestamp & Verification Status */}
    <div className="p-4 rounded-lg bg-slate-950/70 border border-slate-800 space-y-3">
      <div className="font-bold text-amber-400">
        Part B -- DORA Resolution Verification &amp;amp; MTTR Timestamp
      </div>
      <label className="block">
        <span className="text-slate-300">Resolution &amp;amp; verification summary *</span>
        <textarea
          rows={2}
          required
          value={form.resodesc}
          onChange={function onResoDescChange(e) {
```

```
            setForm({ ...form, resodesc: e.target.value });
          }}
          className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
        />
      </label>

      <div className="grid grid-cols-2 md:grid-cols-4 gap-3">
        <label className="block">
          <span className="text-slate-300">Resolution Status *</span>
          <select
            value={form.status}
            onChange={function onStatusSelectChange(e) {
              setForm({ ...form, status: e.target.value as DefectStatus });
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5
  py-1.5"
          >
            {Object.values(DefectStatus).map(function renderDefectStatusOption(st) {
              return (
                <option key={st} value={st}>
                  {st}
                </option>
              );
            })}
          </select>
        </label>
        <label className="block">
          <span className="text-slate-300">Resolved At (DORA MTTR) *</span>
          <input
            type="datetime-local"
            required
            value={form.resolved_at}
            onChange={function onResolvedAtChange(e) {
              setForm({
                ...form,
                resolved_at: e.target.value,
                resocomplete: e.target.value.slice(0, 10),
              });
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5
  py-1.5 font-mono"
          />
        </label>
        <label className="block">
          <span className="text-slate-300">Date of resolution *</span>
          <input
            type="date"
            required
            value={form.resocomplete}
            onChange={function onResoCompleteChange(e) {
              setForm({ ...form, resocomplete: e.target.value });
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5
  py-1.5"
          />
        </label>
        <label className="block">
          <span className="text-slate-300">Date closed *</span>
          <input
            type="date"
            required
            value={form.resoclosed}
            onChange={function onResoClosedChange(e) {
              setForm({ ...form, resoclosed: e.target.value });
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5
```

```
    py-1.5"
            />
          </label>
        </div>
      </div>

      <div className="flex justify-end">
        <button
          type="submit"
          className="px-4 py-2 rounded bg-amber-500 hover:bg-amber-400 text-slate-950
    font-semibold"
        >
          Save IEEE 1044-2009 Resolution &amp;amp; Effort Telemetry
        </button>
      </div>
    </form>
  );
}


================================================================================
SECTION 4.4 Defect Detail & Environment Modal
    (frontend/src/components/DefectDetailModal.tsx)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\components\DefectDetailMo
    dal.tsx
================================================================================

import React, { useEffect, useState } from 'react';
import { defmanApi } from '../api/client';
import {
  DefectPriority,
  type DefectReport,
  DefectSeverity,
  DetectedEnvironment,
  PersonType,
  type Product,
  SymptomCategory,
} from '../types/defman';

export interface DefectDetailModalProps {
  defectId: number;
  canEdit: boolean;
  onClose: () => void;
  onOpenAnalyze?: (id: number) => void;
  onUpdated?: () => void;
}

/**
 * <DefectDetailModal />
 * Displays and updates a normalized defect report along with its IEEE Std 1044-2009
 * symptom category, detection environment ('DEV' | 'CI' | 'STAGING' | 'PROD'),
 * regression flag, and DORA telemetry summary.
 * Endpoint: GET /api/defects/{id} & PUT /api/defects/{id}
 */
export function DefectDetailModal({
  defectId,
  canEdit,
  onClose,
  onOpenAnalyze,
  onUpdated,
}: DefectDetailModalProps): React.ReactElement | null {
  const [defect, setDefect] = useState<DefectReport | null>(null);
  const [products, setProducts] = useState<Product[]>([]);
  const [message, setMessage] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);
```

```
  useEffect(function loadDefectAndProductsEffect() {
    Promise.all([
      defmanApi.getDefectById(defectId),
      defmanApi.getProducts(),
    ]).then(function handleLoadedModalData([defRes, prodRes]) {
      setDefect(defRes);
      setProducts(prodRes.products);
    });
  }, [defectId]);

  if (!defect) {
    return null;
  }

  const currentDefect = defect;

  async function handleSave(e: React.FormEvent): Promise<void> {
    e.preventDefault();
    setMessage(null);
    setError(null);
    try {
      const res = await defmanApi.updateDefectReport(currentDefect.ID, {
        product: currentDefect.Product,
        environment: currentDefect.Environment,
        detected_environment: currentDefect.detected_environment ||
    DetectedEnvironment.PROD,
        symptom_category: currentDefect.symptom_category || null,
        is_regression: Boolean(currentDefect.is_regression),
        title: currentDefect.Title,
        description: currentDefect.Description,
        discovered_by: currentDefect.Discoveredby,
        persontype: currentDefect.Persontype,
        occured: currentDefect.Occured,
        location: currentDefect.Location,
        severity: currentDefect.Severity,
        priority: currentDefect.Priority,
        submitted: currentDefect.Submitted,
        addinfo: currentDefect.Addinfo,
      });
      setMessage(res.message);
      if (onUpdated) {
        onUpdated();
      }
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Update failed');
    }
  }

  function handleOpenAnalyzeClick(): void {
    if (onOpenAnalyze) {
      onOpenAnalyze(currentDefect.ID);
    }
  }

  return (
    <div className="fixed inset-0 z-50 bg-black/75 flex items-center justify-center p-4">
      <div className="bg-slate-900 border border-slate-700 rounded-xl max-w-2xl w-full p-6
    text-slate-100 shadow-2xl max-h-[90vh] overflow-y-auto">
        <div className="flex items-center justify-between border-b border-slate-700 pb-3
    mb-4">
          <div>
            <div className="flex items-center gap-2">
              <h3 className="text-lg font-bold text-amber-400">
                Defect Report -- Defect ID #{currentDefect.ID}
              </h3>
              <span className="text-[11px] px-2 py-0.5 rounded bg-slate-800 border
```

```
    border-slate-700 font-mono text-emerald-300">
                {currentDefect.detected_environment || DetectedEnvironment.PROD}
              </span>
              {Boolean(currentDefect.is_regression) && (
                <span className="text-[11px] px-2 py-0.5 rounded bg-amber-950 border
    border-amber-600 text-amber-300 font-semibold">
                  REGRESSION
                </span>
              )}
            </div>
            <p className="text-xs text-slate-400 mt-0.5">
              Reported by {currentDefect.Who} ({currentDefect.Contactemail}) at
    {currentDefect.Whenreported}
            </p>
          </div>
          <div className="flex items-center gap-2">
            {onOpenAnalyze && (
              <button
                type="button"
                onClick={handleOpenAnalyzeClick}
                className="text-xs px-3 py-1.5 rounded bg-amber-500 hover:bg-amber-400
    text-slate-950 font-semibold"
              >
                Open Telemetry Workbench
              </button>
            )}
            <button
              type="button"
              onClick={onClose}
              className="text-xs px-3 py-1.5 rounded bg-slate-800 hover:bg-slate-700
    text-slate-300"
            >
              Close
            </button>
          </div>
        </div>

        {error && (
          <div className="mb-3 p-2.5 rounded bg-red-950 border border-red-600 text-amber-300
    text-xs">
            {error}
          </div>
        )}
        {message && (
          <div className="mb-3 p-2.5 rounded bg-emerald-950 border border-emerald-600
    text-emerald-300 text-xs">
            {message}
          </div>
        )}

        {/* IEEE 1044-2009 & DORA Telemetry Snapshot if analyzed */}
        {currentDefect.analysis && (
          <div className="mb-4 p-3 rounded-lg bg-slate-950 border border-slate-800 grid
    grid-cols-2 md:grid-cols-3 gap-2 text-xs">
            <div>
              <span className="text-slate-400">Root Cause (1044-2009):</span>{' '}
              <strong className="text-amber-300">
                {currentDefect.analysis.root_cause_type ||
    currentDefect.analysis.classification}
              </strong>
            </div>
            <div>
              <span className="text-slate-400">Symptom:</span>{' '}
              <span className="text-slate-200">
                {currentDefect.analysis.symptom_category || currentDefect.symptom_category
    || '--'}
```

```
            </span>
          </div>
          <div>
            <span className="text-slate-400">Actual Effort:</span>{' '}
            <span className="font-mono text-sky-300">
              {Number(currentDefect.analysis.actual_effort_hours ??
    currentDefect.analysis.fixeffort ?? 0).toFixed(2)}h
            </span>
          </div>
          <div>
            <span className="text-slate-400">Introducing Ref:</span>{' '}
            <span className="font-mono text-slate-200">
              {currentDefect.analysis.introducing_ref || '--'}
            </span>
          </div>
          <div>
            <span className="text-slate-400">Resolution Ref:</span>{' '}
            <span className="font-mono text-emerald-300">
              {currentDefect.analysis.resolution_ref || '--'}
            </span>
          </div>
          <div>
            <span className="text-slate-400">MTTR / Triage Lead:</span>{' '}
            <span className="font-mono text-slate-200">
              {currentDefect.analysis.mttr_hours !== null &&
    currentDefect.analysis.mttr_hours !== undefined
                ? `${currentDefect.analysis.mttr_hours}h`
                : 'open'}{' '}
              /{' '}
              {currentDefect.analysis.triage_lead_time_hours !== null &&
    currentDefect.analysis.triage_lead_time_hours !== undefined
                ? `${currentDefect.analysis.triage_lead_time_hours}h`
                : '--'}
            </span>
          </div>
        </div>
      )}

      <form onSubmit={handleSave} className="space-y-3 text-xs">
        <div className="grid grid-cols-2 gap-3">
          <label className="block">
            <span className="text-slate-400">Product and release *</span>
            <select
              disabled={!canEdit}
              value={currentDefect.Product}
              onChange={function onProductChange(e) {
                setDefect({ ...currentDefect, Product: e.target.value });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5
    py-1.5"
            >
              {products.map(function renderProductOption(p) {
                return (
                  <option key={p.id} value={p.name}>
                    {p.name}
                  </option>
                );
              })}
            </select>
          </label>
          <label className="block">
            <span className="text-slate-400">Title *</span>
            <input
              type="text"
              disabled={!canEdit}
              value={currentDefect.Title}
```

```
              onChange={function onTitleChange(e) {
                setDefect({ ...currentDefect, Title: e.target.value });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5
    py-1.5"
            />
          </label>
        </div>

        <label className="block">
          <span className="text-slate-400">Description of the problem *</span>
          <textarea
            rows={3}
            disabled={!canEdit}
            value={currentDefect.Description}
            onChange={function onDescriptionChange(e) {
              setDefect({ ...currentDefect, Description: e.target.value });
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5
    py-1.5"
          />
        </label>

        {/* IEEE 1044-2009 Symptom, Detection Environment & Regression */}
        <div className="grid grid-cols-3 gap-3 bg-slate-950/60 p-3 rounded-lg border
    border-slate-800">
          <label className="block">
            <span className="text-slate-400">Detected Environment *</span>
            <select
              disabled={!canEdit}
              value={currentDefect.detected_environment || DetectedEnvironment.PROD}
              onChange={function onDetectedEnvironmentChange(e) {
                setDefect({ ...currentDefect, detected_environment: e.target.value });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5
    py-1.5 font-mono"
            >
              {Object.values(DetectedEnvironment).map(function
    renderDetectedEnvOption(env) {
                return (
                  <option key={env} value={env}>
                    {env}
                  </option>
                );
              })}
            </select>
          </label>

          <label className="block">
            <span className="text-slate-400">Symptom Category (IEEE 1044-2009)</span>
            <select
              disabled={!canEdit}
              value={currentDefect.symptom_category || SymptomCategory.OTHER}
              onChange={function onSymptomCategoryChange(e) {
                setDefect({ ...currentDefect, symptom_category: e.target.value });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5
    py-1.5"
            >
              {Object.values(SymptomCategory).map(function renderSymptomCategoryOption(sc)
    {
                return (
                  <option key={sc} value={sc}>
                    {sc}
                  </option>
                );
```

```
              ))}
            </select>
          </label>

          <label className="flex items-center gap-2 pt-5 cursor-pointer">
            <input
              type="checkbox"
              disabled={!canEdit}
              checked={Boolean(currentDefect.is_regression)}
              onChange={function onRegressionFlagChange(e) {
                setDefect({ ...currentDefect, is_regression: e.target.checked });
              }}
              className="rounded border-slate-600"
            />
            <span className="text-slate-300 font-semibold">Regression Defect</span>
          </label>
        </div>

        <div className="grid grid-cols-3 gap-3">
          <label className="block">
            <span className="text-slate-400">Who discovered</span>
            <input
              type="text"
              disabled={!canEdit}
              value={currentDefect.Discoveredby || ''}
              onChange={function onDiscoveredByChange(e) {
                setDefect({ ...currentDefect, Discoveredby: e.target.value });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5
    py-1.5"
            />
          </label>
          <label className="block">
            <span className="text-slate-400">Person type *</span>
            <select
              disabled={!canEdit}
              value={currentDefect.Persontype}
              onChange={function onPersonTypeChange(e) {
                setDefect({ ...currentDefect, Persontype: e.target.value as PersonType });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5
    py-1.5"
            >
              {Object.values(PersonType).map(function renderPersonTypeOption(pt) {
                return (
                  <option key={pt} value={pt}>
                    {pt}
                  </option>
                );
              })}
            </select>
          </label>
          <label className="block">
            <span className="text-slate-400">Location found</span>
            <input
              type="text"
              disabled={!canEdit}
              value={currentDefect.Location || ''}
              onChange={function onLocationChange(e) {
                setDefect({ ...currentDefect, Location: e.target.value });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5
    py-1.5"
            />
          </label>
        </div>
```

```
      <div className="grid grid-cols-4 gap-3">
        <label className="block">
          <span className="text-slate-400">Date Occurred *</span>
          <input
            type="date"
            disabled={!canEdit}
            value={currentDefect.Occured}
            onChange={function onOccuredChange(e) {
              setDefect({ ...currentDefect, Occured: e.target.value });
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5
py-1.5"
          />
        </label>
        <label className="block">
          <span className="text-slate-400">Submitted *</span>
          <input
            type="date"
            disabled={!canEdit}
            value={currentDefect.Submitted}
            onChange={function onSubmittedChange(e) {
              setDefect({ ...currentDefect, Submitted: e.target.value });
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5
py-1.5"
          />
        </label>
        <label className="block">
          <span className="text-slate-400">Severity *</span>
          <select
            disabled={!canEdit}
            value={currentDefect.Severity}
            onChange={function onSeveritySelectChange(e) {
              setDefect({ ...currentDefect, Severity: e.target.value as DefectSeverity
});
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5
py-1.5"
          >
            <option value={DefectSeverity.MINOR}>minor</option>
            <option value={DefectSeverity.MAJOR}>major</option>
            <option value={DefectSeverity.CRITICAL}>critical</option>
          </select>
        </label>
        <label className="block">
          <span className="text-slate-400">Priority *</span>
          <select
            disabled={!canEdit}
            value={currentDefect.Priority}
            onChange={function onPrioritySelectChange(e) {
              setDefect({ ...currentDefect, Priority: e.target.value as DefectPriority
});
            }}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5
py-1.5"
          >
            <option value={DefectPriority.MOST_URGENT}>most urgent</option>
            <option value={DefectPriority.AVERAGE}>average</option>
            <option value={DefectPriority.LESS_URGENT}>less urgent</option>
          </select>
        </label>
      </div>

      <label className="block">
        <span className="text-slate-400">Technical environment details</span>
```

```
            <textarea
              rows={2}
              disabled={!canEdit}
              value={currentDefect.Environment || ''}
              onChange={function onEnvironmentChange(e) {
                setDefect({ ...currentDefect, Environment: e.target.value });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5
    py-1.5"
            />
          </label>

          <label className="block">
            <span className="text-slate-400">Additional information</span>
            <textarea
              rows={2}
              disabled={!canEdit}
              value={currentDefect.Addinfo || ''}
              onChange={function onAddInfoChange(e) {
                setDefect({ ...currentDefect, Addinfo: e.target.value });
              }}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5
    py-1.5"
            />
          </label>

          {canEdit && (
            <div className="pt-2 flex justify-end">
              <button
                type="submit"
                className="px-4 py-2 rounded bg-amber-500 hover:bg-amber-400 text-slate-950
    font-semibold"
              >
                Update Report
              </button>
            </div>
          )}
        </form>
      </div>
    </div>
  );
}


================================================================================
SECTION 4.5 PixiJS v8 WebGPU Flow & Effort Triage Canvas
    (frontend/src/components/DefectTriageCanvas.tsx)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\components\DefectTriageCa
    nvas.tsx
================================================================================

import React, { useEffect, useRef, useState } from 'react';
import {
  Application,
  Container,
  type FederatedPointerEvent,
  Graphics,
  Text,
} from 'pixi.js';
import { defmanApi } from '../api/client';
import {
  type AnalyzerQueueItem,
  DefectSeverity,
  DefectStatus,
  IEEE_FAULT_HEX_COLORS,
  IeeeFaultType,
```

```
} from '../types/defman';

interface TriageLaneConfig {
  id: 'open' | 'analyzing' | 'fixed' | 'resolved';
  label: string;
  targetStatus: DefectStatus;
  xStart: number;
  xEnd: number;
}

export interface DefectTriageCanvasProps {
  items: AnalyzerQueueItem[];
  onSelectDefectForWorkbench: (defectId: number) => void;
  onStatusChanged: () => void;
}

const CANVAS_WIDTH = 1200;
const CANVAS_HEIGHT = 680;
const LANE_WIDTH = CANVAS_WIDTH / 4;

const TRIAGE_LANES: readonly TriageLaneConfig[] = [
  {
    id: 'open',
    label: 'OPEN (Submitted / Postponed)',
    targetStatus: DefectStatus.SUBMITTED,
    xStart: 0,
    xEnd: LANE_WIDTH,
  },
  {
    id: 'analyzing',
    label: 'ANALYZING (Triaged / Opened)',
    targetStatus: DefectStatus.OPENED,
    xStart: LANE_WIDTH,
    xEnd: LANE_WIDTH * 2,
  },
  {
    id: 'fixed',
    label: 'FIXED (Commit / Build Verified)',
    targetStatus: DefectStatus.FIXED,
    xStart: LANE_WIDTH * 2,
    xEnd: LANE_WIDTH * 3,
  },
  {
    id: 'resolved',
    label: 'RESOLVED (QA Verified / Closed)',
    targetStatus: DefectStatus.RESOLVED,
    xStart: LANE_WIDTH * 3,
    xEnd: CANVAS_WIDTH,
  },
];

function mapStatusToLaneIndex(status: DefectStatus | string): number {
  switch (status) {
    case DefectStatus.SUBMITTED:
    case DefectStatus.POSTPONED:
      return 0;
    case DefectStatus.ASSIGNED:
    case DefectStatus.OPENED:
      return 1;
    case DefectStatus.FIXED:
      return 2;
    case DefectStatus.RESOLVED:
    case DefectStatus.CLOSED:
      return 3;
    default:
      return 0;
```

```
  }
}

function getRootCauseHexColor(rootCauseOrClassification: string): number {
  const normalized = (rootCauseOrClassification || '').trim();
  if (normalized in IEEE_FAULT_HEX_COLORS) {
    return IEEE_FAULT_HEX_COLORS[normalized];
  }
  const match = Object.values(IeeeFaultType).find(function matchFaultCase(ft) {
    return ft.toLowerCase() === normalized.toLowerCase();
  });
  return match
    ? IEEE_FAULT_HEX_COLORS[match]
    : IEEE_FAULT_HEX_COLORS[IeeeFaultType.OTHER];
}

function getSeverityGlowColor(
  severity: DefectSeverity | string,
  priority: string,
  isRegression: boolean
): {
  color: number;
  alpha: number;
  width: number;
} {
  if (severity === DefectSeverity.CRITICAL) {
    return { color: 0xff2e4c, alpha: 0.92, width: 5 };
  }
  if (isRegression || severity === DefectSeverity.MAJOR || priority === 'most urgent') {
    return { color: 0xf59e0b, alpha: 0.8, width: 3.5 };
  }
  return { color: 0x38bdf8, alpha: 0.45, width: 2 };
}

function computeNodeRadiusFromEffort(
  actualEffortHours: number,
  mttrOrLeadHours: number
): number {
  const effortComponent = Math.sqrt(Math.max(1, actualEffortHours)) * 6.5;
  const flowComponent = Math.log10(Math.max(2, mttrOrLeadHours + 2)) * 5.5;
  return Math.min(54, Math.max(22, Math.round(14 + effortComponent + flowComponent)));
}

/**
 * <DefectTriageCanvas />
 * Powered by PixiJS v8 (WebGPU primary with WebGL fallback)
 * Renders anomalies with IEEE Std 1044-2009 & DORA Flow Telemetry:
 *   - Node fill color by IEEE Std 1044-2009 Root Cause Type
 *   - Outer ring glow by Severity, Priority & Regression status
 *   - Node radius scaled proportionally to Actual Effort Hours & Flow Lead Time / MTTR
 *   - Pan/zoom viewport, hover telemetry tooltip, click-to-open <DefectAnalyzerWorkbench
    />,
 *     and drag-and-drop across status lanes triggering PUT /api/defects/{id}/resolve
 */
export function DefectTriageCanvas({
  items,
  onSelectDefectForWorkbench,
  onStatusChanged,
}: DefectTriageCanvasProps): React.ReactElement {
  const hostRef = useRef<HTMLDivElement | null>(null);
  const [hoveredItem, setHoveredItem] = useState<{
    item: AnalyzerQueueItem;
    screenX: number;
    screenY: number;
  } | null>(null);
  const [rendererBackend, setRendererBackend] = useState<string>('Initializing WebGPU...');
```

```
  const [zoomLevel, setZoomLevel] = useState<number>(1);

  useEffect(function setupPixiCanvasEffect() {
    const hostElement = hostRef.current;
    if (!hostElement) {
      return;
    }
    const containerElement: HTMLDivElement = hostElement;

    let destroyed = false;
    const app = new Application();

    async function initPixiCanvas(): Promise<void> {
      await app.init({
        preference: 'webgpu',
        antialias: true,
        width: CANVAS_WIDTH,
        height: CANVAS_HEIGHT,
        backgroundColor: 0x090d16,
        resolution: window.devicePixelRatio || 1,
        autoDensity: true,
      });

      if (destroyed) {
        app.destroy(true);
        return;
      }

      containerElement.innerHTML = '';
      containerElement.appendChild(app.canvas);

      const rendererType = app.renderer.name?.toLowerCase().includes('webgpu')
        ? 'WebGPU'
        : 'WebGL2';
      setRendererBackend(`PixiJS v8 (${rendererType})`);

      const world = new Container();
      app.stage.addChild(world);

      const lanesGraphics = new Graphics();
      world.addChild(lanesGraphics);

      TRIAGE_LANES.forEach(function renderLaneColumn(lane, idx) {
        const bgColor = idx % 2 === 0 ? 0x0f172a : 0x111c33;
        lanesGraphics
          .rect(lane.xStart, 0, LANE_WIDTH, CANVAS_HEIGHT)
          .fill({ color: bgColor, alpha: 0.85 })
          .stroke({ color: 0x1e293b, width: 1.5 });

        const headerText = new Text({
          text: lane.label,
          style: {
            fontFamily: 'Inter, monospace',
            fontSize: 12,
            fontWeight: 'bold',
            fill: 0xfbbf24,
          },
        });
        headerText.position.set(lane.xStart + 16, 16);
        world.addChild(headerText);
      });

      const laneCounts = [0, 0, 0, 0];

      items.forEach(function renderDefectNode(defect) {
        const laneIndex = mapStatusToLaneIndex(defect.status);
```

```
        const slotIndex = laneCounts[laneIndex]++;
        const col = slotIndex % 2;
        const row = Math.floor(slotIndex / 2);

        const initialX = TRIAGE_LANES[laneIndex].xStart + 90 + col * 125;
        const initialY = 95 + row * 115;

        const effortHours = Number(defect.actual_effort_hours ?? defect.fixeffort ?? 0);
        const flowHours = Number(
          defect.mttr_hours ?? defect.triage_lead_time_hours ?? defect.days_open * 24
        );
        const radius = computeNodeRadiusFromEffort(effortHours, flowHours);
        const rootCauseLabel = (defect.root_cause_type || defect.classification ||
    IeeeFaultType.OTHER) as string;
        const fillHex = getRootCauseHexColor(rootCauseLabel);
        const glow = getSeverityGlowColor(
          defect.severity,
          defect.priority,
          Boolean(defect.is_regression)
        );

        const nodeContainer = new Container();
        nodeContainer.position.set(initialX, initialY);
        nodeContainer.eventMode = 'static';
        nodeContainer.cursor = 'grab';

        const circleGfx = new Graphics();
        circleGfx
          .circle(0, 0, radius + glow.width + 3)
          .fill({ color: glow.color, alpha: 0.18 })
          .stroke({ color: glow.color, width: glow.width, alpha: glow.alpha });

        circleGfx
          .circle(0, 0, radius)
          .fill({ color: fillHex, alpha: 0.92 })
          .stroke({ color: 0xffffff, width: 1.5, alpha: 0.8 });

        nodeContainer.addChild(circleGfx);

        const envTag = defect.detected_environment || 'PROD';
        const idLabel = new Text({
          text: '#${defect.id} [${envTag}]\n${effortHours.toFixed(1)}h',
          style: {
            fontFamily: 'monospace',
            fontSize: 10,
            fontWeight: 'bold',
            fill: 0xffffff,
            align: 'center',
          },
        });
        idLabel.anchor.set(0.5);
        nodeContainer.addChild(idLabel);

        let isDragging = false;
        let pointerMovedDistance = 0;
        let dragStartPos = { x: 0, y: 0 };

        function handlePointerOver(e: FederatedPointerEvent): void {
          setHoveredItem({
            item: defect,
            screenX: e.global.x,
            screenY: e.global.y,
          });
        }

        function handlePointerOut(): void {
```

```
        if (!isDragging) {
          setHoveredItem(null);
        }
      }

      function handlePointerDown(e: FederatedPointerEvent): void {
        e.stopPropagation();
        isDragging = true;
        pointerMovedDistance = 0;
        nodeContainer.cursor = 'grabbing';
        nodeContainer.alpha = 0.85;
        dragStartPos = world.toLocal(e.global);
      }

      function handleGlobalPointerMove(e: FederatedPointerEvent): void {
        if (!isDragging) {
          return;
        }
        const currentLocal = world.toLocal(e.global);
        const dx = currentLocal.x - dragStartPos.x;
        const dy = currentLocal.y - dragStartPos.y;
        pointerMovedDistance += Math.hypot(dx, dy);
        nodeContainer.position.x += dx;
        nodeContainer.position.y += dy;
        dragStartPos = currentLocal;
      }

      async function finishInteraction(): Promise<void> {
        if (!isDragging) {
          return;
        }
        isDragging = false;
        nodeContainer.cursor = 'grab';
        nodeContainer.alpha = 1;

        if (pointerMovedDistance < 6) {
          onSelectDefectForWorkbench(defect.id);
          return;
        }

        const dropX = Math.max(0, Math.min(CANVAS_WIDTH - 1, nodeContainer.position.x));
        const targetLaneIndex = Math.min(
          TRIAGE_LANES.length - 1,
          Math.floor(dropX / LANE_WIDTH)
        );
        const targetLane = TRIAGE_LANES[targetLaneIndex];

        nodeContainer.position.x = targetLane.xStart + LANE_WIDTH / 2;

        if (targetLane.targetStatus !== defect.status) {
          await defmanApi.resolveDefect(defect.id, {
            status: targetLane.targetStatus,
          });
          onStatusChanged();
        }
      }

      nodeContainer.on('pointerover', handlePointerOver);
      nodeContainer.on('pointerout', handlePointerOut);
      nodeContainer.on('pointerdown', handlePointerDown);
      nodeContainer.on('globalpointermove', handleGlobalPointerMove);
      nodeContainer.on('pointerup', finishInteraction);
      nodeContainer.on('pointerupoutside', finishInteraction);

      world.addChild(nodeContainer);
    });
```

```
    let stagePanning = false;
    let panLast = { x: 0, y: 0 };

    app.stage.eventMode = 'static';
    app.stage.hitArea = app.screen;

    function handleStagePointerDown(e: FederatedPointerEvent): void {
      stagePanning = true;
      panLast = { x: e.global.x, y: e.global.y };
    }

    function handleStagePointerMove(e: FederatedPointerEvent): void {
      if (!stagePanning) {
        return;
      }
      world.position.x += e.global.x - panLast.x;
      world.position.y += e.global.y - panLast.y;
      panLast = { x: e.global.x, y: e.global.y };
    }

    function handleStagePointerUp(): void {
      stagePanning = false;
    }

    function onWheel(e: WheelEvent): void {
      e.preventDefault();
      const factor = e.deltaY < 0 ? 1.1 : 0.9;
      const nextScale = Math.min(2.2, Math.max(0.55, world.scale.x * factor));
      world.scale.set(nextScale);
      setZoomLevel(Number(nextScale.toFixed(2)));
    }

    app.stage.on('pointerdown', handleStagePointerDown);
    app.stage.on('globalpointermove', handleStagePointerMove);
    app.stage.on('pointerup', handleStagePointerUp);
    app.canvas.addEventListener('wheel', onWheel, { passive: false });
  }

  initPixiCanvas();

  return function cleanupPixiCanvas(): void {
    destroyed = true;
    try {
      app.destroy(true, { children: true });
    } catch {
      // Safe cleanup if unmount occurs mid-init
    }
  };
}, [items, onSelectDefectForWorkbench, onStatusChanged]);

return (
  <div className="bg-slate-900 border border-slate-700 rounded-xl p-4 text-slate-100
  shadow-xl space-y-3 relative">
    <div className="flex flex-wrap items-center justify-between gap-3">
      <div>
        <h2 className="text-lg font-bold text-amber-400">
          Visual Defect Triage &amp;amp; IEEE 1044-2009 / DORA Flow Canvas
        </h2>
        <p className="text-xs text-slate-400">
          Drag nodes across lanes (Open -> Analyzing -> Fixed -> Resolved) to trigger{' '}
          <code className="text-amber-300">PUT /api/defects/&#123;id&#125;/resolve</code>,
  or click any node to open{' '}
          <code className="text-amber-300">&lt;DefectAnalyzerWorkbench /&gt;</code>.
        </p>
      </div>
```

```
        <div className="flex items-center gap-3 text-xs">
          <span className="px-2.5 py-1 rounded bg-slate-800 border border-slate-700
    font-mono text-emerald-400">
            {rendererBackend}
          </span>
          <span className="px-2.5 py-1 rounded bg-slate-800 border border-slate-700
    font-mono">
            Zoom: {Math.round(zoomLevel * 100)}%
          </span>
        </div>
      </div>

      {/* IEEE Std 1044-2009 Root Cause Color Legend */}
      <div className="flex flex-wrap items-center gap-2 text-[11px] bg-slate-950/70 p-2.5
    rounded-lg border border-slate-800">
        <span className="font-semibold text-slate-400 mr-1">
          IEEE 1044-2009 Root Cause:
        </span>
        {Object.values(IeeeFaultType).map(function renderLegendChip(fault) {
          const hex = IEEE_FAULT_HEX_COLORS[fault] ?? 0x64748b;
          return (
            <span
              key={fault}
              className="flex items-center gap-1.5 px-2 py-0.5 rounded bg-slate-900 border
    border-slate-800"
            >
              <span
                className="w-2.5 h-2.5 rounded-full inline-block"
                style={{
                  backgroundColor: `#${hex.toString(16).padStart(6, '0')}`,
                }}
              />
              <span>{fault}</span>
            </span>
          );
        })}
      </div>

      {/* PixiJS v8 Viewport Host */}
      <div
        ref={hostRef}
        className="w-full overflow-hidden rounded-lg border border-slate-800 bg-slate-950"
      />

      {/* Hover Telemetry Tooltip */}
      {hoveredItem && (
        <div
          className="pointer-events-none absolute z-30 bg-slate-950/95 border
    border-amber-500/60 rounded-lg p-3 text-xs shadow-2xl max-w-xs space-y-1"
          style={{
            left: Math.min(CANVAS_WIDTH - 280, hoveredItem.screenX + 24),
            top: Math.max(80, hoveredItem.screenY + 40),
          }}
        >
          <div className="font-bold text-amber-400">
            #{hoveredItem.item.id} -- {hoveredItem.item.title}
          </div>
          <div className="text-slate-300">
            <strong>Product:</strong> {hoveredItem.item.product}
    ({hoveredItem.item.detected_environment || 'PROD'})
          </div>
          <div>
            <strong>IEEE 1044-2009 Root Cause:</strong>{' '}
            {hoveredItem.item.root_cause_type || hoveredItem.item.classification}
          </div>
          <div>
```

```
            <strong>Symptom:</strong> {hoveredItem.item.symptom_category || '--'}
          </div>
          <div>
            <strong>Severity / Priority:</strong> {hoveredItem.item.severity} /{' '}
            {hoveredItem.item.priority}
            {Boolean(hoveredItem.item.is_regression) ? ' * [REGRESSION]' : ''}
          </div>
          <div>
            <strong>Actual Effort:</strong>{' '}
            {Number(hoveredItem.item.actual_effort_hours ?? hoveredItem.item.fixeffort ??
    0).toFixed(2)}h
          </div>
          <div>
            <strong>Triage Lead / MTTR:</strong>{' '}
            {hoveredItem.item.triage_lead_time_hours !== null &&
    hoveredItem.item.triage_lead_time_hours !== undefined
              ? `${hoveredItem.item.triage_lead_time_hours}h`
              : '--'}{' '}
            /{' '}
            {hoveredItem.item.mttr_hours !== null && hoveredItem.item.mttr_hours !==
    undefined
              ? `${hoveredItem.item.mttr_hours}h`
              : 'open'}
          </div>
          {(hoveredItem.item.introducing_ref || hoveredItem.item.resolution_ref) && (
            <div className="font-mono text-[10px] text-emerald-300">
              {hoveredItem.item.introducing_ref ? `Intro:
    ${hoveredItem.item.introducing_ref} ` : ''}
              {hoveredItem.item.resolution_ref ? `Fix: ${hoveredItem.item.resolution_ref}` :
    ''}
            </div>
          )}
          <div className="text-[10px] text-amber-300 pt-1">
            Click node to open Telemetry Workbench * Drag across lanes to update status
          </div>
        </div>
      )}
    </div>
  );
}
```